



《需求工程：软件建模与分析》

第十一章：面向对象分析

李传艺

南京大学软件学院

2025/5/29

目录

- 面向对象分析与UML

面向对象方法的基本思路
UML与面向对象分析

- 对象模型

- 建立领域模型

- 行为模型--交互图

- 建立交互图

- 行为模型--状态图

- 建立状态图

- 对象约束语言OCL

- 基于CRC卡面向对象分析方法

- 需求细化

面向对象方法的基本思路

- 面向对象分析采用面向对象方法学的世界观，将系统看作是一系列对象的集合。每个对象具有独立的职责，完成独立的任务，对象之间通过消息机制互相协作，共同实现系统的目标。

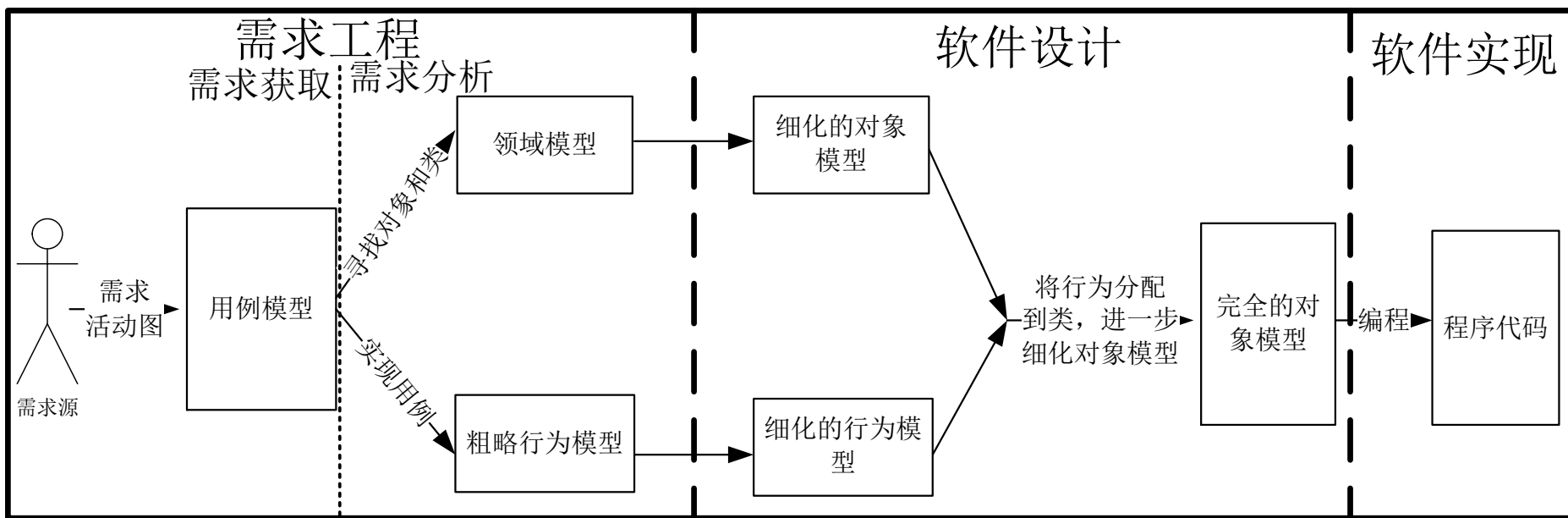
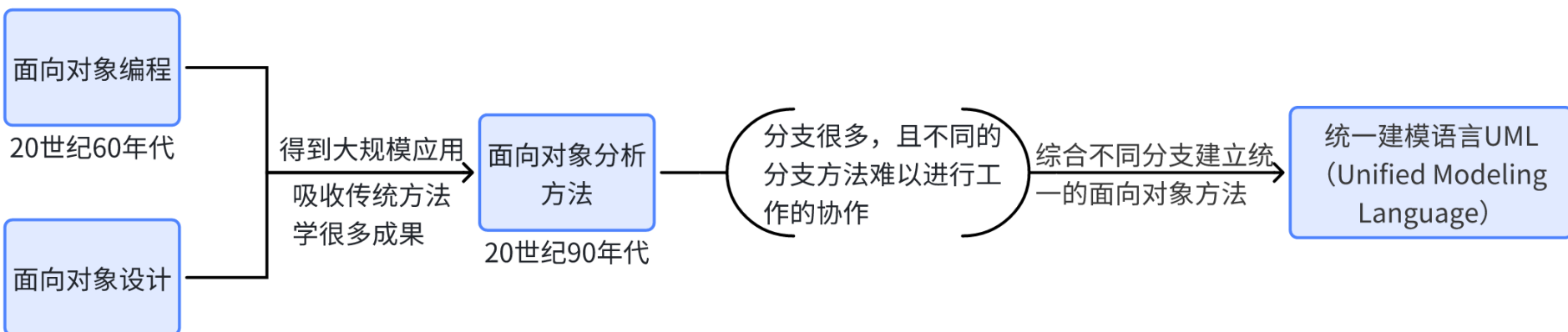


图11-1 面向对象方法的技术路线

UML 与面向对象分析

➤UML的诞生



➤UML后来被OMG采纳作为面向对象建模的**标准方法**，得到了广泛的认同。

虽然被称为统一建模语言，但UML其实是**很多种技术的综合体**。这些技术在一个**统一的框架**下，能够很好地实现相互之间的协同，共同构成完整的面向对象开发方法。

在需求分析中涉及的UML技术有：

- 用例图（用例模型）；
- 类图（对象模型）；
- 交互图（顺序图/通信图，行为模型-对象协作）；
- 状态图（行为模型-状态机）；
- 活动图（行为模型-业务过程）；
- 对象约束语言 OCL。

前面章节已经介绍了活动图和用例模型，所以本章主要描述其他几个UML模型。

目录

- 面向对象分析与UML

- 对象模型

- 建立领域模型

- 行为模型——交互图

- 建立交互图

- 行为模型——状态图

- 建立状态图

- 对象约束语言OCL

- 基于CRC卡面向对象分析方法

- 需求细化

对象

对象之间的关系

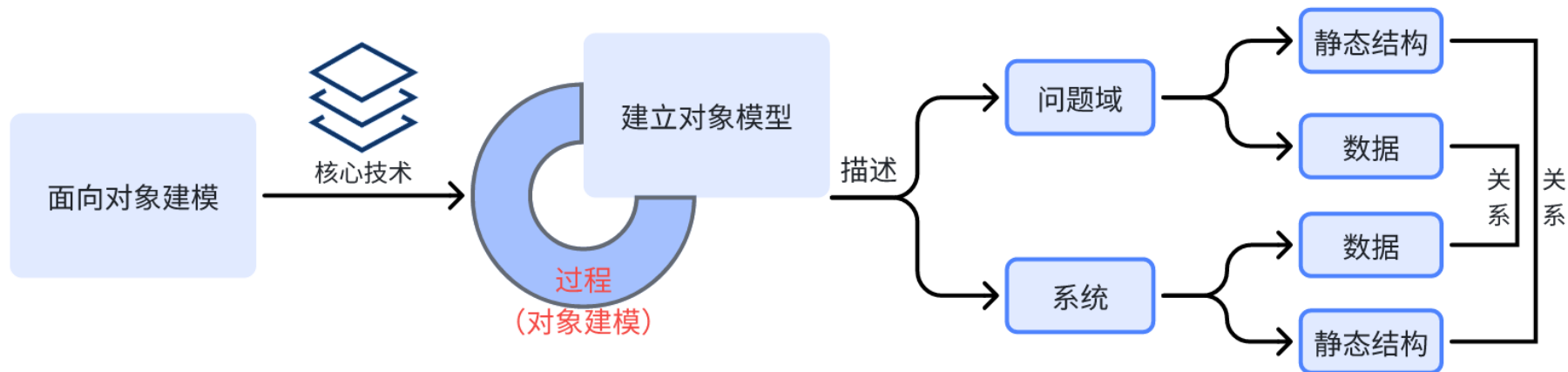
类

类之间的关系

领域模型

对象模型

➤对象模型以对象和类的概念为**基础**，描述了系统中的对象和对象**之间的关系**。



对象建模

对象

一、什么是对象？

对象

➤ **对象的概念**是对象模型的**基础**。从理解现实世界的角度来看，它是对现实世界事物的**抽象**，也就是说对象代表了现实世界的事物。



➤ 在软件设计当中可能会出现一些在现实世界中**没有**对应事物的对象。但是在**需求分析**当中所有对象都应该代表**相应的现实世界事物**。



例如，视窗、按钮、算法、连接、数据库、网络等**软件设计**中常见的对象类型是**不可以**出现在需求分析中的。

➤对象描述的是一种比较常见的存在，很多事物都可以被模型化为对象。常见的有：

和系统存在交互的外部实体，例如人、设备、其他的软件系统等；

问题域中存在的事物，例如报表、信息展示、信号等；

在系统的上下文环境中发生的事件，例如一次外部控制行为、一次资源变化等；

人们在与系统的交互之中所扮演的角色，例如系统管理人员、用户管理人员、普通用户等；

和应用相关的组织单位，例如分公司、部门、团队、小组等；

问题域中问题发生的地点，例如车间、办公室等；

事物组合的结构关系，例如部分与整体的关系等。



对象

二、对象的定义



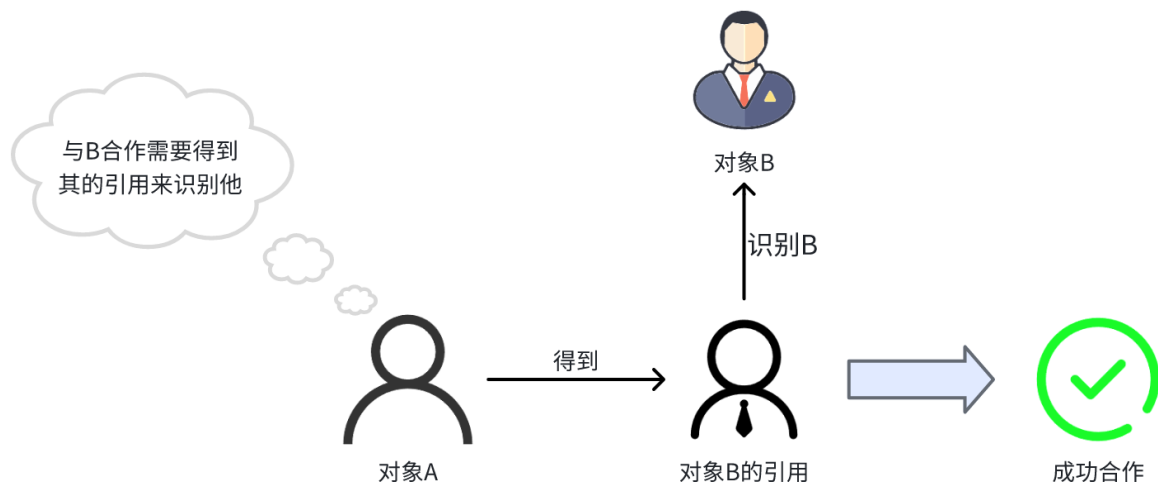
- 一个建议的对象定义为：对象是指在一个应用当中具有明确角色的独立可确认的实体。



- 在这个定义当中，强调了一个事物可以被抽象为对象的两个条件：**独立可确认**；**有明确的角色**。

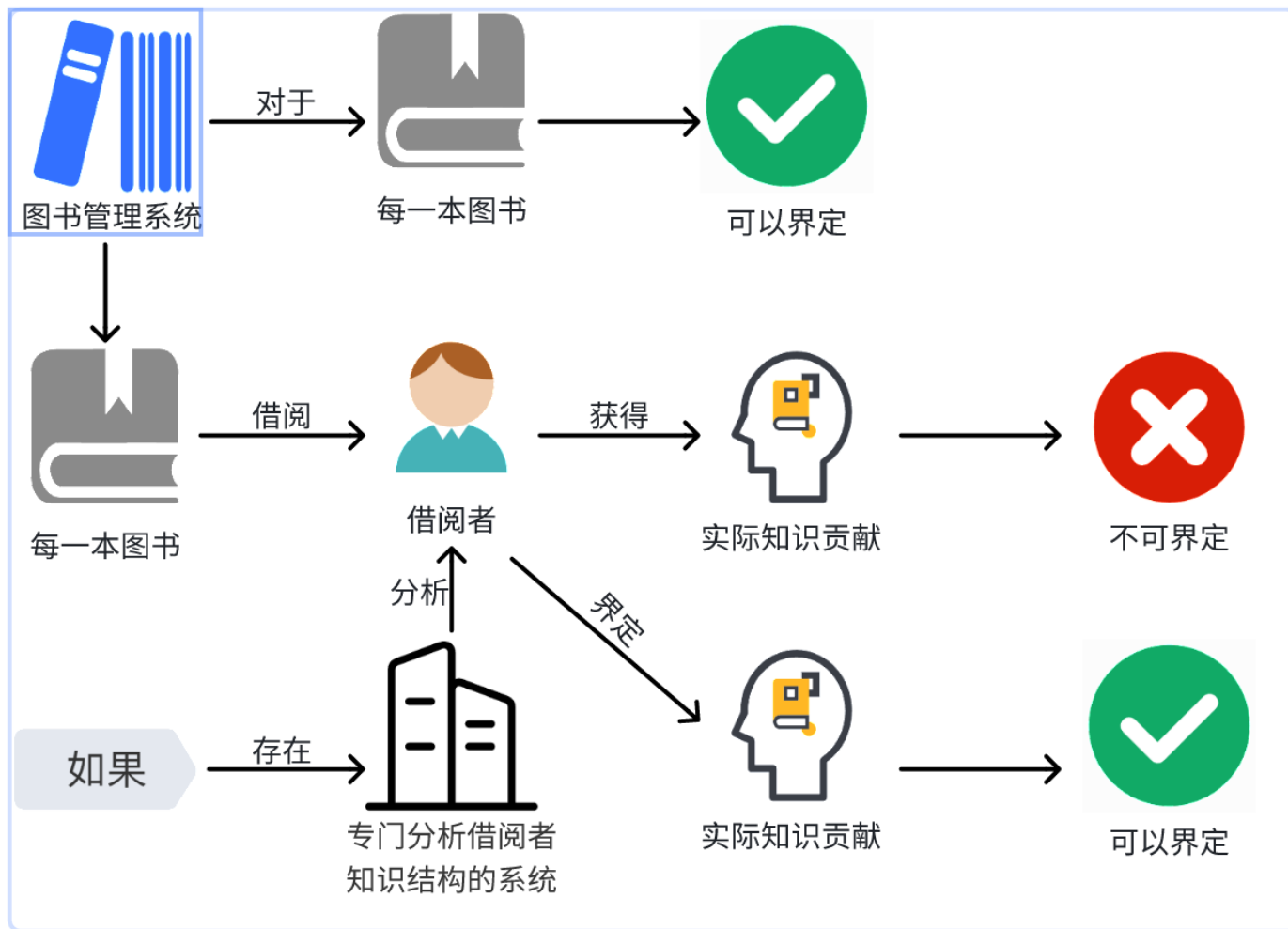
(1) 独立可确认

- 要求对象能够从周围的环境当中**界定自己**。
- 一个事物是否可界定是相对于环境的，是**相对于问题域的**。
- 界定的对象需要拥有一个**标识符**以**唯一**的标识自己。和实体关系模型中使用键来标识实体**不同**，对象模型使用**对象的引用**作为对象的标识。



图例：对象的引用

例：图书管理系统中部分对象的界定

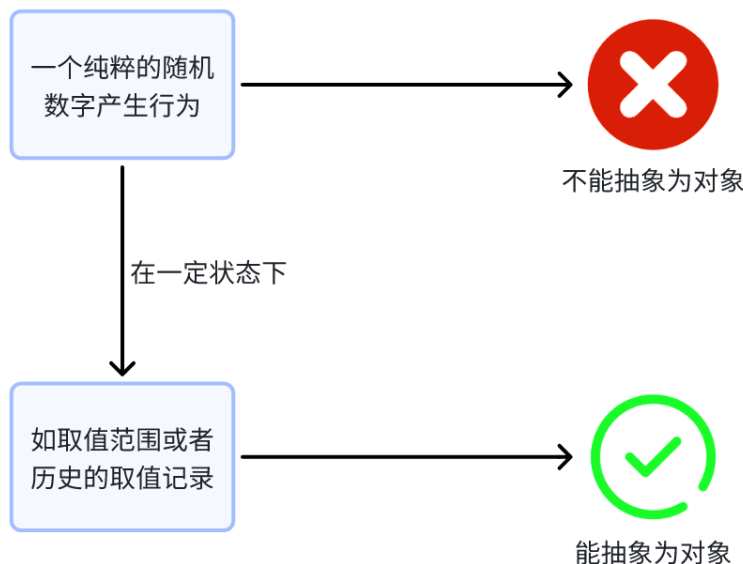


(2) 有明确角色

- 对象角色 (Role) 被认为是对象职责 (Responsibility) 的体现, 职责是指**对象持有、维护特定知识并基于知识行使固定职能的能力**。
- 因此, 要求对象有明确的角色就是要求对象在应用中维护一定的知识和行使固定的职能, 简单的说就是要拥有**状态和行为**。

➤ 状态

- 对象的**特征描述**，包括对象的**属性**和**属性的取值**。
- 对象的**行为**通常是**依赖于状态**的，如果状态发生了变化，那么对象的行为往往也会随之变化。
- 在需求分析当中，没有状态的事物是不能被抽象为对象的，例如：



没有状态的事物无法抽象为对象

➤ 行为

- 对象在其状态发生改变或者接收到外界消息时**所采取的行动**。
- 对象的行为是基于其状态的，而其状态又是历史行为的累积，所以对象的多个行为之间往往具有**相关性**。
- 如果对象的所有行为都互有相关性，那么可以认为它拥有一个**内聚**的状态和行为，此时称该对象具有**单一职责**，是理想的抽象结果。



现实世界当中**只有状态没有行为**的事物**不能**被抽象为对象，例如蓝色、5M等原子属性及其取值。

现在，再来重新认定对象角色的含义

- 一个对象**维护其自身的状态**需要对外公开一些方法，**行使其职能**也要对外公开一些方法，这些方法组合起来定义了该对象**允许外界访问的方法**，或者说限定了外界可以期望的表现，它们是对象需要对外界履行的**协议 (Protocol)**。
- 一个对象的整体协议可能会分为多个**内聚的逻辑行为组**，划分后的每一个逻辑行为组就描述了对对象的一个独立**职责**，体现了对象的一个**独立角色**。
- 如果一个对象拥有多个行为组，就意味着该对象拥有多个不同的职责，需要扮演多个不同的角色（例如下图的学生对象）。
- **理想的单一职责对象应该仅仅扮演一个角色。**



对象之间的关系

- 对象之间互相协作的关系称为**链接 (Link)**，它描述了对象之间的**物理**或**业务联系**。



- 链接通常是**单向**的，当然也有**双向**的链接存在。
 - 如果一个对象a存在指向b的链接，那就意味着a可以依赖b的对外协议拥有对b的**假设**，关于b的行为和行为效果的假设。也就是说，b需要满足a的某些**行为期望**。

- 由a指向b的链接还意味着a能够在**链接的指引**下，正确的找到并将消息发送给b。这种情况下，b是对a**可见**的，或者说a拥有b的**可见性 (Visibility)**。
 - 如果一个对象a拥有了对象b的可见性，那么a就可以按照b的协议发送消息，请求b的服务。

类

一、类的概念



➤ 类是共享**相同属性**和**行为**的对象的**集合**，它为属于该类的**所有对象**提供统一的**抽象描述**和**生成模板**。

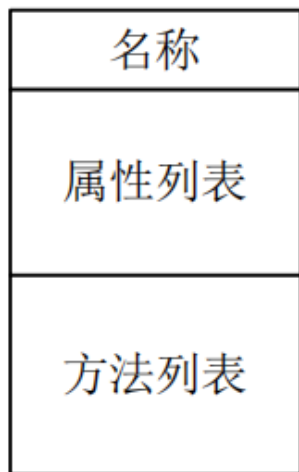


- 抽象描述称为**接口 (Interface)**，定义了类所含对象对外的（其他类和对象）的统一协议。
- 生成模板称为**实现 (Implementation)**，说明了类所含对象的生成机制和行为模式。

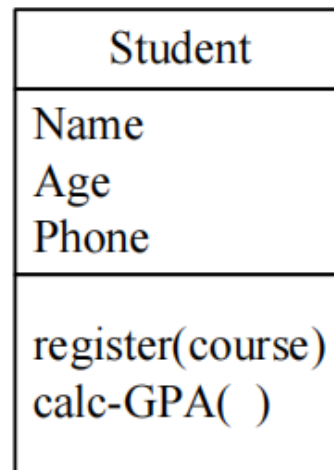


➤ 类就是将对象进行**归类**和**抽象**的结果。

- 每个类都有能够**唯一**标识自己的名称，同时包含有**属性**和**行为方法**。
- 类的UML的表示如图11-2（a）所示，图11-2（b）展示了一个类Student的UML图示。
- 在需求分析中，**不关注**类的**方法**，只描述**重要的属性**。



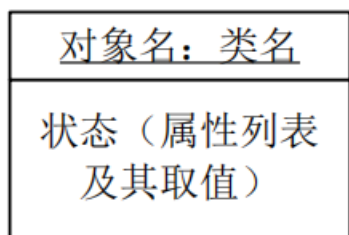
(a)



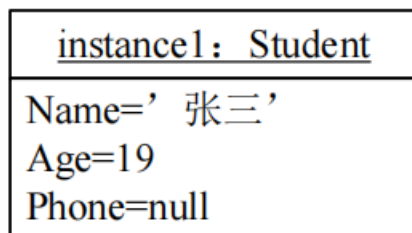
(b)

图11-2 类的UML表示

- 对象是类的**实例**，一个Student对象的UML图示如图11-3所示。
- 因为对象的行为都是和类的方法声明**保持一致**的，所以在对象的图示当中**没有方法列表**。



(a)



(b)

图11-3 对象的UML图示

二、类的产生——分类

➤ “类”的概念体现了它的含义，它是众多对象分类后产生的类别。因此，分类方法的不同自然会导致所产生的结果类不同。目前常见的类产生方法有 **数据驱动（Data-Driven）** 和 **职责驱动（Responsibility-Driven）** 两种。

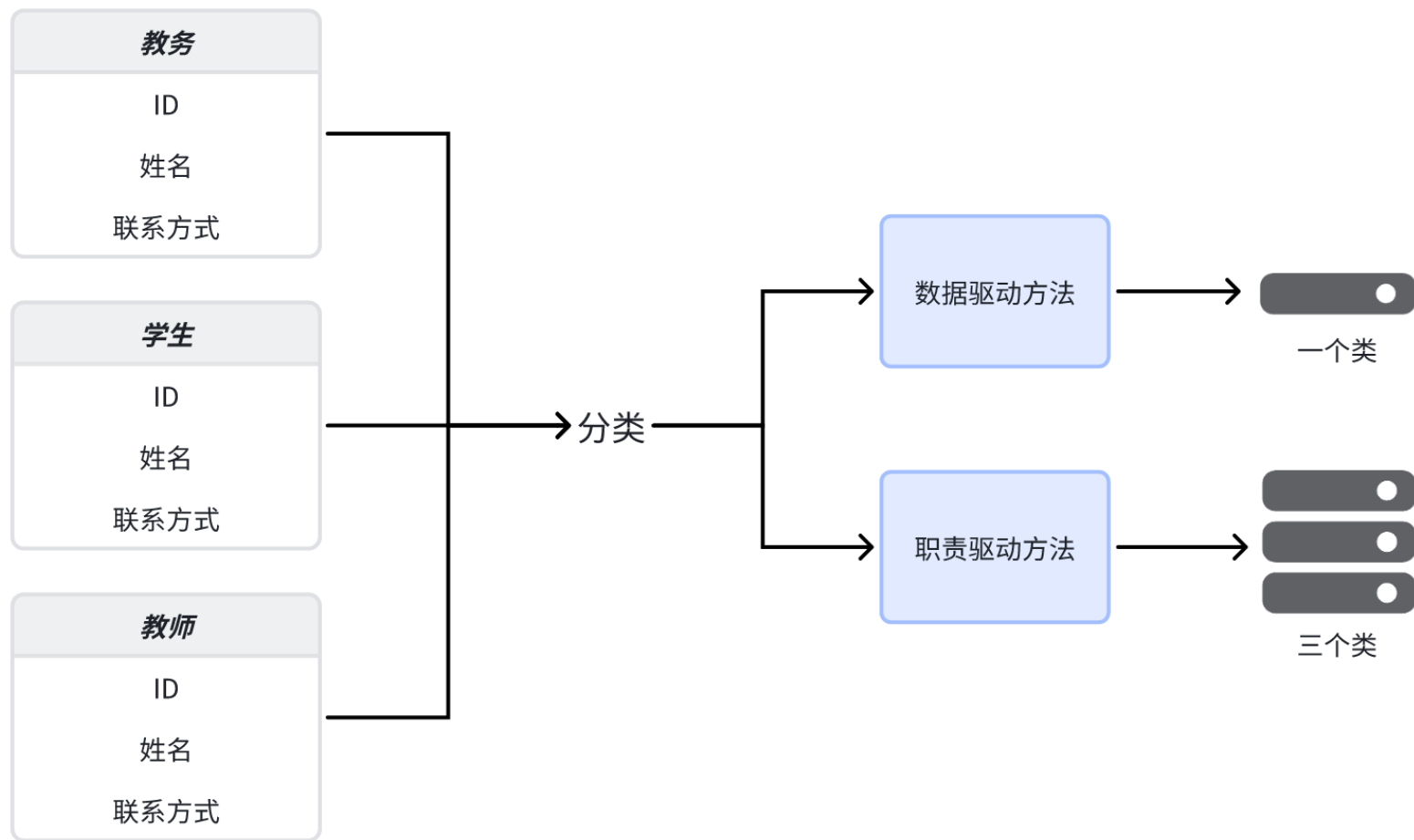
➤ 数据驱动（Data-Driven）

- 将具有**相同属性**的对象归为一类
- 产生自哲学上传统的**经典分类理论（Classical Categorization Theory）**
 - ▣ 所有**具有一个给定特性或共同特性集**的实体组成一个类

➤ 职责驱动（Responsibility-Driven）

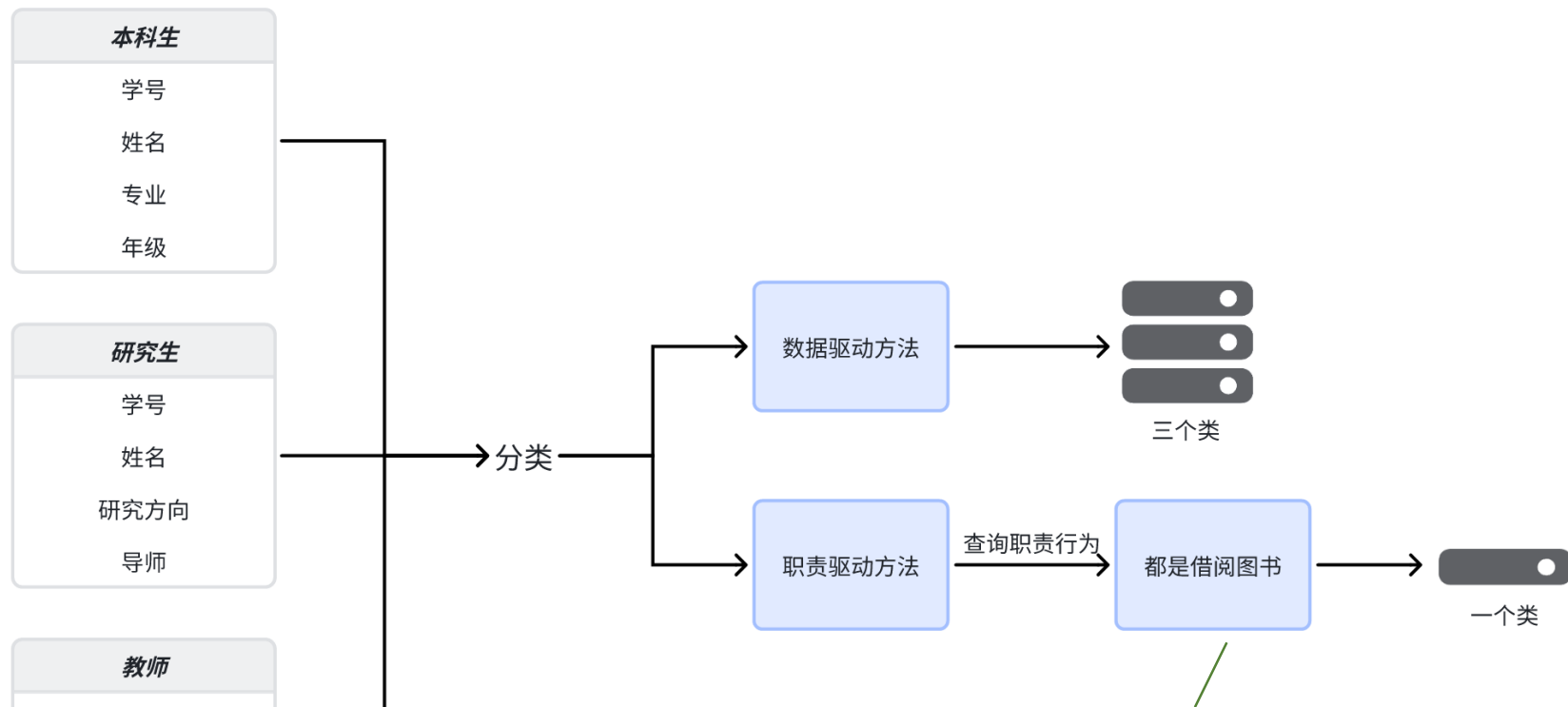
- 会依据事物的**相似性**而不是完全的相同性来进行事物的分类
- 产生自哲学上的**概念聚类（Conceptual Clustering）**
 - ▣ 使用**概念描述**而不是指定的特征来描述类别和事物，在进行事物分类时它会考虑概念之间的相似性，并将事物归入和其概念最为相似的类别

比较示例



示例一 选课系统下的方法对比

比较示例



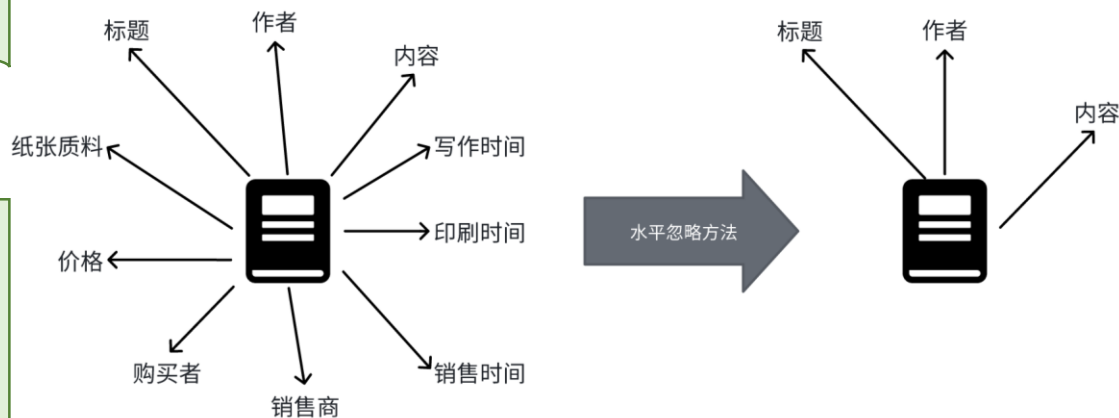
示例二 图书管理系统下的方法对比

三、类的产生——抽象

- 抽象是指在事物的众多特征当中只注意那些和目标**密切相关的特征**，同时忽略那些不相关的特征，进而找出事物的**本质**和**共性**。
- **抽象 (Abstract) 原则**是从众多的对象当中归纳出共同的类描述的过程依据。
- 抽象是人们在理解事物时常用的手段。人们运用抽象原则，通过以特定的目标和要求为导向，**忽略**不重要的特征，使得人们摆脱具体事物的细节困扰，将面对的问题**简单化**。

忽略特征法一：水平层次上的忽略

在**水平层次**上忽略一些特征，多用于对复杂事物的简化。





忽略特征法二：垂直层次上的忽略

在特征的**垂直表达层次**上忽略底层的细节，在一个更高的层次上分析事物，多用于归纳众多事物的**共性**。

现实世界的事物

抽象

对象

对象就是对**现实世界事物**的抽象结果，它表达了系统所需要的现实世界事物特征，抛弃了那些系统不需要的特征。

对象集

抽象

类

类是**对象集**的抽象结果，它忽略了具体某个对象在特定时间和空间的细节状态，从对象集的全局出发，在一个更高的**逻辑层次**上，描述了对**对象集**的**共性**。

➤ 后面将要介绍的类的继承关系也是一种抽象，在类集合中寻找共性的抽象。

四、类的封装

- 类并不是简单的将属性和行为放置在一起，它还需要进行信息的**隐藏 (Information Hiding)**，又称**封装 (Encapsulation)**。
- 封装是指尽可能隐藏构造单位**内部的实现细节**，只通过有限的对外接口保持对外联系的一种软件构造策略。



类的封装要求类只对外公开其对象为履行职责所必需的**协议 (Protocol)**，其他的实现细节都要隐藏起来，包括不允许外界直接访问的**属性**和**内部使用的局部方法**。

- 类只通过封装后的协议与外界交互，所以类之间是**平等的**，不存在一个类控制其他类的现象。
- 如果一个类需要使用其他类的功能，那么就按照对方的协议，向其发送**消息请求**，并从对方的**响应消息**中得到结果。

类之间的关系

一、关联

- 类之间的关系被称为**关联（Association）**，它指出了类之间的某种**语义联系**。
- 关联是类对其对象实例之间的无数潜在**关系的描述**，对象实例依据关联所带有的信息进行链接的建立和撤销。

类之间的关系有着多种特征，如图11-4所示：

- （1）为关联赋予名称，表示关系的**语义内涵**；
- （2）将参与关联的类的数量表示为**度数**，用来度量关联的复杂度，常见的有一元关联、二元关联和N元关联；
- （3）使用最大基数和最小基数来明确一个类在关联中可能参与的实例数量；
- （4）如果一个关联同时具有了**独立的状态和行为**，那么这个关联也可以被看作是类，被称为**关联类**。



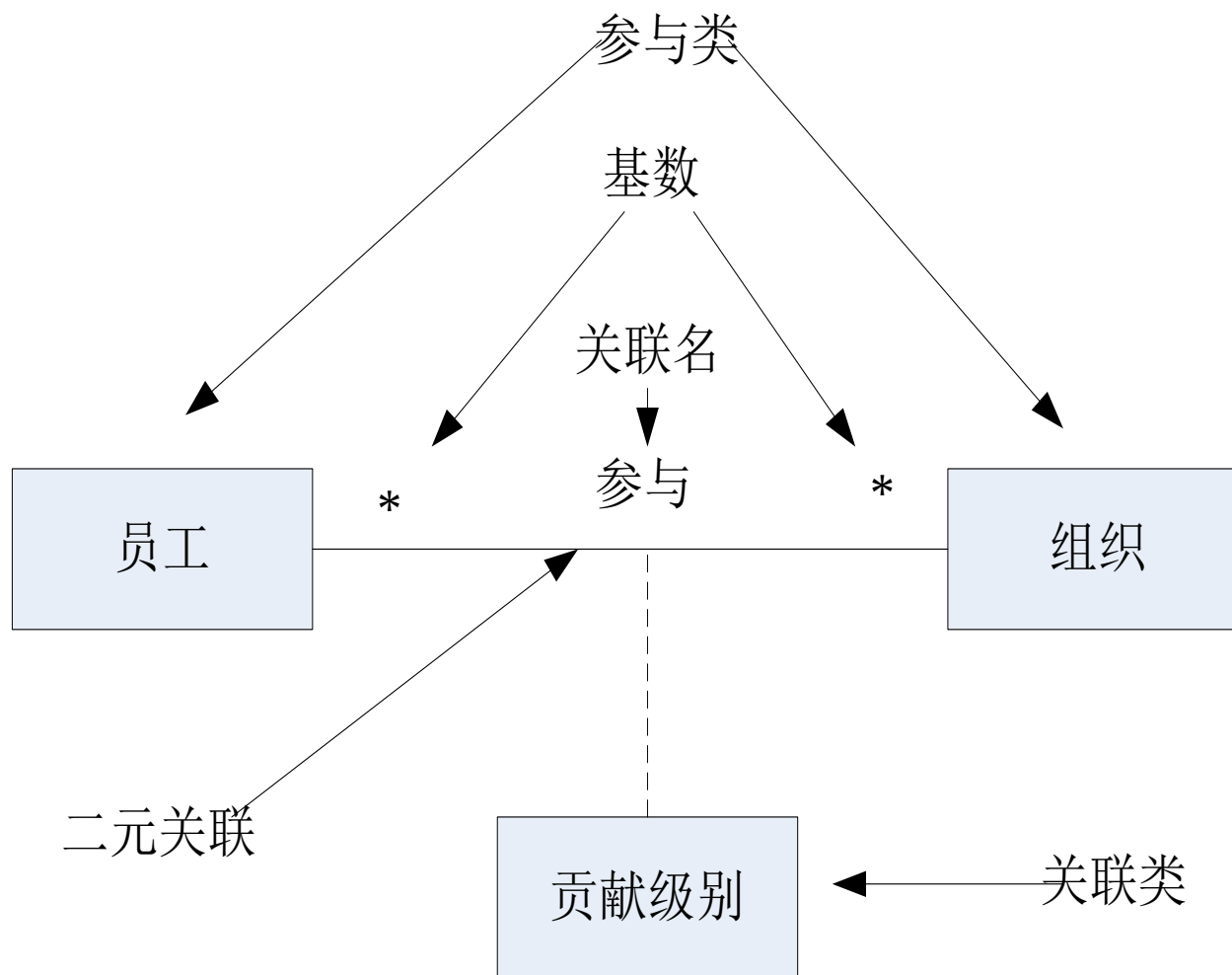


图11-4 关联的描述

除了上述的特性之外，对象模型还描述了关联的**其他特性**（如图11-5所示）：

(1) **角色 (Role)**。在类参与关联关系时，依据类在关系中扮演的角色为**关联端 (Association End)** 赋予角色。关联端是类与关联的**连接点**，它定义了关联中类的参与行为。关联端的角色是可选的，可以标记出来，也可以不予指定。

(2) **可见性 (Visibility)**。除了角色之外，在关联端上还可以标记**可见性信息**，用来说明某个关联端的类是否对关联中的其他类可见，也就是用来说明其他类的对象实例能否通过关联的链接实例访问关联端类的对象实例。可见性也是可选的，可以标记出来，也可以不标记出来，而且通常很少被标记出来。

(3) **方向**。对象实例之间的链接是**单向**的，但是类之间的关联通常是**双向**的。不过也可以将关联标记为单向的，可以为关联赋予方向。在分析阶段，通常**不标记**关联的方向。



(4) **限定关联 (Qualified Association)**。在关联当中，一些类可以通过为关联端指定一些属性来将远端关联的类实例区分开来，这种关联被称为限定关联。

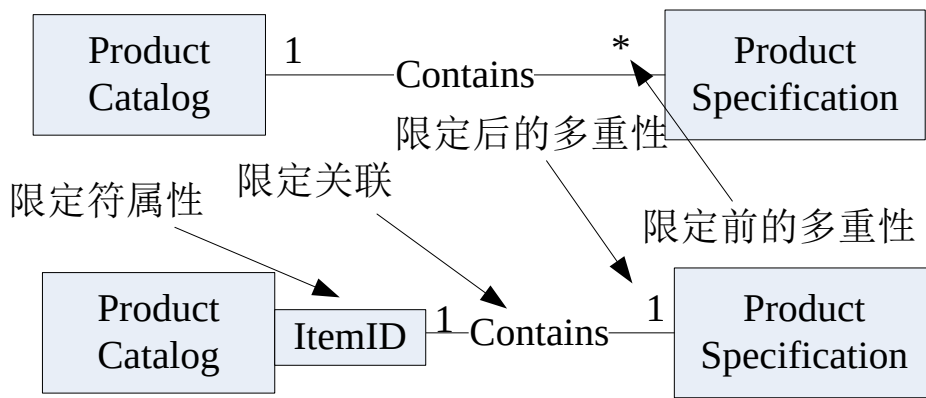
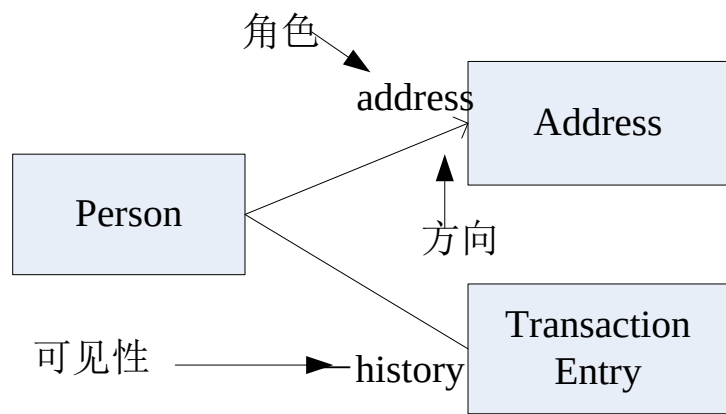


图11-5 关联的其他特性描述

二、聚合与组合

(1) 聚合

聚合 (Aggregation) 是类之间的一种特殊的关联，表示**部分与整体**之间的关系，如图11-6左边所示。

(2) 组合

如果整体除了包含部分之外，还对部分有完全的管理职责，即一旦一个部分属于某个整体，那么该部分就无法**同时属于其他整体**，也无法**单独存在**，则这种聚合关联被称为组合 (Composition)，如图11-6右边所示。

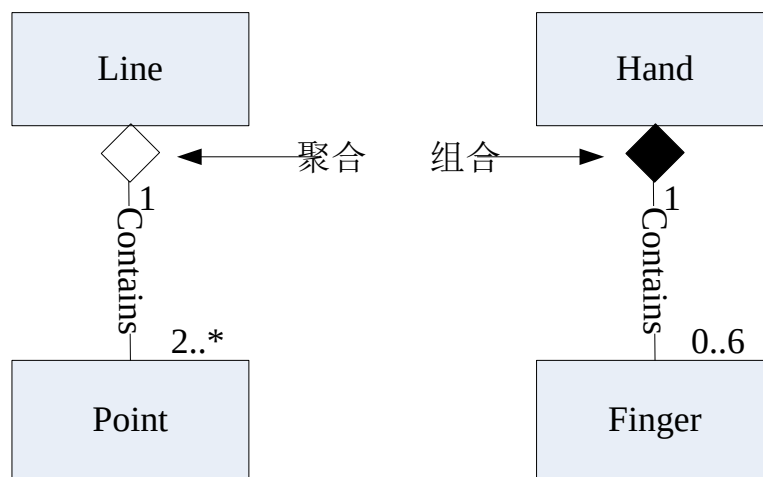


图11-6 聚合与组合示例

三、继承

- 类之间还有一种比较基本的关系，被称为**继承 (Inherit)**。
- 如果一个类B继承了类A，那么B就自然具有A的全部属性和服务，同时B也会拥有一些自己特有的属性和服务，这些特有部分是A所不具备的。其中，A被称为**父类 (或者超类)**，B被称为**子类**。
- 在继承关系当中，可以认为子类**特化**了父类，或者说父类是子类的**泛化**，所以继承关系又被称为**泛化 (Generalization) 关系**。

面向对象的继承概念**来源于人工智能AI (Artificial Intelligence) 领域**：

- 假设有下面两组条件Pc1和Pc2： $Pc1 = F1(.) \wedge F2(.)$ 、 $Pc2 = F1(.) \wedge F2(.) \wedge F3(.)$ ，则如果Pc1能够满足规则Q，即 $Pc1 \rightarrow Q$ ，那么Pc2就也能满足规则 $Pc2 \rightarrow Q$ 。Pc1和Pc2的关系就可以被认为是**继承关系**，Pc2继承了Pc1。



- ➡ 如果一个子类继承了父类，那么子类就应该拥有父类**所有**的元素，同时额外追加自己**特有**的元素。
- ➡ 继承关系中子类要能够完成父类的**工作**，履行父类的**职责**。
- ➡ 继承关系的图示如图11-7所示，它被描述为带有三角形箭头的实线。

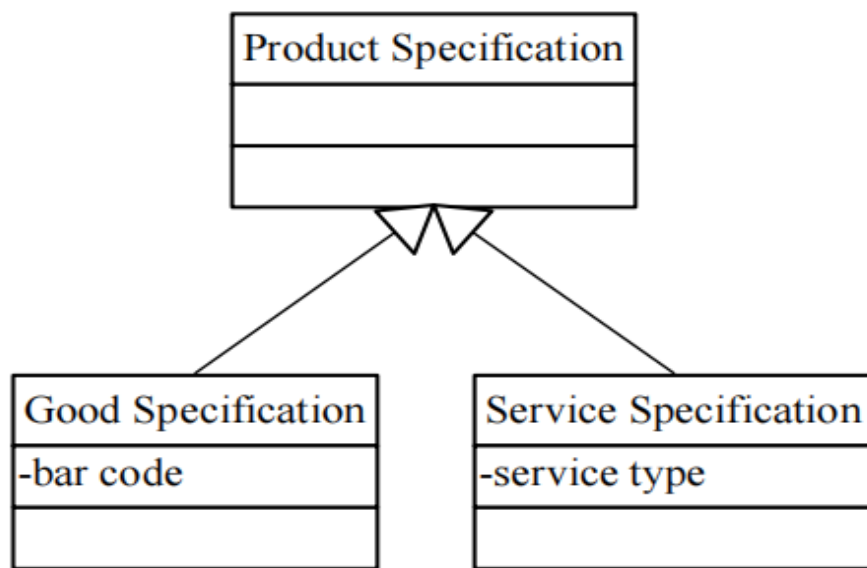


图11-7 继承关系图示

四、多态

多态 (Polymorphism) 是对象模型的一个重要概念。多态有**广义**和**狭义**之分。

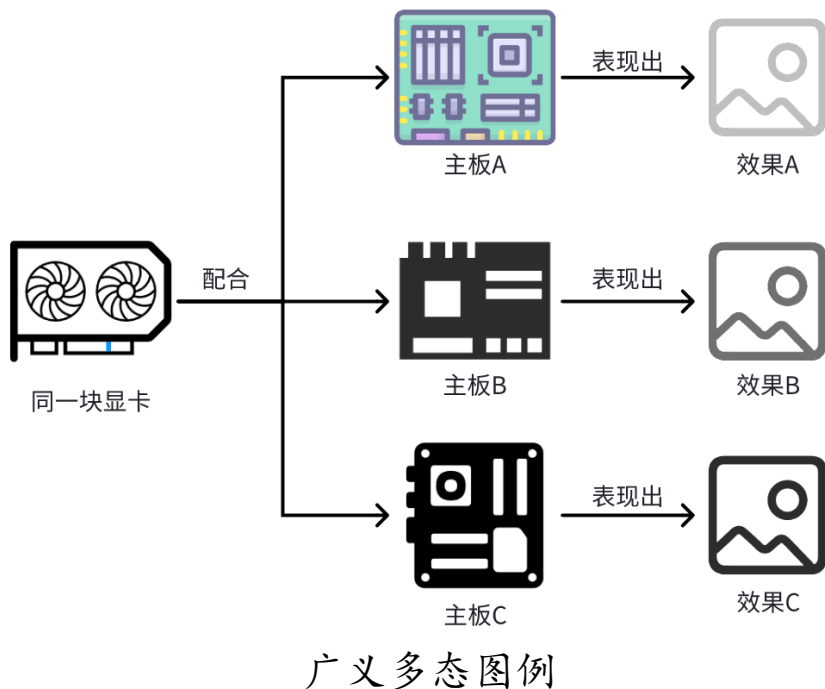
(1) 广义多态

- 一个对象在不同情景下表现出不同形态，或者不同对象在相同情景中表现出相同形态的现象
 - 同一对象在**不同情景**中表现出**不同形态**的现象的抽象
 - 重载 (Overloading)
 - 依据参数或返回值的不同为协议定义不同的**版本**，每一个版本都可以表现出自己**独特**的行为
 - 泛型 (Generality)
 - 使用同一个**通用**的实现处理**不同的数据类型**，然后根据数据类型的不同表现出不同的行为

(2) 狭义多态 (是广义多态的一种类型)

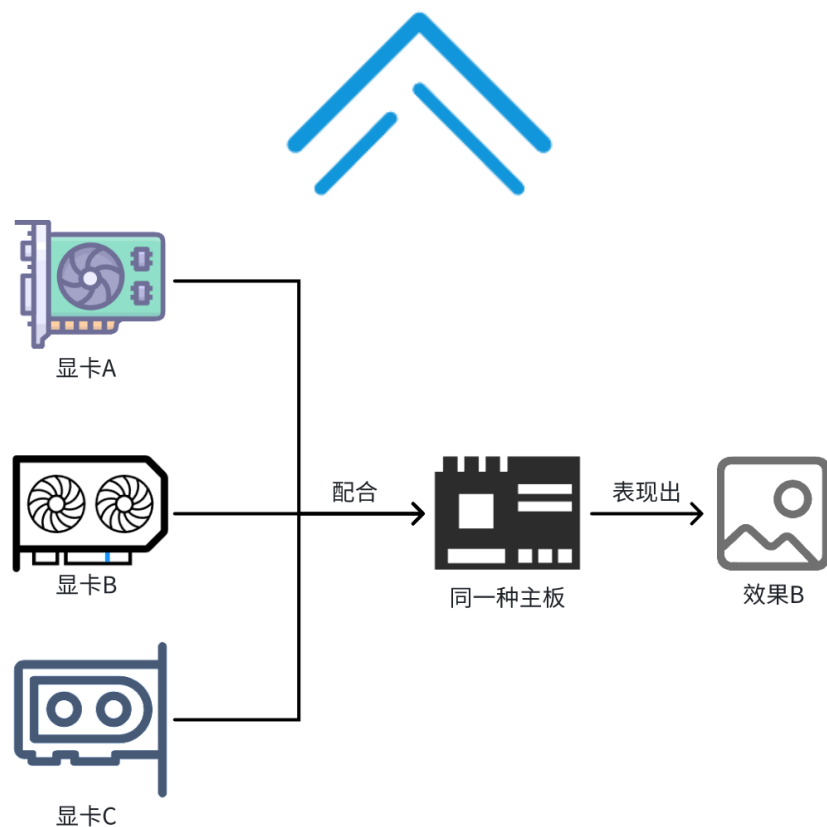
- 对多个对象在**同一情景**中表现出**相同形态**的现象的抽象。
- 面向对象方法使用**继承**作为狭义多态的实现方式。





同一块显卡，可以配合不同的主板配置，表现出不同的效果。

不同的显卡，可以在同一套机器配置中实现相同的显示效果。



领域模型

- 在进行**系统分析**时，开发人员关注的仅仅是**实际的业务范围**，分析阶段产生的对象模型是关注用户问题域的对象模型，它被称为**领域模型（Domain Model）**，又被称为领域类图（Domain Class Diagram）、概念类图（Concept Class Diagram）或分析类图（Analysis Class Diagram）。
- 类大多是**概念类（Concept Class）**，是一个能够**代表现实世界事物**的概念
- 概念类之间存在**指明语义联系**的关联，这些关联通常不标记方向，也不标记关联端的可见性
- 概念类会显式的描述自己的一些**重要属性**，但不是全部の詳細属性，而且概念类的属性通常没有类型的约束
- 概念类不显式的标记类的行为，即概念类**不包含**明确的**方法**



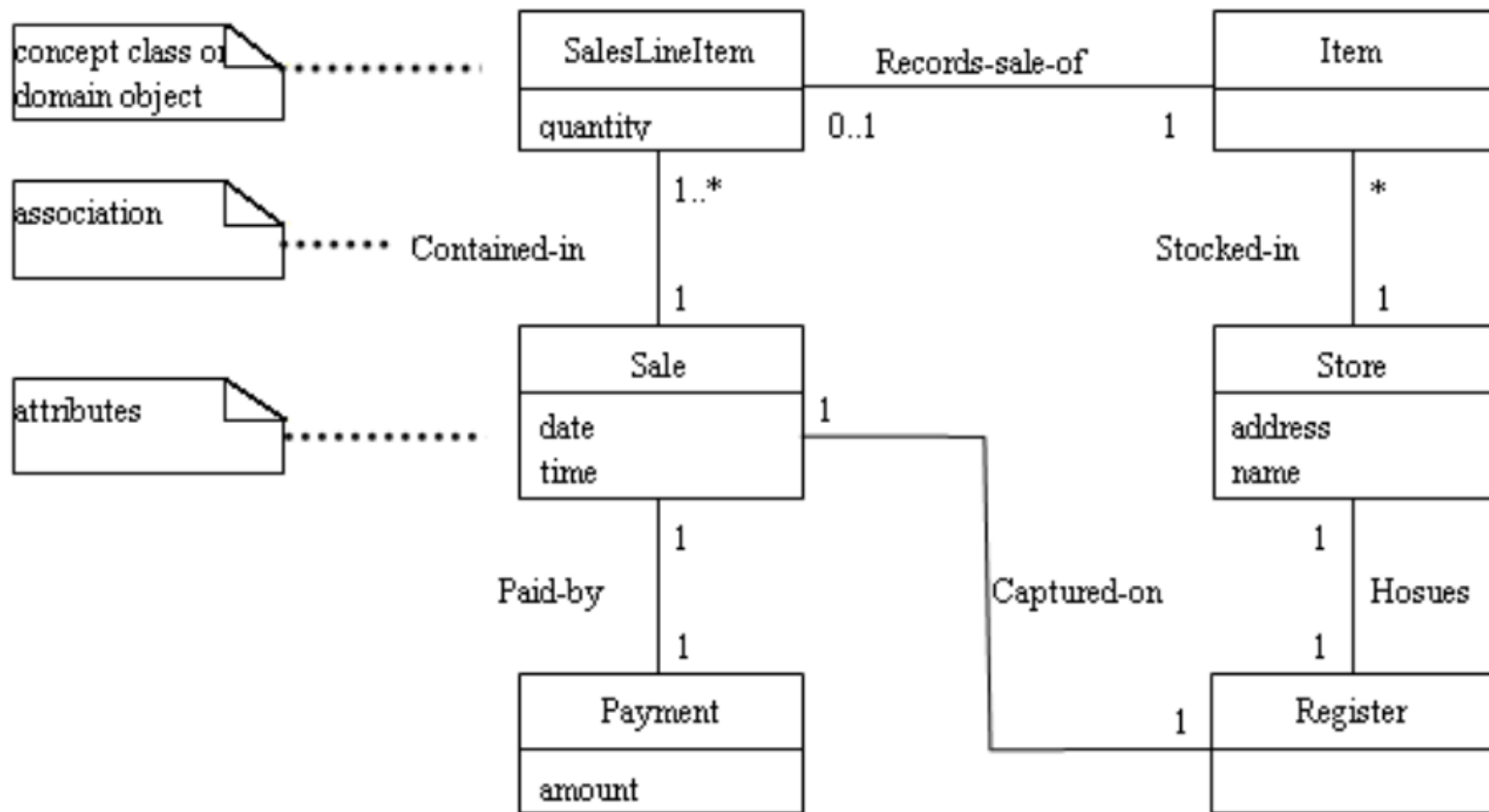


图11-8 领域模型示例

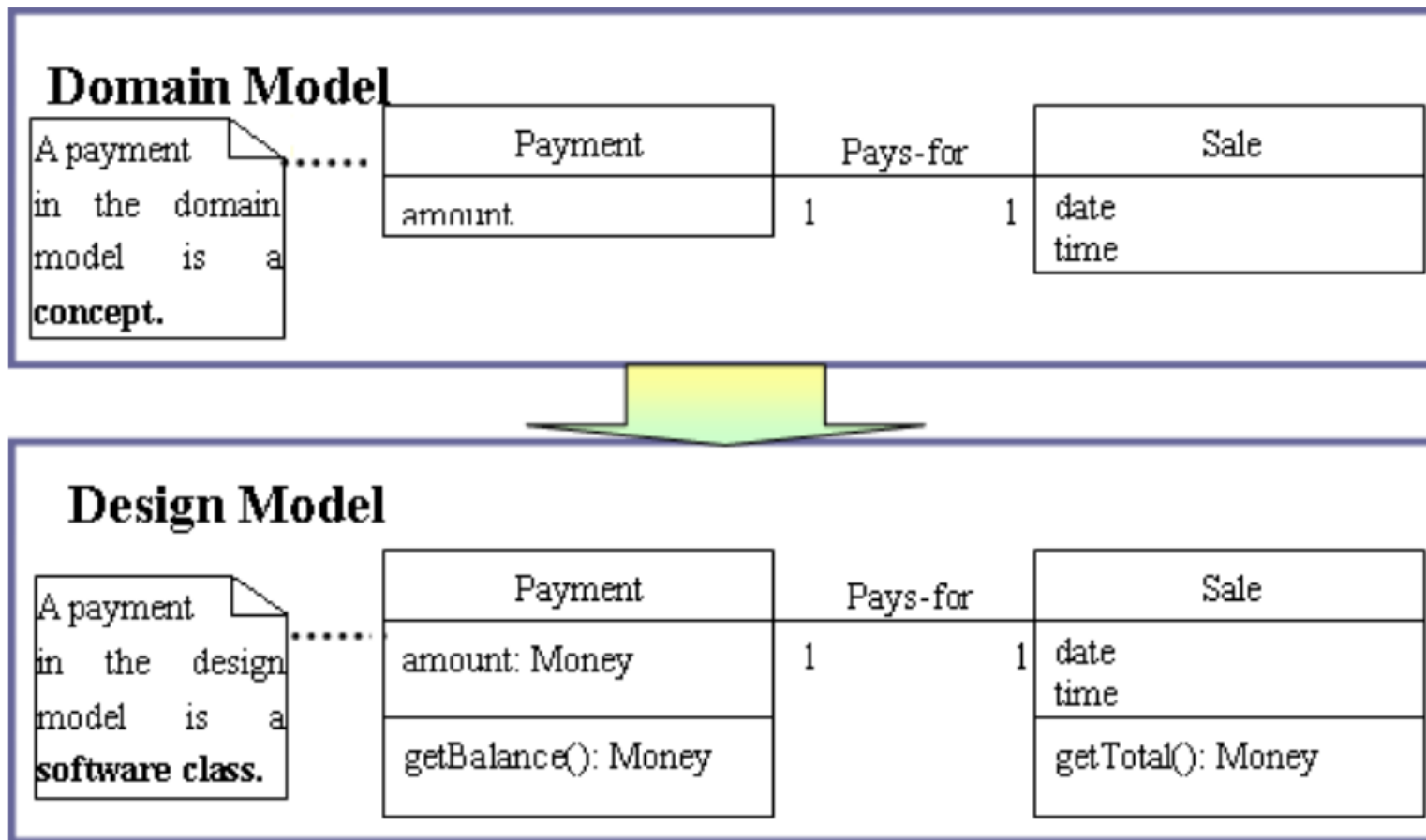


图11-9 领域模型和完整类图的比较示例

目录

- 面向对象分析与UML

- 对象模型

- 建立领域模型

- 行为模型——交互图

- 建立交互图

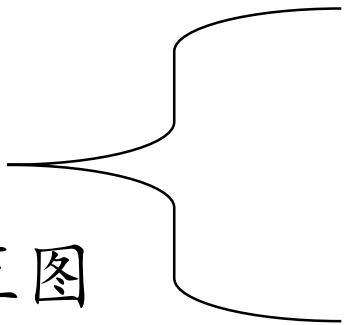
- 行为模型——状态图

- 建立状态图

- 对象约束语言OCL

- 基于CRC卡面向对象分析方法

- 需求细化



识别候选对象与类
确定概念类
建立类之间的关联
添加类的重要属性
领域模型的分析作用

建立领域模型

➤领域模型的建立过程如图11-10所示，包括四个步骤。



图11-10 领域模型建立过程

识别候选对象与类

- 面向对象分析方法认为对象与类是**本来就存在**于应用领域之中的，分析人员的任务是找到已经存在的对象与类，所以是**“识别”**而非**“建立”**。

识别



建立



- 识别候选对象与类的基本思路是**分析**应用领域的**描述信息**，从中发现可能的对象与类。发现对象和类时注意使用**概念类分类列表**和**名词分析法**。

一、概念类分类列表方法

- 这种方法事先给出一个概念类的**分类列表**，然后由分析人员在需求信息当中寻找**相应类别的候选对象**，最后对候选对象进行确定和归纳，形成概念类。
- 一些常见的概念类分类方式包括：角色、地点、事物、组织结构、概念、需要被记录的事件等。
- 例如，图11-11 a)是一个CD商店网上购物系统的购物用例的简单描述。根据概念分类列表，可以发现候选对象如图11-11 b)所示

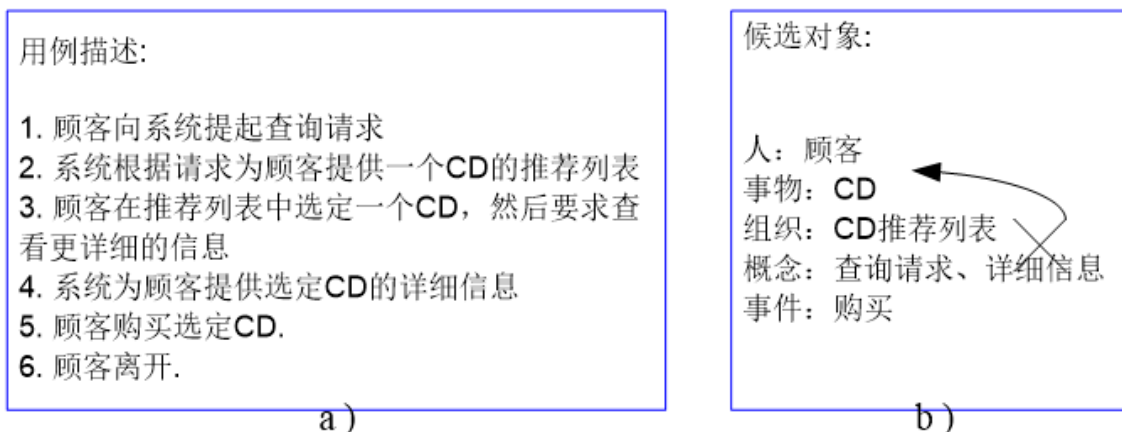


图11-11 利用概念类分类列表发现对象和类的示例

二、名词分析方法

➤ 名词分析是一种运用**语言分析**的实用方法。名词分析从**文本描述**中识别出有关的名词和名词短语，然后将它们作为候选的对象，最后对候选对象进行**确定**和**归纳**，形成概念类。

➤ 例如，一个商店销售处理的用例简单描述如图11-12所示。

用例描述：

1. 顾客携带商品到销售终端POS前.
2. 收银员开始一个新的销售处理.
3. 收银员输入物品项标识.
4. 系统记录销售的物品项列表并且显示物品描述、价格和总价.
收银员重复步骤3-4，直至输入所有物品项.
5. 系统显示最后的总价.
6. 收银员告诉顾客总价，要求顾客支付账款.
7. 顾客付款，系统结账.
8. 系统记录整个销售处理，更新产品库存目录.
9. 系统打印收据.
10. 顾客离开.

识别其中的名词和名词短语，使用**下划线**进行标记

识别时，**重复**出现的名词和名词短语只需要**标记一次**即可

图11-12 利用名词分析发现对象和类

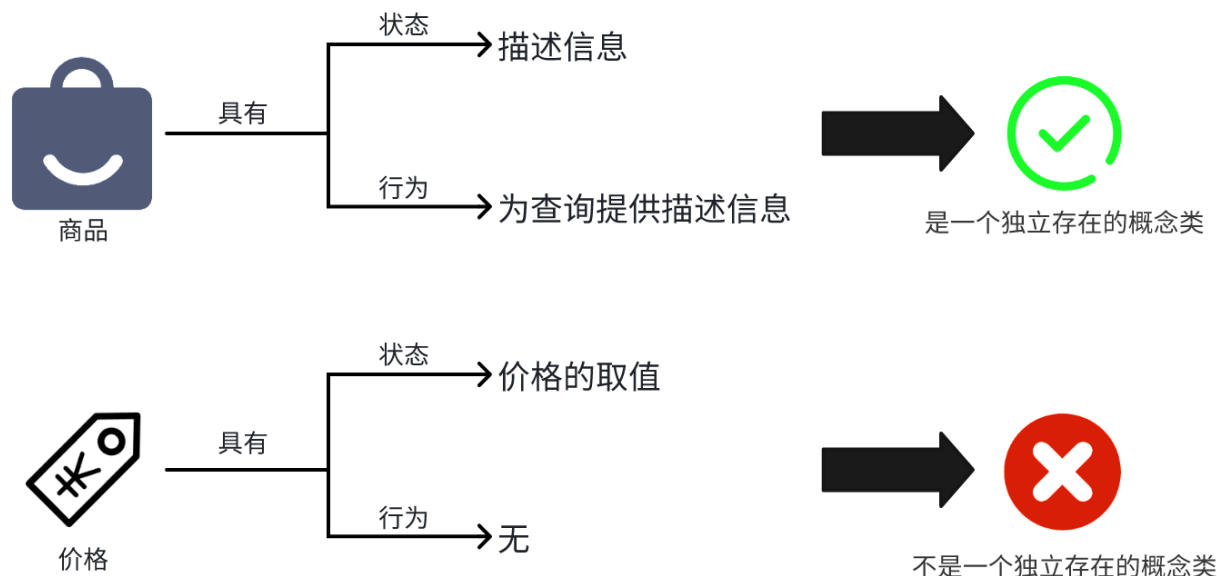
的示例

确定概念类

一、确定概念类的准则

➤ 选定了候选类之后，判断其是否应该作为一个概念类存在的标准就是在应用背景（尤其是需求要求）下考察候选类的内涵，看其是否**同时具有状态和行为特征**：

➤ 如果候选类既维持一定的**状态**，又依据状态表现一定的**行为**，那么它就应该是一个独立存在的概念类。（如下图中的商品，而价格便不是）

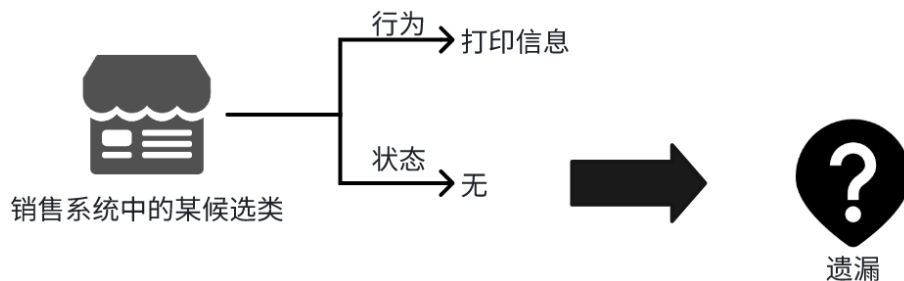


确定候选类图例

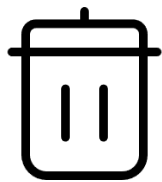
➤ 如果候选类**只有状态没有行为**，那么就要分析它的状态是否是系统需要的数据。

- 如果系统**需要**它的状态数据，那么该候选类就应该作为**其他类的属性**出现在最终的领域模型当中。（如上图中的“价格”就属于只有状态没有行为的候选类，系统需要它的状态数据，所以它最终会被抽象为“商品”类的一个属性。）
- 否则，该候选类应该被**摒弃**。

➤如果候选类**只有行为没有状态**，那么往往意味着需求信息的**遗漏**，因为行为的表现总是要以状态为**基础**的。此时就需要**重新进行需求的获取**。



候选类只有行为没有状态图例



➤既没有状态也没有行为的候选类与系统没有关系，会被摒弃。

二、避免实体关系建模思想的误区

- 在判断一个候选类是否应该成为独立类时一定要紧密结合**类的含义**。尤其是要避免实体关系建模思想所带来的误区。

(1) “二维”形式与复杂属性

- 误区之一是属性的复杂度问题。在进行实体关系建模时，因为实体往往需要以二维关系表的形式表达出来，因此人们就会使用“**二维**”形式来限制实体的属性。
- 在面向对象分析当中，“二维”的限制就**不存在**了。

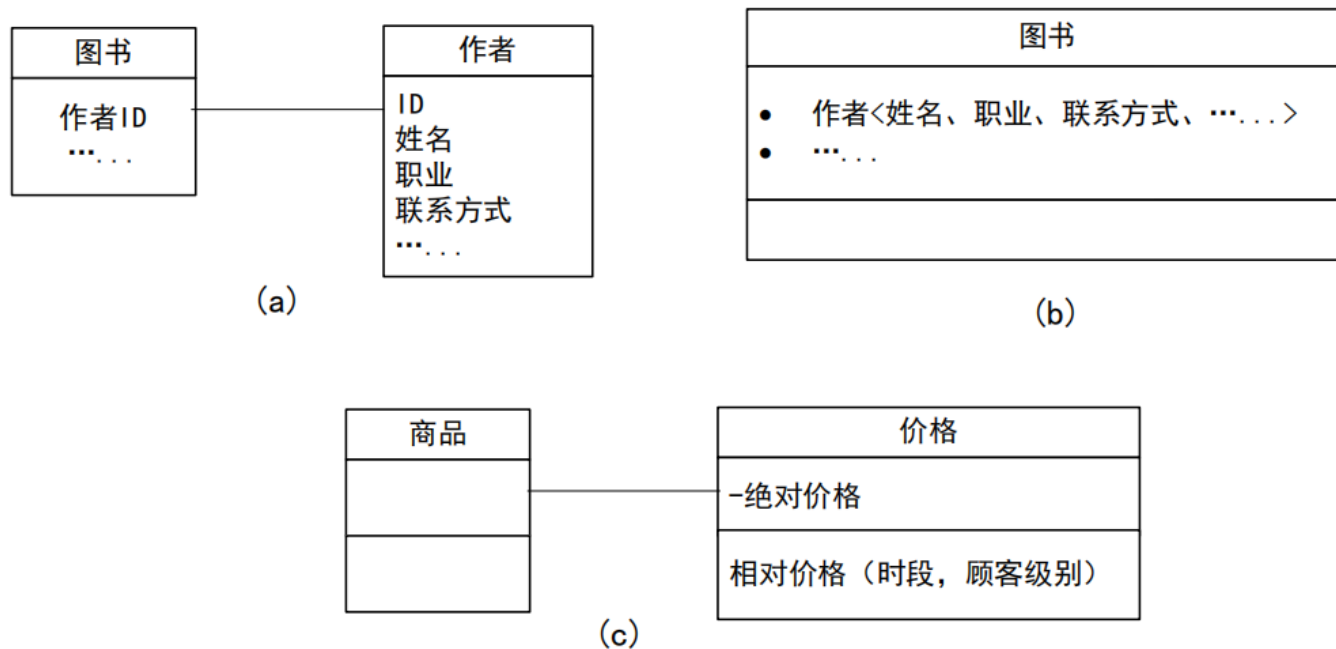


图11-13 实体关系图建模与类图建模的对比实例

(2) 单属性对象

➤人们易于武断地将**单属性类**抽象为**其他类的属性**。

- 例如，在前面考察销售系统当中的“商品”和“价格”两个候选类的例子当中，很多人会因为“价格”仅仅包含一个属性（价格：Integer）而武断地将其作为“商品”的属性。
- 特殊场景，如图11-13（c）所示。

➤除了“价格”之后，其他常见的单属性类及其特殊场景有：

- ① **标识ID**，在标识有**验证行为**（ID需要符合特定规则，例如身份证号）或递增行为（例如数据表主键）时，是单属性类。
- ② **数量**，在数量有**相对概念**时（不同场景换算不同数量，例如：初级用户使用“个”、高级用户使用“万个”），是单属性类。
- ③ **状态**，在状态有**转换功能**时（例如：“开”状态转“关”状态、“关”状态转“开”状态），是单属性类。
- ④ **度量单位（unit）**，度量单位有**场景性**时（例如不同场景使用米、厘米、尺、寸），是单属性类。
- ⑤ **有结构的字符串**，如果字符串需要**验证结构合法性**（例如邮件地址、居住地址），是单属性类。

三、示例

(1) 例一

用例描述:

1. 顾客向系统提起查询请求
2. 系统根据请求为顾客提供一个CD的推荐列表
3. 顾客在推荐列表选定一个CD, 然后要求查看更详细的信息
4. 系统为顾客提供选定CD的详细信息
5. 顾客购买选定CD.
6. 顾客离开.

a)

候选对象:

人: 顾客
事物: CD
组织: CD推荐列表
概念: 查询请求、详细信息
事件: 购买



b)

概念类:

顾客,
CD,
CD推荐列表,
查询请求,
购买

c)

图11-14 从候选类中确定概念类示例一

(2) 例二

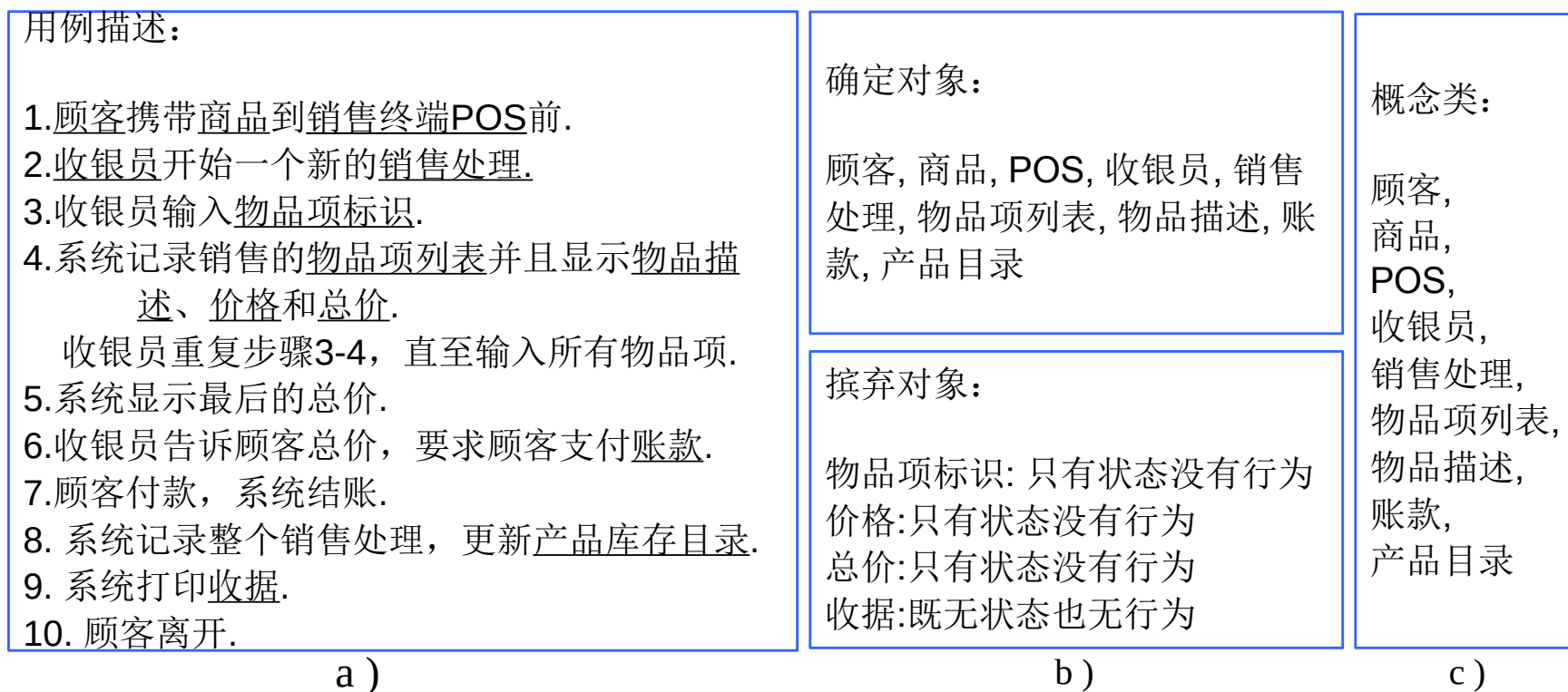


图11-15 从候选类中确定概念类示例二

建立类之间的关联

- 要建立孤立的概念类之间的**关联**，把它们联系起来可以从两个方面着手：
- ① 分析问题域内的**静态结构关系**，发现概念类之间的**整体部分关系**和**明显的语义联系**；
 - ② 分析概念类之间的**协作**，协作能够体现概念类之间明显以及不明显的语义联系。

一、建立关联的规则

建立类之间的关联时，要注意下列规则：

(1) 保证类之间协作所必需的可见性

两个需要实现互相之间的协作的对象实例，就应该具有能够保证可见性的关联。为保证类之间协作而建立的关联是**必要关联**，被称为“**需要知道**”（**Need to Know**）型关联，是对象模型必不可少的部分。



(2) 适当使用**问题域内的关联**，增强领域模型的**可理解性**

- 有些类之间不需要互相协作，但是它们的**对象实例**在问题域内存在某些重要而且固定的**关系**。
- 对问题域内关系的识别要**适可而止**，因为问题域内的关系是复杂和繁多的，因为它们建立太多的关联不仅不能有效的表示领域模型，反而会使领域模型变得混乱。

(3) 避免显示**冗余**和**导出**的关联

- 发现关联后使用合适的动词短语为关联命名，描述每个关联端的角色和多重性。关联名称通常按照**自上至下**、**自左至右**的方式表达概念类之间的关系。
- 在分析阶段，一般不会描述关联的方向和关联端的可见性。
 - 不过在**非常必要**的情况下（例如存在重要的约束或者某些类有特殊要求），也可以描述关联的方向和关联端的可见性。

(4) **不要**在关联的识别上**花费太多的时间**

- 识别概念类比识别关联**更加重要**。

二、示例

➤根据图11-16 a)的需求信息（其概念类如图11-14 c)），可以从协作和问题域两个方面为其发现关联如图11-16 b)所示，并建立图11-16 c)所示的领域模型。

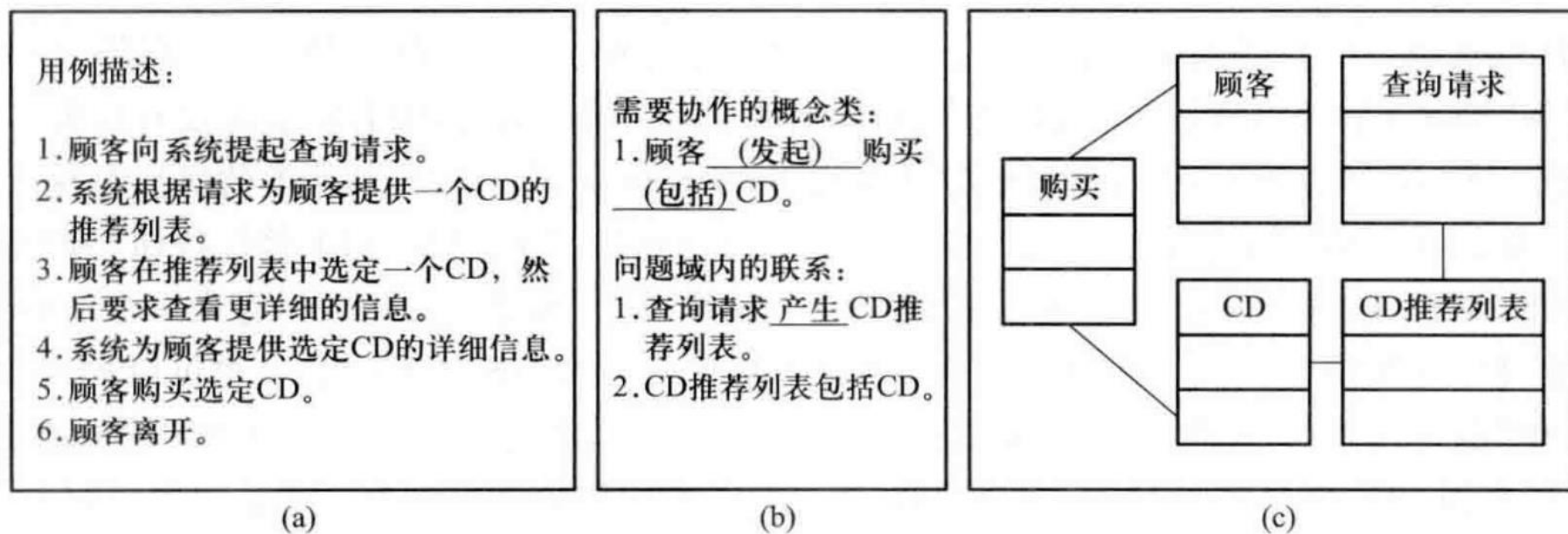


图11-16 关联建立示例

► 为图11-16 c)的领域模型关联添加了详细的描述之后，产生的领域模型如图11-17所示。

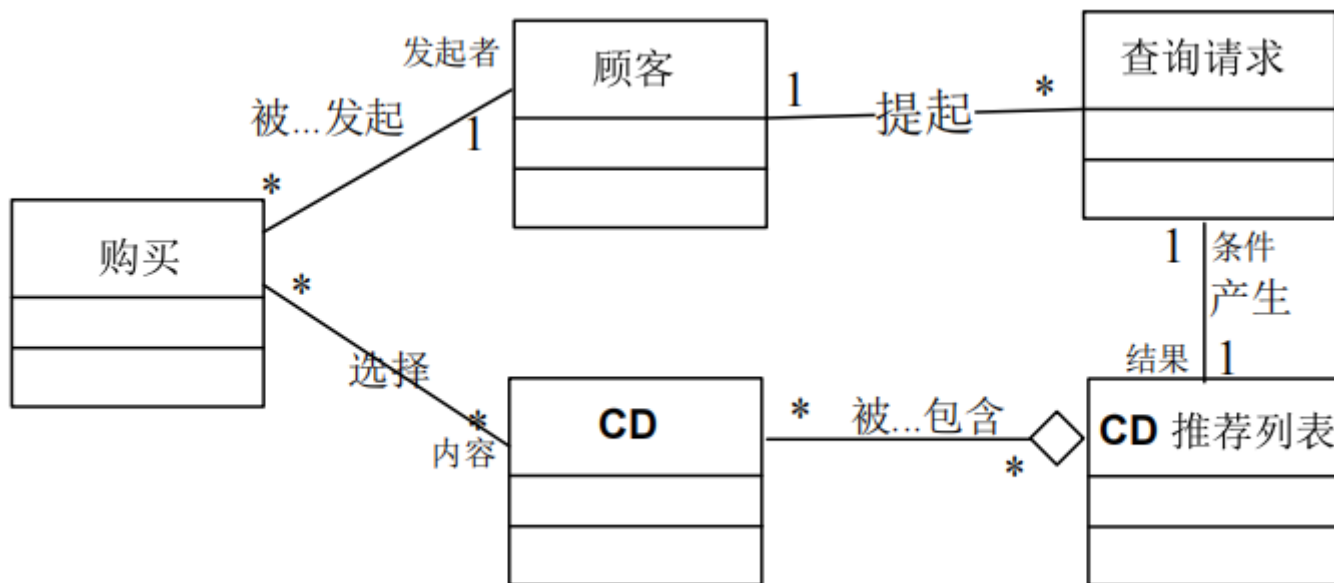


图11-17 建立关联后的领域模型示例

➤ 请大家按照上述步骤自行为图11-15的概念类建立关联

用例描述:

1. 顾客携带商品到销售终端POS前.
2. 收银员开始一个新的销售处理.
3. 收银员输入物品项标识.
4. 系统记录销售的物品项列表并且显示物品描述、价格和总价.
 收银员重复步骤3-4, 直至输入所有物品项.
5. 系统显示最后的总价.
6. 收银员告诉顾客总价, 要求顾客支付账款.
7. 顾客付款, 系统结账.
8. 系统记录整个销售处理, 更新产品库存目录.
9. 系统打印收据.
10. 顾客离开.

a)

确定对象:

顾客, 商品, **POS**, 收银员, 销售处理, 物品项列表, 物品描述, 账款, 产品目录

摒弃对象:

物品项标识: 只有状态没有行为
价格: 只有状态没有行为
总价: 只有状态没有行为
收据: 既无状态也无行为

b)

概念类:

顾客,
商品,
POS,
收银员,
销售处理,
物品项列表,
物品描述,
账款,
产品目录

c)

图11-15

➤ 其参考结果如图11-18所示

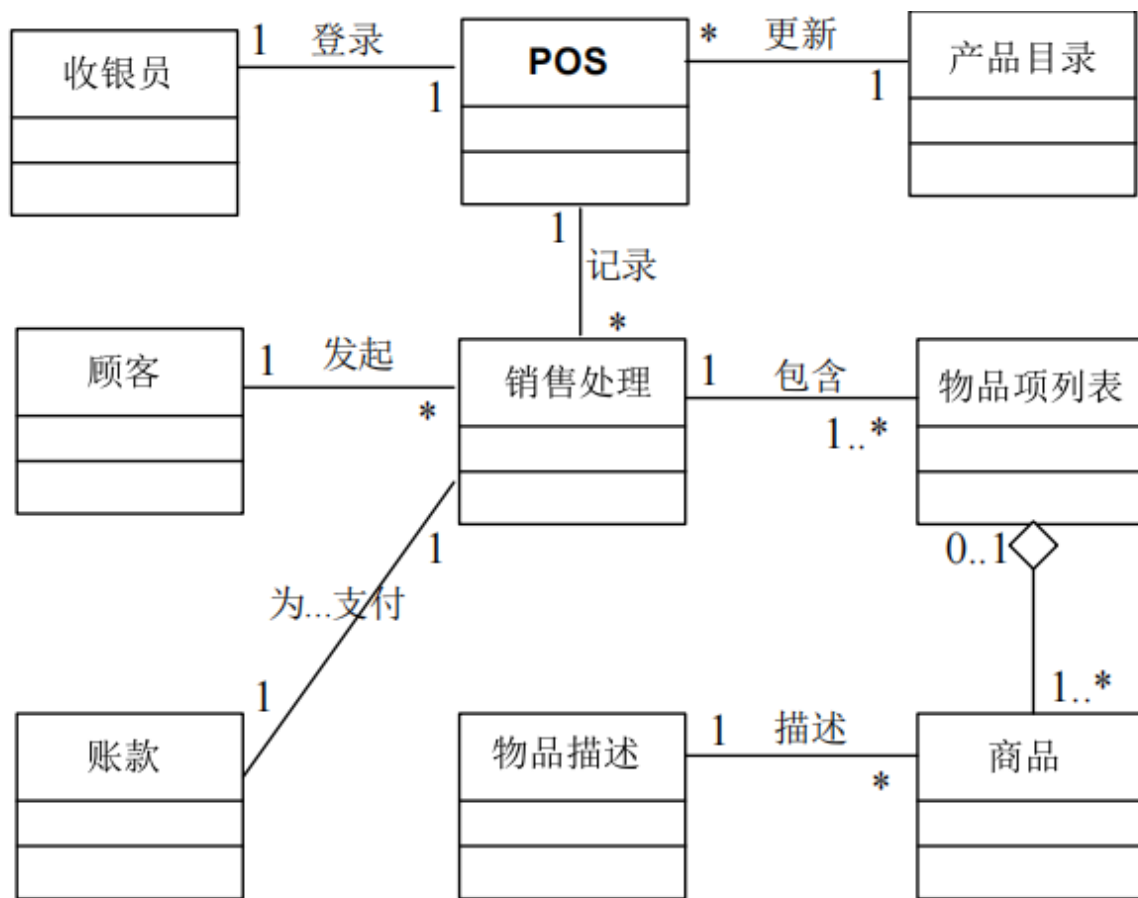


图11-18 应用的参考示例

添加类的重要属性

- 是建立领域模型的最后一个步骤
- 实现类协作时**必要的信息**，是协作的条件、输入、结果或者过程记录。
- 用例的描述**当中可以发现关于属性的信息，在**需求获取阶段获取的硬数据**是重要的资料。
- 使用**用户的描述方式**，不进行类型和约束的严格定义
- 遇到复杂的属性时，按照提高质量的考虑，应该进行**简化处理**。如果是为了理解需要，可以先不加处理，设计阶段再行考虑。

➤例如，可以根据前面用例描述中的有限信息，为图11-17的模型添加属性，建立如图11-19所示的领域模型

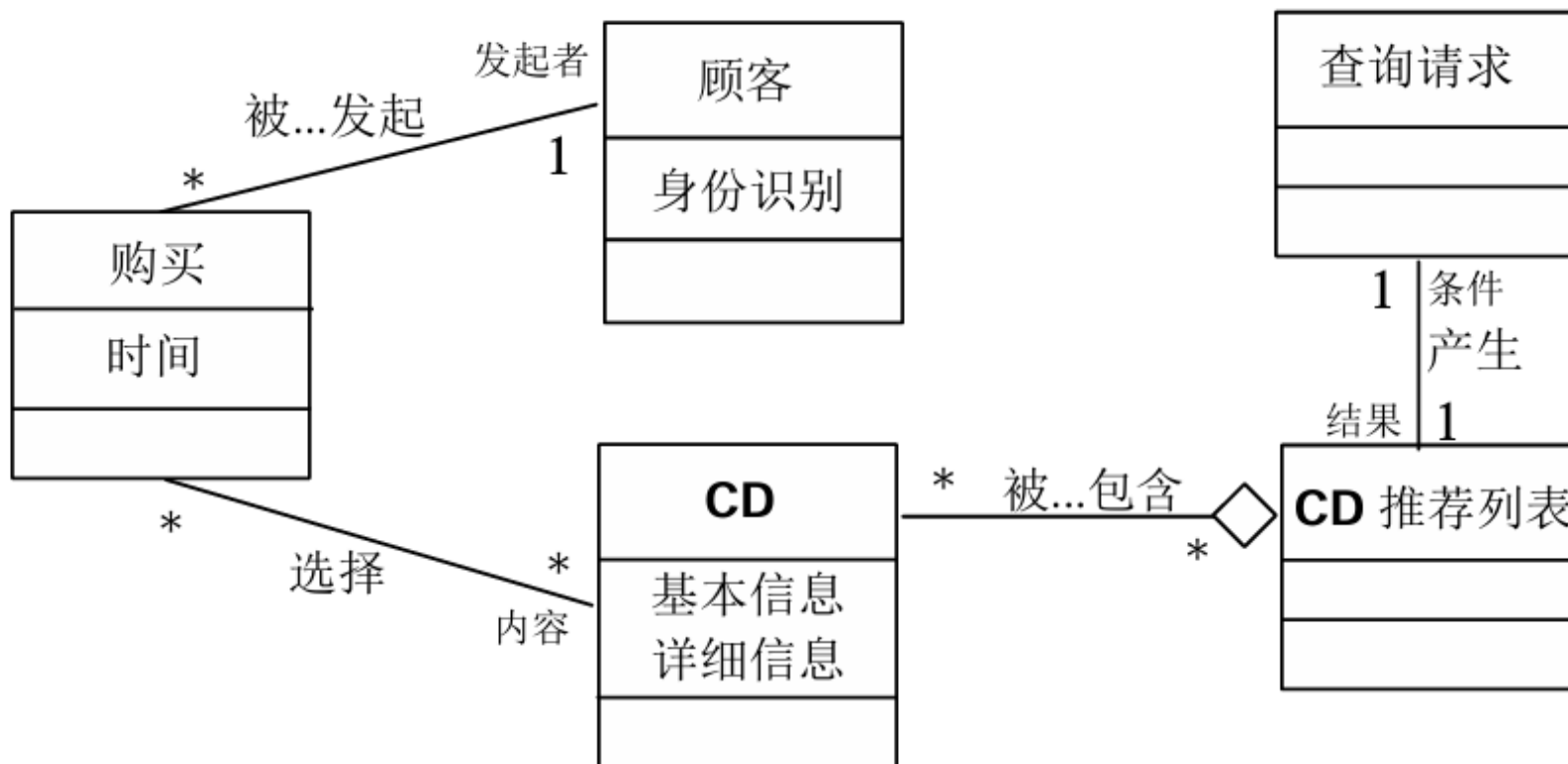


图11-19 添加了属性的领域模型示例

领域模型的分析作用

- 要记住需求分析的目的是**理解需求内容**，发现其中的**缺陷与不足**，而不是简单机械地建立一个图形模型。
- 主要作用是**描述数据**，所以过程中最能发现的需求缺陷与不足也是在数据方面，表现为数据的定义、加工与使用。

CD推荐的案例中，发现的需求缺陷

顾客是否需要注册？注册哪些信息？

查询请求包括哪些条件？

CD推荐列表包含哪些信息？

CD详细信息包含哪些信息？

购买行为操作哪些数据？

.....

商品销售的案例中，发现的需求缺陷

物品项标识需要标准定义吗？

总价的计算规则是怎样的？

结账操作的数据有哪些，如何操作的？

产品库存目录是怎样定义的？

收据包含哪些信息，格式怎样？

.....

领域模型的分析作用

- 分析中发现的需求缺陷有些可以通过**分析其他用例**来解决。

例子：如分析“顾客注册”用例可以解决问题“顾客是否需要注册？注册哪些信息？”

- 一个系统中**所有用例**的领域模型合并起来，才构成完成的需求信息，所以单个用例的领域模型有缺陷是正常的，可以在其他用例的领域模型中得到解决。
- 有些分析中发现的需求缺陷的确是**未获取过的信息**或者分析中出现的**错误**，这时就需要根据发现的缺陷**重新获取**相关内容或者修正错误的分析模型。
- 如果一个用例的领域模型分析**没有发现缺陷**，那么该用例的需求内容就是比较**完备**的，不再需要后续的获取过程。

目录

- 面向对象分析与UML

- 对象模型

- 建立领域模型

- 行为模型--交互图

- 建立交互图

- 行为模型--状态图

- 建立状态图

- 对象约束语言OCL

- 基于CRC卡面向对象分析方法

- 需求细化

概述

顺序图

通信图

系统顺序图

概述

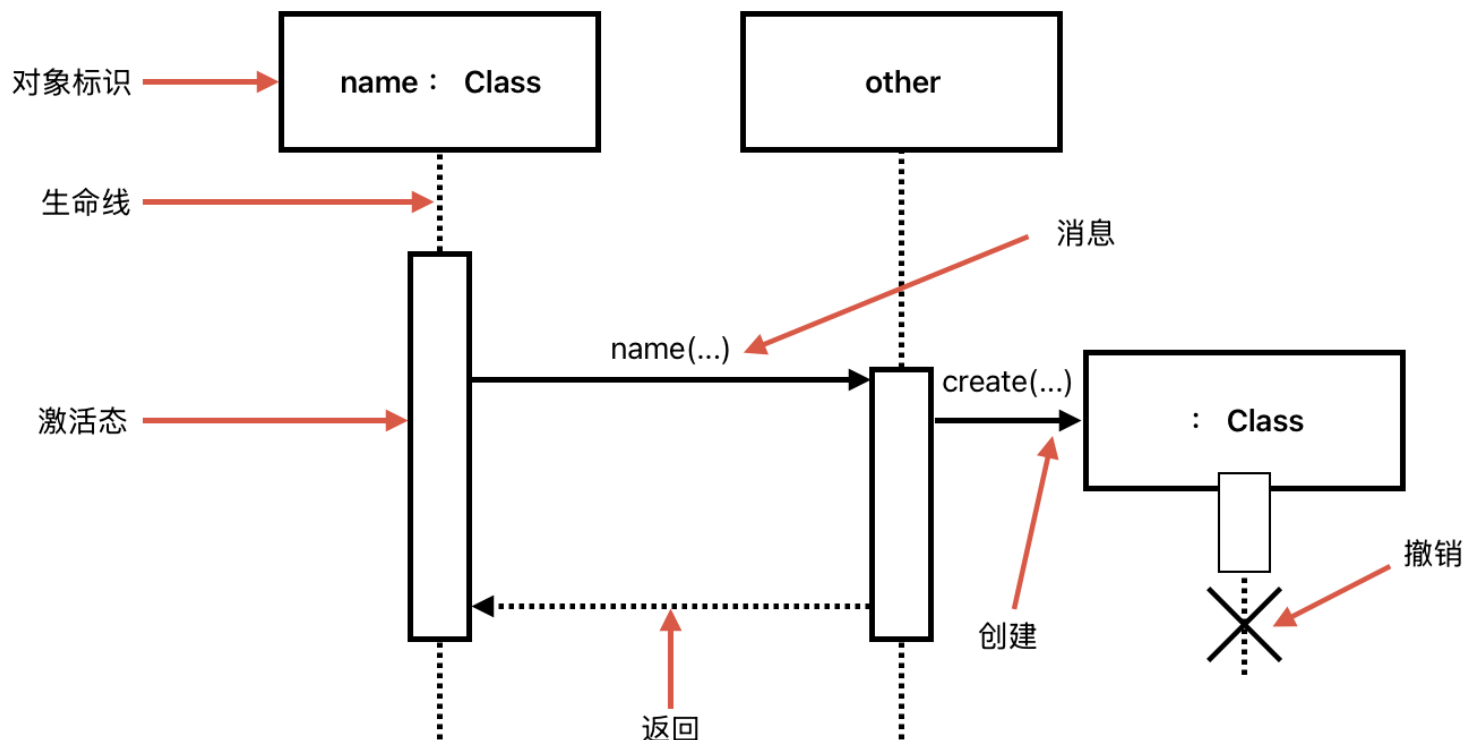
➤ 交互图

- 以一组对象为中心的交互描述技术
- 描述在特定上下文环境中一组对象的交互行为
- 其通常描述的是单个用例的场景
- 交互图中的每一个交互都描述了环境中的对象为了实现某个目标而执行的一系列消息交换，其上下文环境就是被实现用例的一个或多个场景
- UML的交互图包括顺序图（Sequence Diagram）、通信图（Communication Diagram）、交互概述图（Interaction Overview Diagram）和时间图（Timing Diagram）
- 需求分析中常用的是顺序图和通信图

顺序图

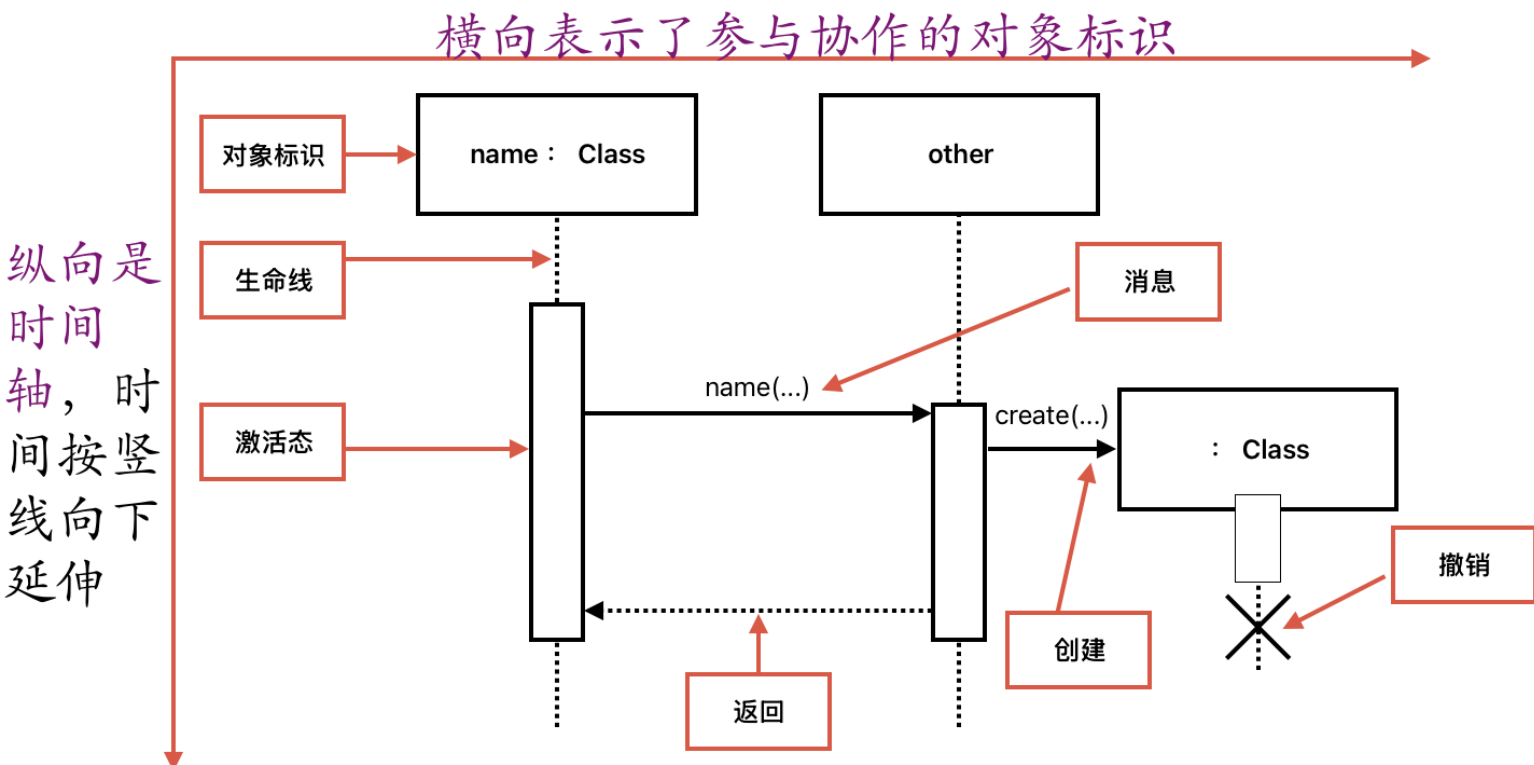
➤ 顺序图可以突出消息的时间顺序

➤ 一个简单的顺序图图示



顺序图

➤ 顺序图将交互表示成一个二维图表



每个对象标识用一条带有头符号和竖线的垂直栏（生命线）表示

- 当对象存在但不工作时，标识用一条虚线表示
- 在对象处于工作状态时，生命线是一个双道线，表示对象处于激活状态。

顺序图

➤ 顺序图显示了交互行为中的消息序列

- 消息用从一个对象生命线到另一个对象生命线的箭头来表示。



同步消息

- 箭头依时间顺序在图中从上到下排列。
消息有同步、异步和返回消息之分，分别用不同的图形符号进行表示。



异步消息

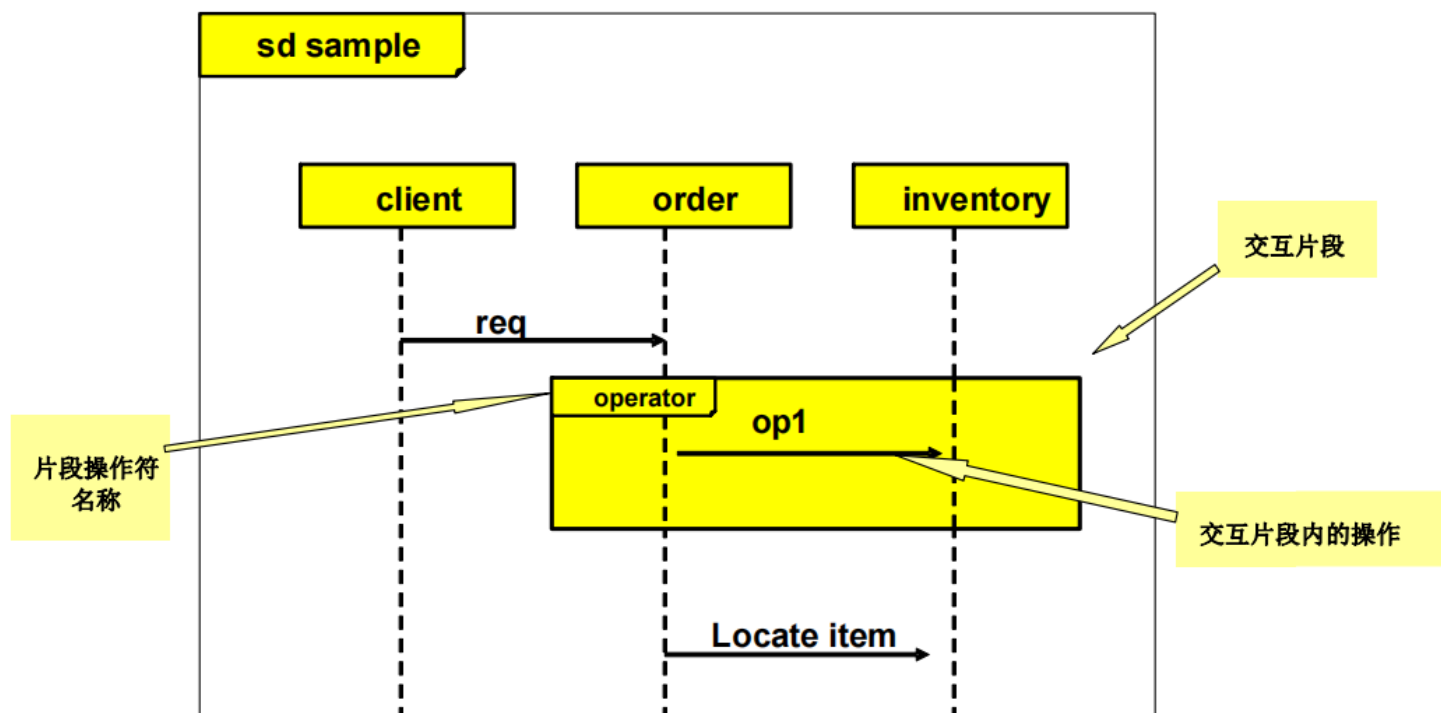


返回消息

- 消息箭头的标注语法为：
[attribute=]name[(argument)][:return-value]，其中attribute是生命线所代表对象的可选属性名称，用于保存返回值。
- 同步消息通常需要有相应的返回消息，异步消息则不需要返回消息。

顺序图

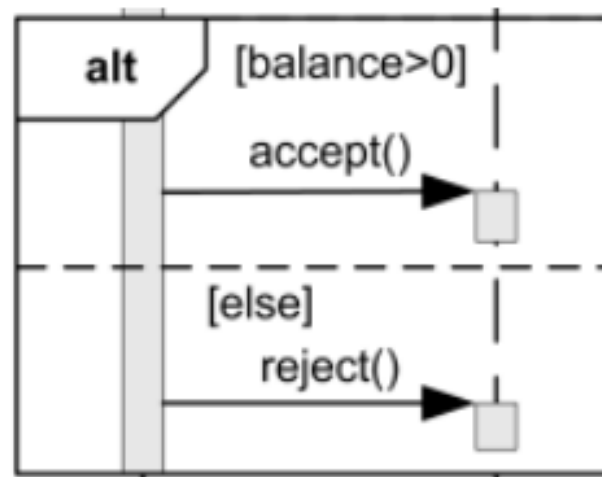
➤ 顺序图还使用了一些表达复杂情景的组合片段
(Combined Fragment)



顺序图

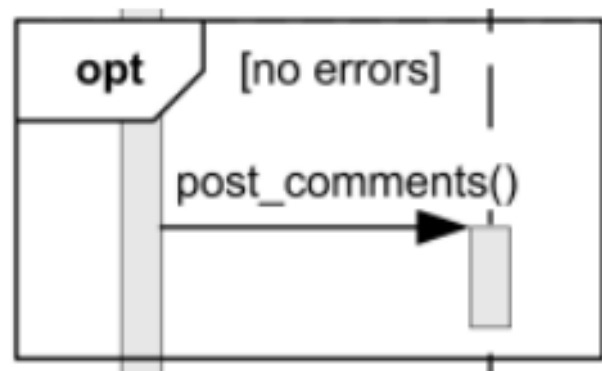
➤ 选择（Alternatives）：

- 操作符alt，表示要从多个行为中根据监护条件选择一个交互行为执行。在用例中存在分支流程时，往往要使用多选一组合片段。



➤ 可选（Option）：

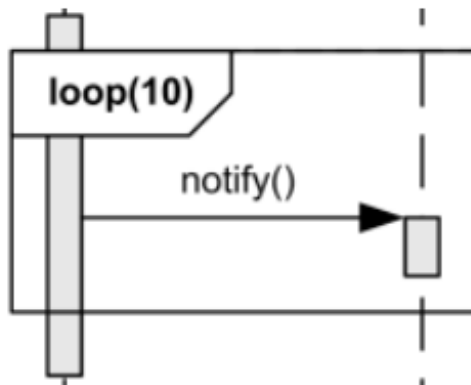
- 操作符opt。如果符合监护条件，那么片段内的交互行为就会得到执行，否则就不执行。



顺序图

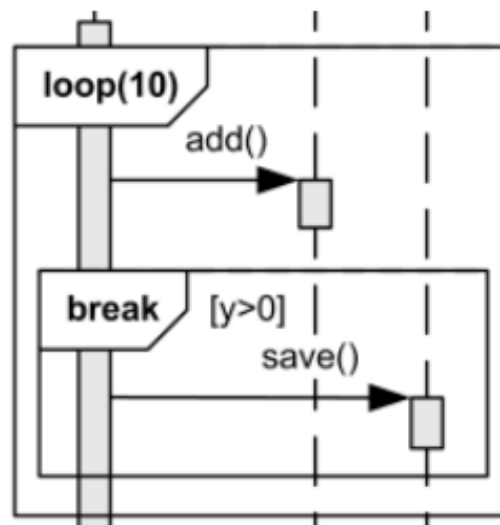
➤ 循环 (Loop) :

- 操作符loop。只要监护条件为真，片段内的交互行为就会被反复执行。



➤ 中断 (Break) :

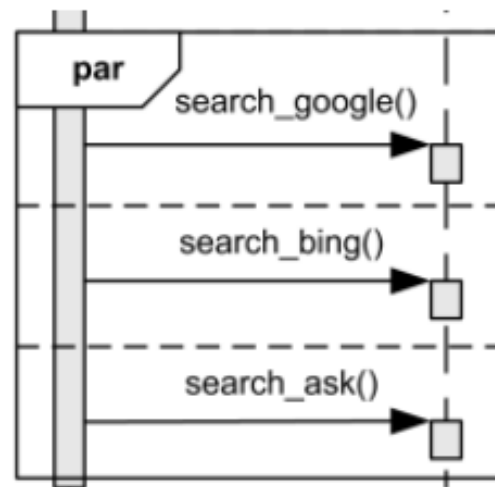
- 操作符break。如果监护条件为真，片段内的交互行为就会被执行，并在执行后退出中断片段所在的顺序图，也就是说如果中断发生，那么顺序图中中断片段之后的交互行为就不会得到执行。



顺序图

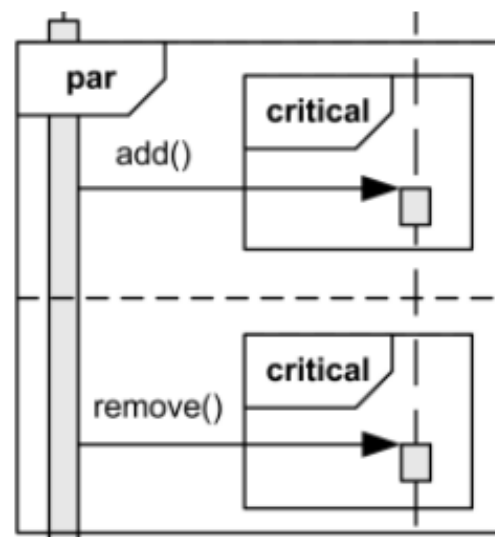
➤ 并行 (Parallel) :

- 操作符par。并行片段内的不同交互行为可以不遵循顺序图的时间线顺序，可以并行、交织进行。



➤ 关键区域 (Critical Region) :

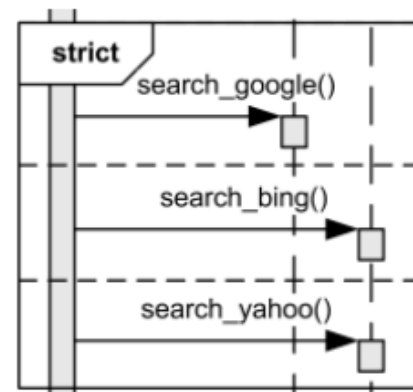
- 操作符critical。区域内的交互行为是一个原子操作，一旦开始执行就必须执行完成，在执行过程中不能被打断，例如其另一个并发片段中发生了Break行为，关键区域内容的交互行为仍然要执行完成后才能退出其所在的顺序图。



顺序图

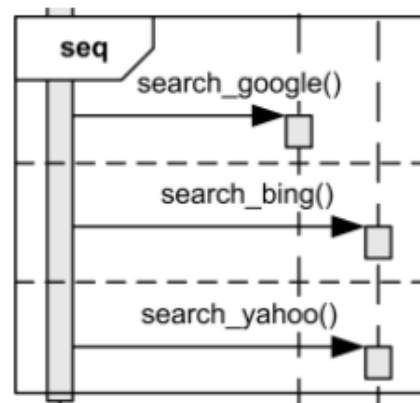
➤ 强顺序 (Strict Sequencing)

- 操作符strict。强顺序内的交互行为必须顺序执行，例如图内的行为必须按照“search_google()→search_bing()→Search_yahoo()”的顺序执行。



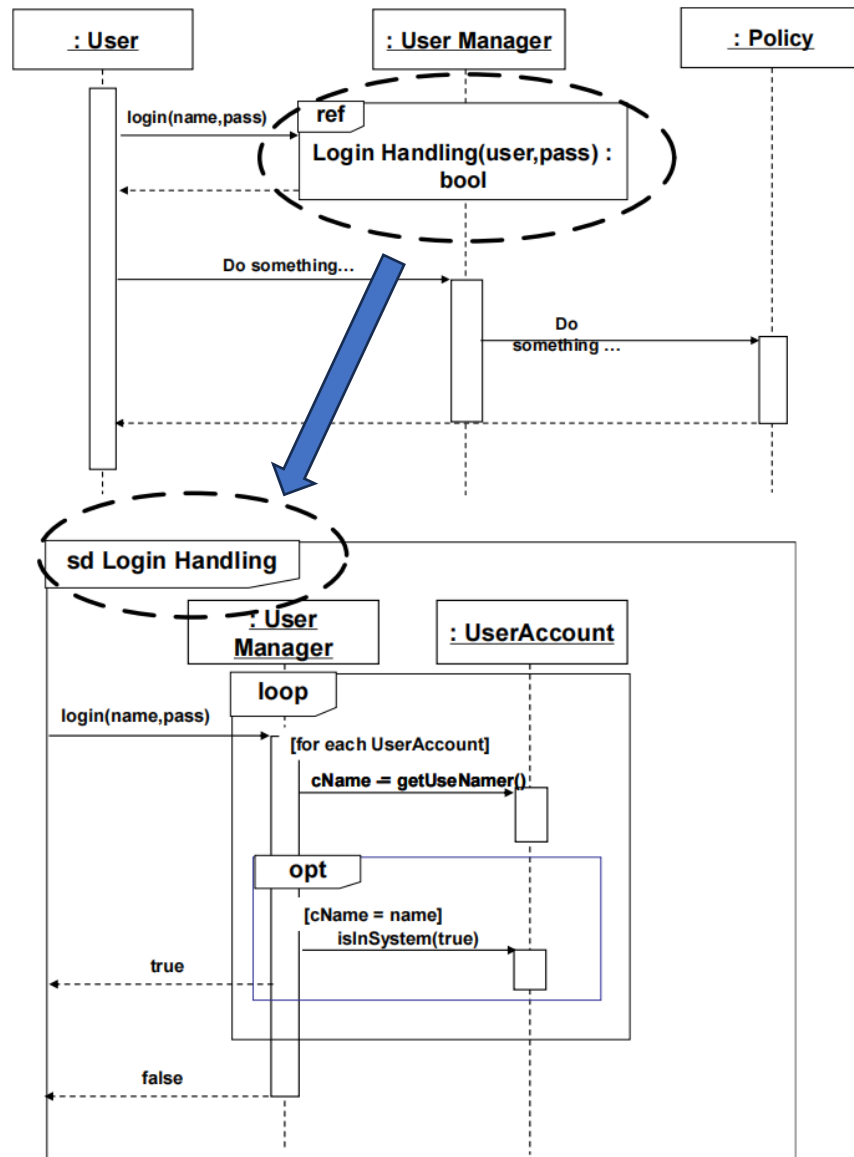
➤ 弱顺序 (Weak Sequencing) :

- 操作符seq。在弱顺序下：
 - ①每个操作组内的交互行为要维持顺序关系；
 - ②不同生命线上的不同操作组可以按照任何顺序执行；
 - ③同一个生命线上的不同操作组要按照顺序执行。例如，Search_bing()与Search_yahoo()必须先后执行，但是Search_goolge()是“Search_bing() → Search_yahoo()”是并发关系。



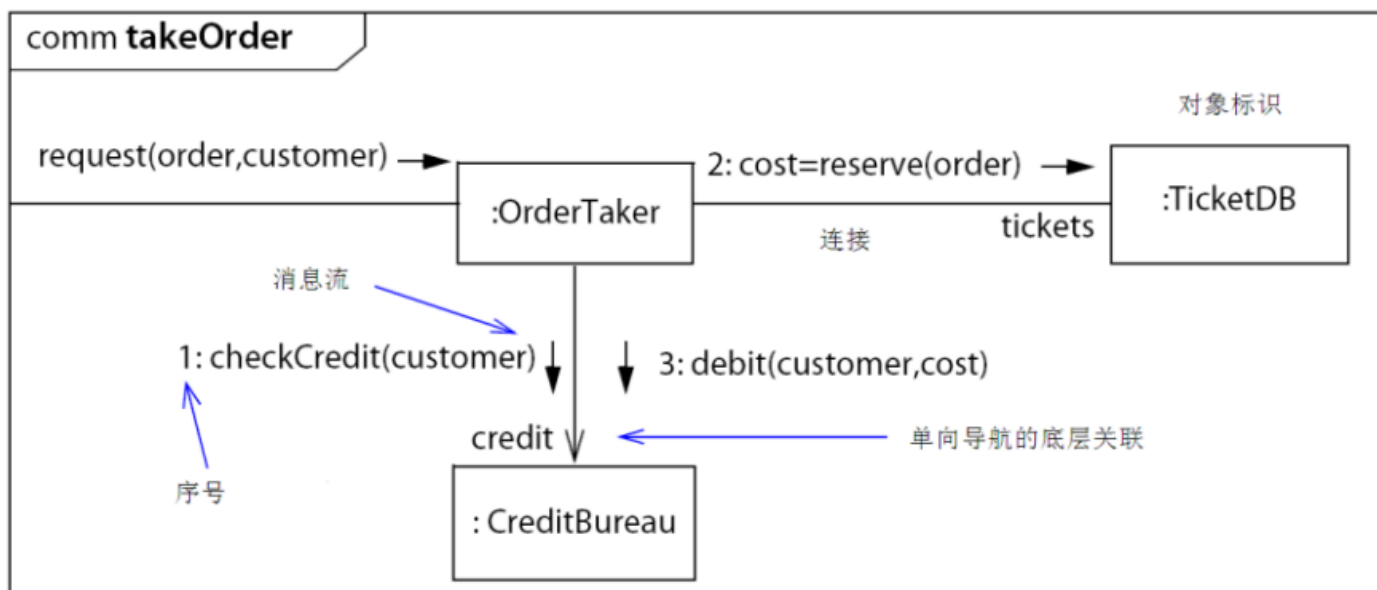
顺序图

- 引用不是一种组合片段，但它使用了类似于组合片段的图示（操作符ref），使得一个顺序图中可以引用其他的顺序图，从而实现将一个复杂顺序图分解为多个简单顺序图的目的。
- 如图所示，上面的ref片段引用了下面的顺序图，ref片段的交互刺激和响应与下面顺序图是一致的。



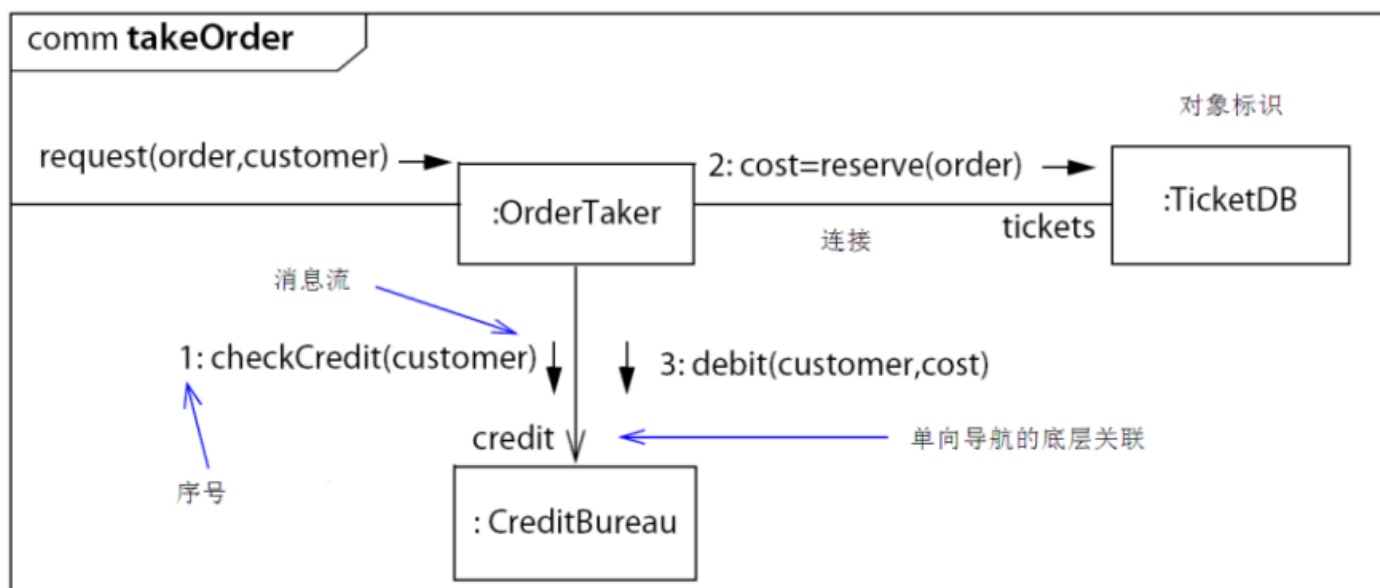
通信图

- 通信图能够突出交互当中协作对象之间的关系
- 在结构上它有些类似于概念类图
- 一个简单的通信图示例



通信图

- 对象标识及其连接说明了在上下文环境中可能出现的对象和对象间的链接情况
- 对象之间的消息用附加在连接上带标签的箭头来表示



通信图

➤消息的标注为：`[sequence-expression:]message`

- **message**描述了消息的内容，其格式和序列图中的消息标注相同
- **sequence-expression**是对消息执行顺序的描述，其格式为
`label [iteration-expression]`
- label是整数或者名称
 - 整数代表了消息的执行序号，如消息3.1.4要发生在3.1.3之后
 - 消息也能表达活动的嵌套层次，如消息3.1.4是在消息3.1的嵌套范围内发生的
 - 名称代表了并发的控制线程。最后名称不同的消息在所处的嵌套层次上是并发的，如3.1a和3.1b就是在3.1嵌套范围内的两个并发消息，它们之间不分先后，是完全平等的
- iteration-expression表示消息应该条件执行或者迭代执行，即进行0次或多次消息发送。其形式为`::*[iteration-clause]`或者`[condition-clause]`
 - 要表示一个消息需要发送n次，则一个示例为 `3.1*[i=1..n]:update()`
 - 要表示一个消息需要满足条件才能执行，示例为 `1b.4 [x<0]:invert(x, color)`

系统顺序图

顺序图和通信图从不同的侧重点对用例的典型场景进行了完全等价的实现。在实现当中，**顺序图和通信图都需要和领域模型保持一致。**

一致性是指在交互图中出现的对象应该在领域模型中有相应的对象存在。

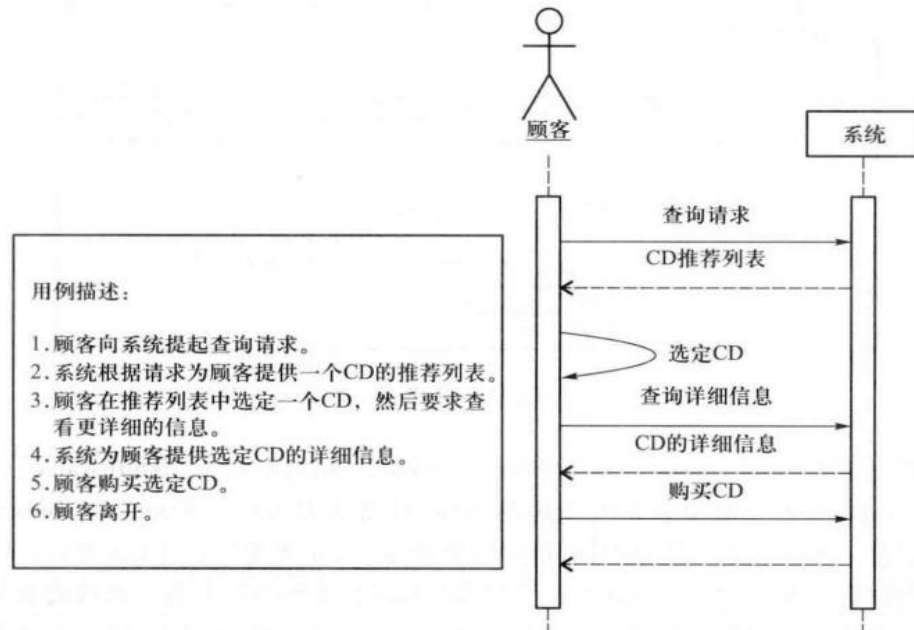
- 系统分析的过程是一个不断迭代和深入的过程，所以**一致性的要求会使得用例的实现出现迟滞现象。**
- 因为对象的发现无法在项目初始阶段全部完成的，所以**实现用例的详细交互图时难以提前确定参与协作的对象**，进而无法形成实现用例的详细交互图。

系统顺序图

➤ 一个更常见的做法是在分析阶段的开始开发系统顺序图，而不是直接实现在多个对象之间展开的比较详细的交互图。

➤ 系统顺序图将整个系统看作一个黑箱的对象，强调外部参与者和系统的交互行为，重点展示系统级事件。

➤ 一个系统顺序图示例，左为用例的场景描述，右为对应的系统顺序图



目录

- 面向对象分析与UML

- 对象模型

- 建立领域模型

- 行为模型——交互图

- 建立交互图

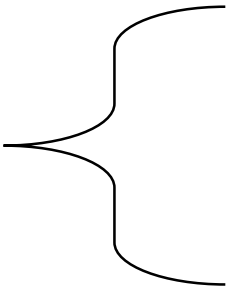
- 行为模型——状态图

- 建立状态图

- 对象约束语言OCL

- 基于CRC卡面向对象分析方法

- 需求细化



建立典型场景的系统顺序图
建立用例（多场景）系统顺序图
建立详细顺序图
交互图的分析作用

建立典型场景的系统顺序图

- 通信图和顺序图是等价的图示，创建的方法也类似，这里只展示顺序图的建立过程，不再描述通信图的建立过程。
- 最基础的交互图是用例典型场景（通常是主流程场景）的系统顺序图。
- 面向对象需求分析中最基本的工作也是为复杂用例的典型场景建立系统顺序图。

建立典型场景的系统顺序图

➤ 建立步骤

① 确定系统顺序图的上下文环境

- 系统顺序图的交互是在一定的场景环境下发生的，离开这个上下文环境的限定，对交互行为的描述和理解都会出现一定的问题。因此，建立系统顺序图需要首先确定下文环境，限定描述范围。
- 上下文环境的前置条件和后置条件应该被分配给系统顺序图中的相应行为，这个工作会在为交互行为添加契约说明的时候得到实现。

② 找出参与交互的对象

- 在场景环境中寻找参与交互的对象，寻找的目标是系统、系统之外的对象和其他系统。

建立典型场景的系统顺序图

③ 根据发现的对象建立交互图框架

- 将对象平行排列，并添加对象的生命线。

④ 添加消息，描述交互行为

- 以消息的方式，将对象之间的交互行为描述出来，并建立行为之间的顺序，要注意维护对象生命线的激活状态。
- 描述时仅仅需要考虑系统与外界的交互行为，忽略那些与系统无关的（外部对象之间的）或系统内部的交互行为。

建立典型场景的系统顺序图

►对商品销售的用例描述，按照步骤建立系统顺序图

用例描述：

1. 顾客携带商品到销售终端POS前。
2. 收银员开始一个新的销售处理。
3. 收银员输入物品项标识。
4. 系统记录销售的物品项列表并且显示物品描述、价格和总价。
收银员重复步骤3-4，直至输入所有物品项。
5. 系统显示最后的总价。
6. 收银员告诉顾客总价，要求顾客支付账款。
7. 顾客付款，系统结账。
8. 系统记录整个销售处理，更新产品库存目录。
9. 系统打印收据。
10. 顾客离开。

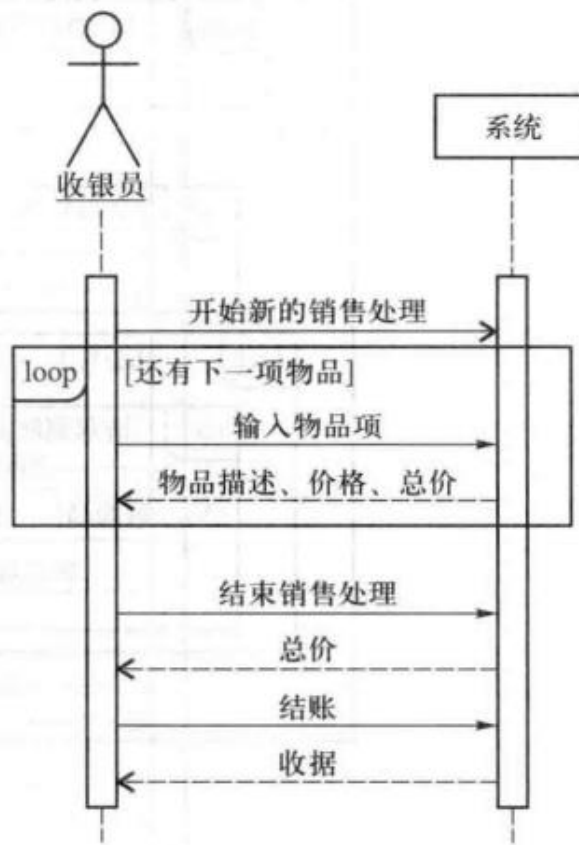
- **确定上下文环境**，以用例描述中的流程为场景环境。例子中的场景描述相对比较独立，没有对其他用例或场景的引用，因此建立系统顺序图的过程和结果也相对比较简单。
- 根据用例描述可以找到顾客、收银员和系统三个交互对象。仔细分析后可以发现顾客和系统之间没有直接的交互，因此可以剔除。最后**发现的交互对象为收银员和系统**。
- 按照用例描述中的流程顺序，**逐步添加消息**，并在进行详细信息描述后建立系统顺序图。

建立典型场景的系统顺序图

➤ 商品销售的系统顺序图

用例描述：

1. 顾客携带商品到销售终端POS前。
2. 收银员开始一个新的销售处理。
3. 收银员输入物品项标识。
4. 系统记录销售的物品项列表并且显示物品描述、价格和总价。
收银员重复步骤3—4，直至输入所有物品项。
5. 系统显示最后的总价。
6. 收银员告诉顾客总价，要求顾客支付账款。
7. 顾客付款，系统结账。
8. 系统记录整个销售处理，更新产品库存目录。
9. 系统打印收据。
10. 顾客离开。



建立用例（多场景）系统顺序图

➤ 组合片段使得顺序图能够处理复杂分支，使得顺序图能够同时描述多个场景

➤ 所以，在描述多场景用例的系统顺序图时

- 可以为主流程场景建立基础的系统顺序图
- 然后根据分支场景与异常场景的分支点、异常点，
建立组合片段描述



从而在一个系统顺序图中描述多个场景。

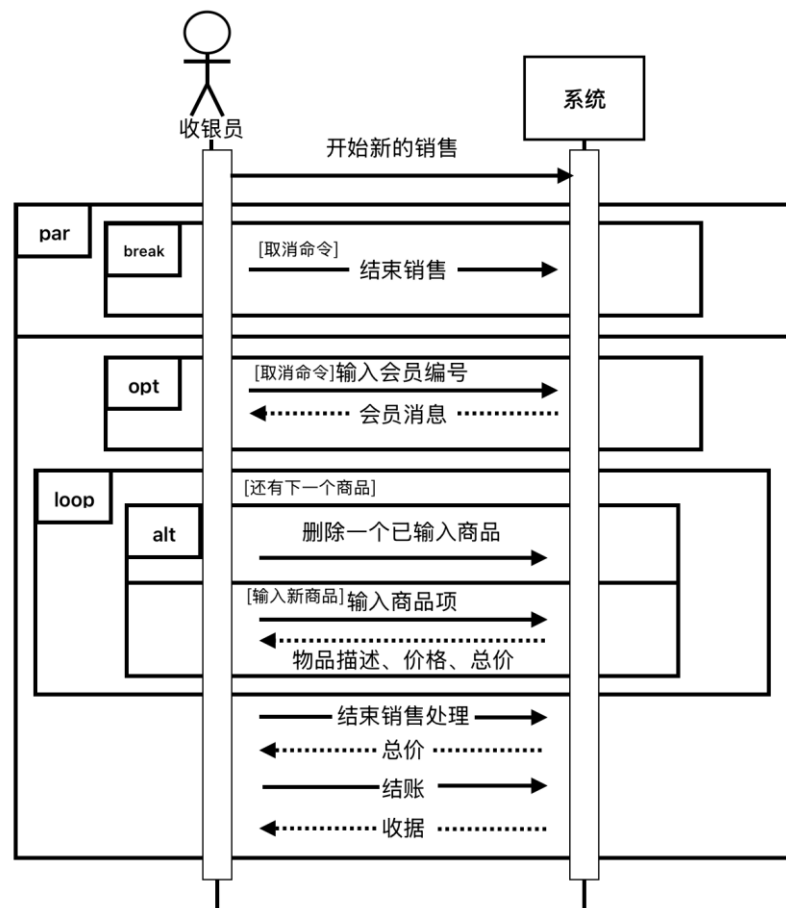
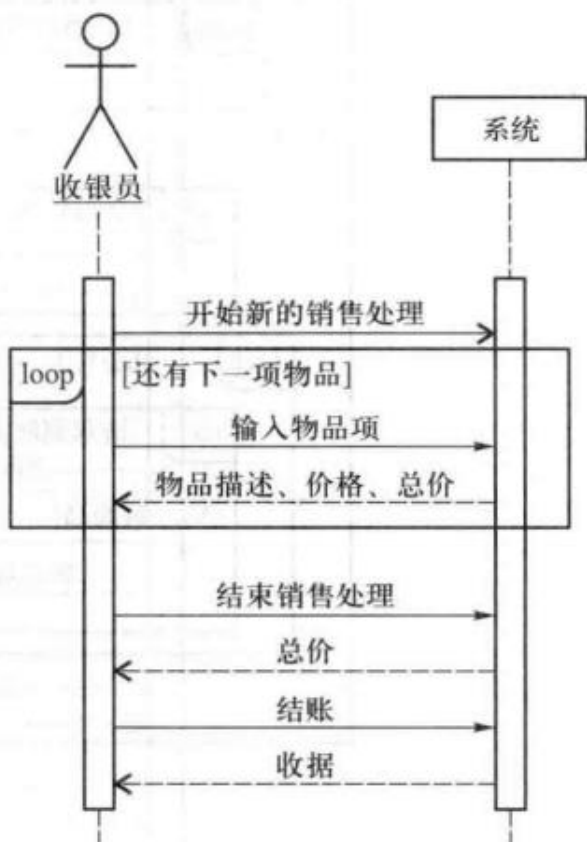
建立用例（多场景）系统顺序图

➤如果在商品销售的系统顺序图中添加下列场景

- 在销售开始时“输入会员编号”，分支点在“开始新的销售处理”之后，为可选场景（opt片段）；
- “删除一个已输入商品”，分支点在“输入物品项”及其返回消息之间，为选择场景（alt片段）；
- “取消销售”，建立一个中断场景（break片段）；
- 在“开始新的销售处理”之后、“结账”之前的任意一点，都允许“取消销售”，所以将中断场景与“开始新的销售处理”——“结账”设置为并发场景（par片段）。

建立用例（多场景）系统顺序图

➤ 为商品销售的系统顺序图添加组合片段之后，可以建立如图所示的系统顺序图

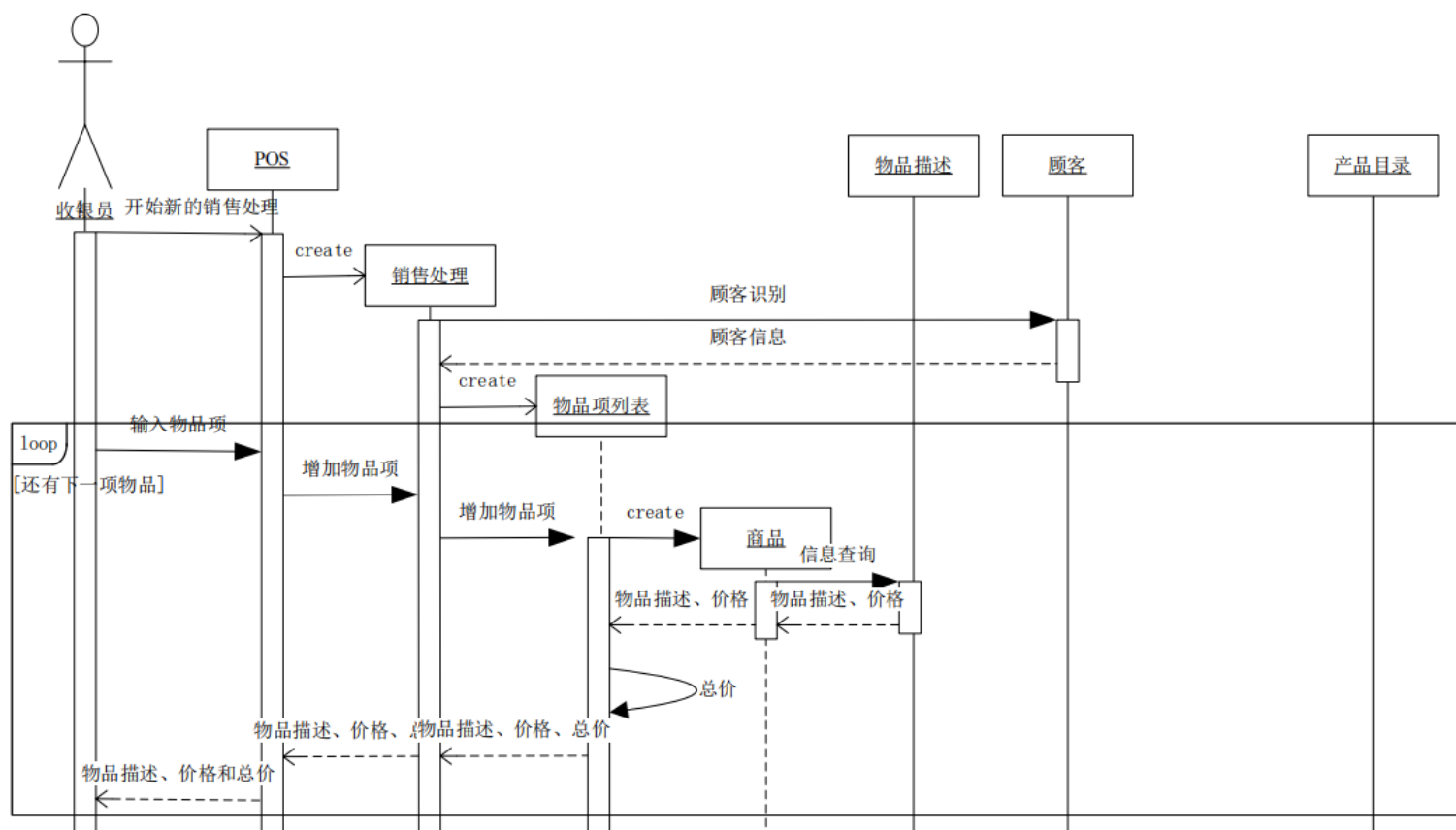


建立详细顺序图

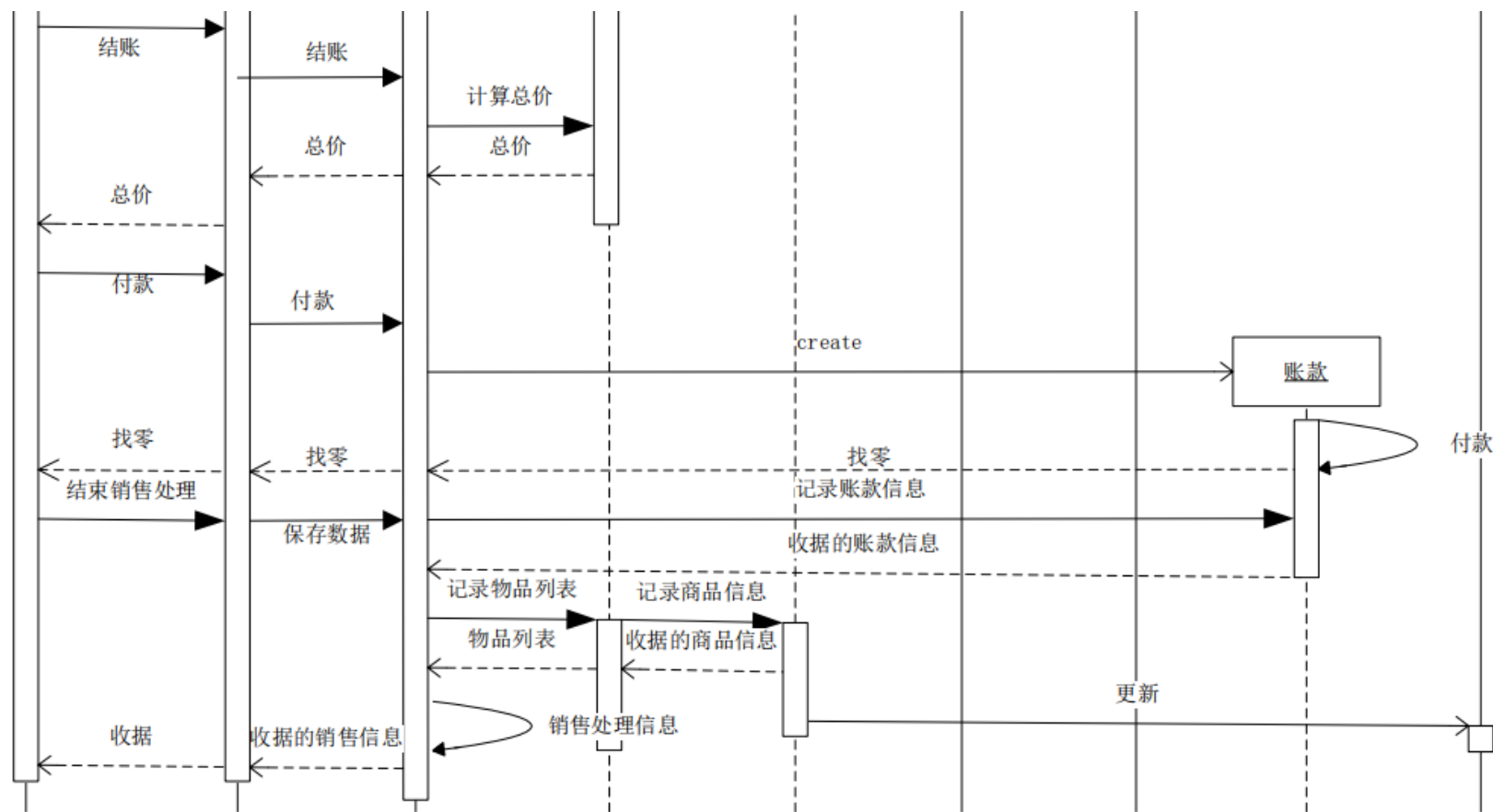
- 对简单项目进行需求分析时，建立系统顺序图描述就足够了。如果项目比较复杂，系统顺序图中的交互行为还嫌粒度太大，可以考虑适度使用详细顺序图。
- 建立详细顺序图的关键是正确识别参与交互的对象，这个可以借鉴领域模型的工作。一个用例的详细顺序图中参与对象应该与该用例的领域对象是一致的。
- 确定交互对象之后，将每一个外部交互转化为内部交互序列，就可以建立详细顺序图。
- 详细顺序图仍然只是分析模型，因为它没有添加设计因素，所以到了设计阶段还需要被设计师继续细化，添加界面、数据模型等设计要素。

建立详细顺序图

- 对商品销售的系统顺序图，参考其领域模型进行交互对象的确定，可以建立详细的顺序图



建立详细顺序图



交互图的分析作用

- 建立交互图时，**可以发现其所描述需求内容中的缺陷**
 - 主要是从行为方面发现问题，例如行为缺失、行为的数据不清晰等等
- 建立交互图时**最常发现的问题是系统的交互行为缺失**
 - 如果外界发起了同步交互，但是系统没有给出响应（返回消息），那么就意味着响应行为缺失
 - 如果在外界没有任何请求的情况下，系统主动给出了响应，那么就意味着请求行为的缺失
- **组合片段和消息描述可能会发现行为数据的问题**
 - 如果一个交互消息的数据内容在领域模型中没有描述，就意味着其数据内容是缺失的
 - 如果组合片段监护条件使用的数据内容在领域模型中没有描述，也意味着其数据内容的缺失

目录

- 面向对象分析与UML

- 对象模型

- 建立领域模型

- 行为模型——交互图

- 建立交互图

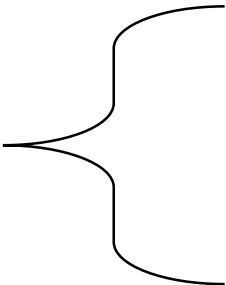
- 行为模型——状态图

- 建立状态图

- 对象约束语言OCL

- 基于CRC卡面向对象分析方法

- 需求细化



状态图的发展历程
有限状态机FSM
David Harel的发展
UML的状态图

状态图的发展历程

- 状态图是**以状态机理论为基础**建立的**对系统行为的描述手段**。

状态机是以“状态”概念为基础解释系统行为的一种技术，它在对系统行为的描述上得到了大量的应用

- **最简单的状态机是有限状态机FSM（Finite State Machine）**，它从理论上阐释了如何使用状态机模型来表示系统的行为。
 - 在各种方法的FSM应用当中，产生了状态转移图STD（State Transition Diagram）、Yourdon状态图示、SDL（Specification and Description Language）状态图示和状态转移矩阵STM（State Transition Matrix）等多种表示法。

状态图的发展历程

- 在大量的应用当中，人们发现了FSM的很多不足。
- 为此，David Harel对FSM进行了发展，补充的很多模型元素，建立了状态图SC（State Chart）。
- Harel的方法被OMT（Object Modeling Technique）采纳，并最终融入了UML，成为UML中的状态图SD（State Diagram）。



有限状态机 FSM

➤理论

- 状态机理论认为，系统总是处于一定的状态之中。而且，在某一时刻，系统只能处于一种状态之中。
- 系统在任何一个状态中都是稳定的，如果没有外部事件触发，系统会一直持续维持该状态。
- 如果发生有效的触发事件，系统将会响应事件，从一种状态转移到唯一的另一种状态。

➤依据上述的状态机理论，如果能够罗列出系统所有可能的状态，并发现所有有效的外部事件，那么就能够从状态转移的角度完整的表达系统的所有行为。这就是有限状态机的基本思想，它用来描述那些状态和事件数目有限的系统的行为。



有限状态机 FSM

➤FSM可以被看作是一个5元组： $FSM=(Q,\Sigma,\delta,q_0,F)$

- Q 是系统所有可能的状态集合

- 每个状态都记录了系统曾经发生过的行为，是过去的系统行为的累积结果。状态包含的信息要能够决定系统可能的下一步去向。

- Σ 是系统所有可能面对的触发事件

- 事件通常由一个刺激（stimulus）因素引起，并要求系统做出一定的响应（response）。
- 可能的事件包括三种类型：外部事件（external event）、内部事件（internal event）和时序事件（time-based event）。
- 外部事件是由系统外部的刺激因素引起的，例如用户的输入。
- 内部事件的刺激因素是系统内部的数据状态满足了事先预定义的条件，例如某一数据达到了最大或最小边界。
- 时序事件是由系统时钟触发的事件，例如时间到了某个指定时刻或者累积时间达到了限值。

有限状态机 FSM

- δ 是状态转移函数

- 它决定了在某个状态下，发生触发事件时，系统将转移到哪一个后续状态

- $\delta : Q \times \Sigma \rightarrow Q$

- $q_0 \in Q$ ，是系统的初始状态，也就是系统一开始时所处的状态，可以是空集

- $F \subseteq Q$ ，是系统可能的结束状态的集合

- 在持续的行为当中，一旦系统转移到了某个结束状态，系统就将结束自己的行为，否则会继续执行自己的行为，可以是空集。



有限状态机 FSM

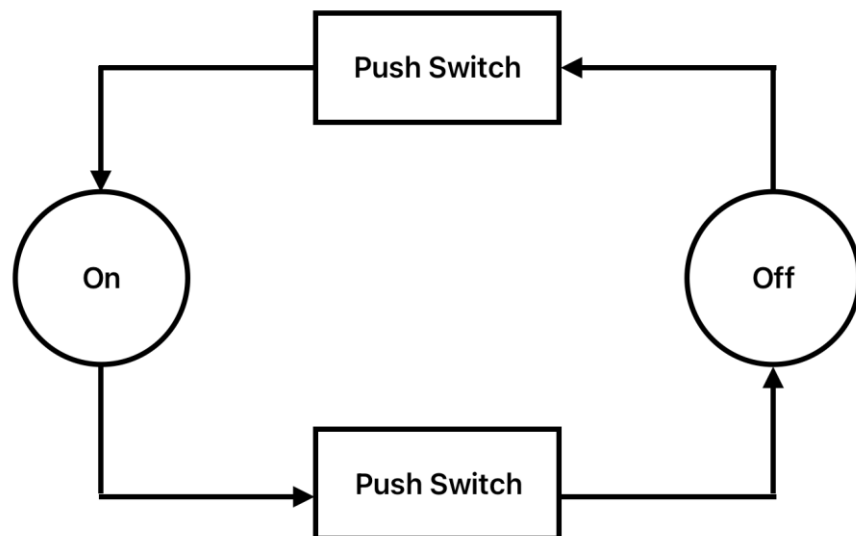
- FSM的5元组为FSM建立了完善的数学基础。所以FSM是在行为描述的诸多技术手段当中少数的形式化技术之一。
- 这样，FSM就在易于理解和易于使用之余还具备了严谨性，这也正是FSM以及它的后续技术得到广泛应用的重要原因。



有限状态机 FSM

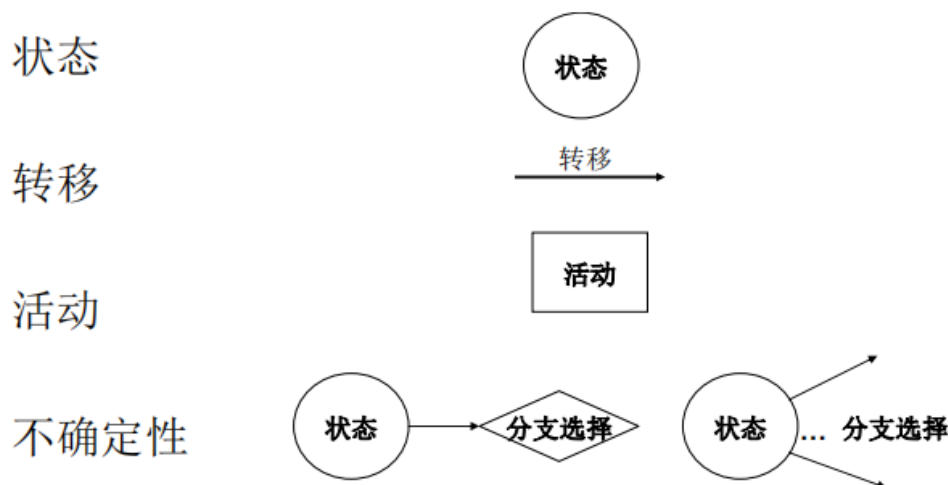
➤ 图示

- 为了更好的利用FSM描述系统的行为，人们为它定义了图形表示法，将FSM用直观的图形符号描述出来。
- 例如下图就是一个简单的FSM图示，它说明了一个电灯系统（灯泡和开关）的系统行为：



有限状态机 FSM

➤ 状态转移图STD是最为常见的FSM表示法



不确定性是指状态的转移情况不确定，需要依据一些其他的数据才能做出决策，即常说的分支选择。

- 前文的电灯系统的FSM图示使用的就是STD的图形表示法

有限状态机 FSM

➤ 状态转移矩阵STM也常常被用来进行FSM的描述

- STM的行表示了系统的状态列表。
- STM的列代表了可能的触发事件。
- 如果第i行第j列的单元格内容为k，则表示在状态i下如果发生事件j则系统从状态i转向状态k。
- 矩阵最后一列的action是特殊列，用来说明进入某个状态后要执行的活动。

有限状态机 FSM

➤ 常见的ATM取款机的STM描述

可能的触发事件

action

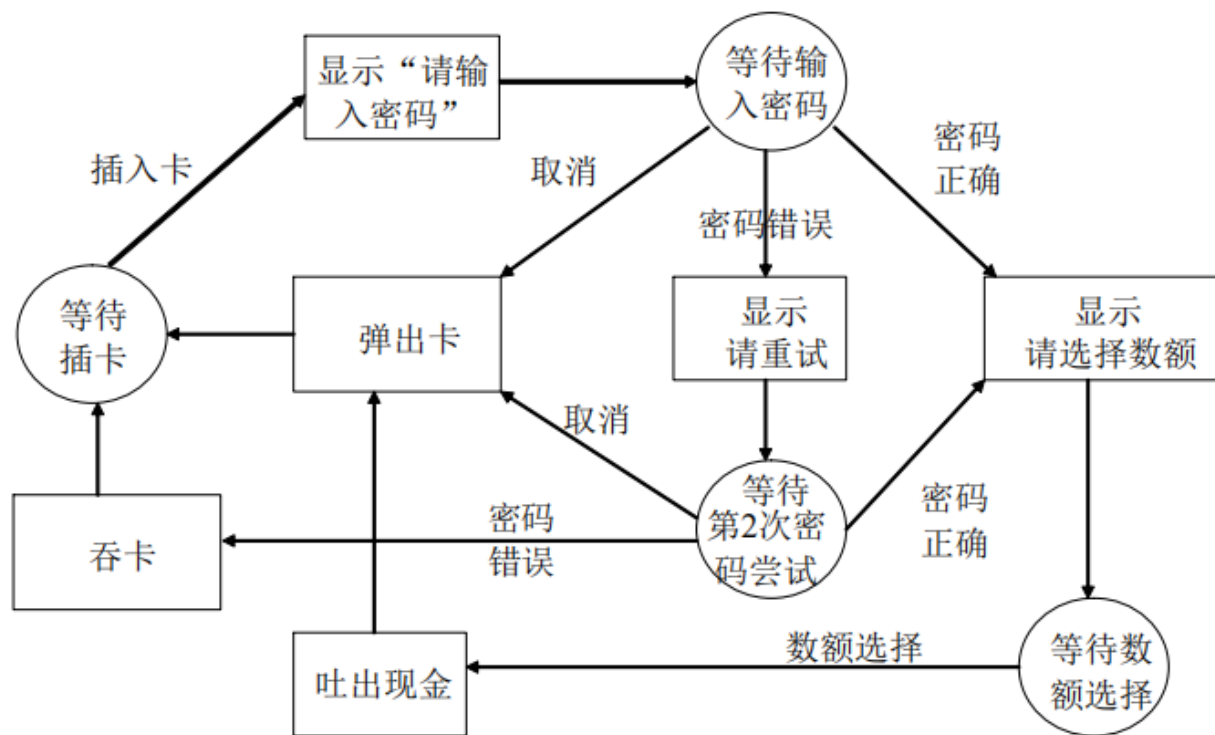
	卡插入	密码正确	密码错误	请求结束	输入数额	动作
1 等待插卡	2					显示 “请插卡 ”
2 等待输入密码		4	3	1		显示 “请输入密码 ”
3 等待二次输入密码		4	1	1		显示 “重试 ”
4 等待输入数额				1	1	显示 “输入数额 ”

系统的状态列表

状态2下发生密码正确则
系统从状态2转向状态4

有限状态机 FSM

➤与上述STM描述描述等价的STD描述如下



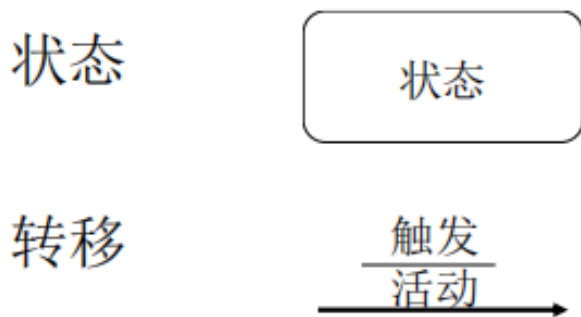
David Harel 的发展

- FSM有自己的优点，但是在实践当中人们也逐渐发现了FSM的局限性，尤其是它所固有的“平坦”（flat）性，即任何一对状态之间都可以通过一个转移来连接。
- 因此它不能适用于状态数量较多的系统，而这偏偏是复杂系统都会出现的。

David Harel 的发展

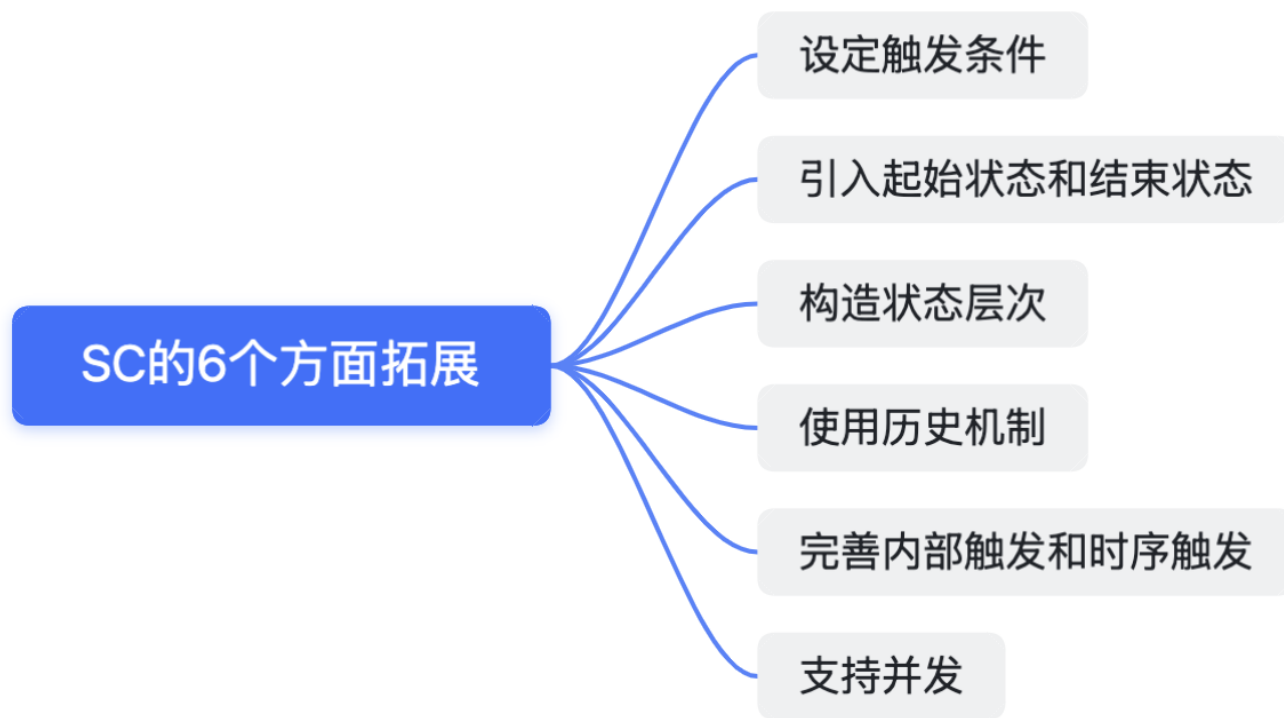
➤为了解决FSM的很多局限性，David Harel对FSM进行了扩展，建立了状态图SC（State Chart）表示法。

➤SC的FSM基本元素表示如图所示



David Harel 的发展

➤ 在FSM的基本元素之上，SC进行了6个方面的扩展



David Harel 的发展

➤ 设定触发条件

- 状态转移的触发是有条件触发，只有在满足条件的情况下，触发才是有效的。

$\frac{\text{Trigger(condition)}}{\text{Action}}$

$\frac{\text{Trigger[condition]}}{\text{Action}}$

➤ 引入起始状态和结束状态

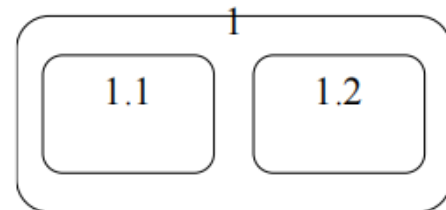
- 起始状态（左）表示了FSM的初始状态，结束状态（右）表示了FSM的结束状态集。按照FSM的定义，一个系统只能有一个初始状态，但可以有多个结束状态。



David Harel 的发展

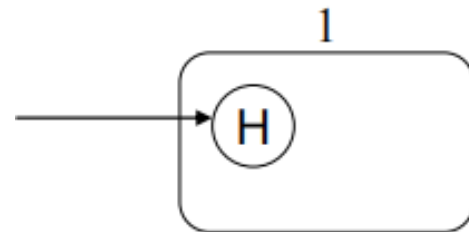
➤构造状态层次

- 在系统中引入了超状态（Super-State）的概念。超状态可以包含其他状态。这样，通过将一些联系紧密的系统状态包装成超状态，就可以对系统行为进行层次式的描述，减少了FSM“平坦”性所带来的图示复杂性。



➤使用历史机制

- 为了更好的构造状态层次，使用了历史机制。当转移返回到超状态中时，通常要返回到该超状态中最近经历过的子状态。



David Harel 的发展

Example 结合了状态层次和历史状态的状态示意图

- 开始触发事件 (T1) 导致进入状态1和子状态1.1

- 在状态1和子状态1.1时, 如果发生触发事件T2, 则进入状态1和子状态1.2

- 在状态1和子状态1.2时, 如果发生触发事件T3, 则进入状态2

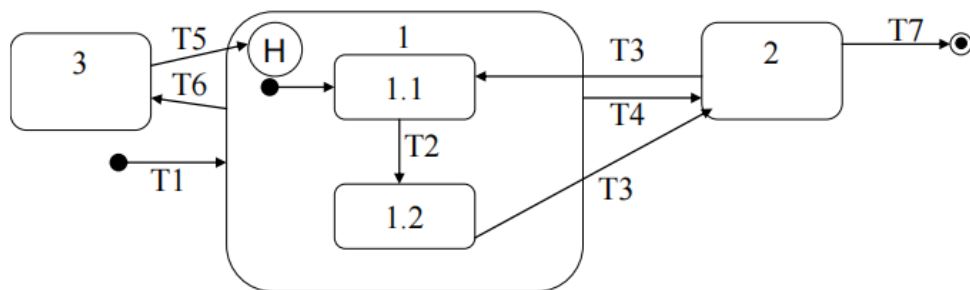
- 在状态1 (即子状态1.1或者1.2) 时, 如果发生触发事件T4, 则进入状态2

- 在状态1 (即子状态1.1或者1.2) 时, 如果发生触发事件T6, 则进入状态3

- 在状态2时, 如果发生触发事件T3, 则进入状态1和子状态1.1

- 在状态2时, 如果发生触发事件T7, 则状态机结束运行

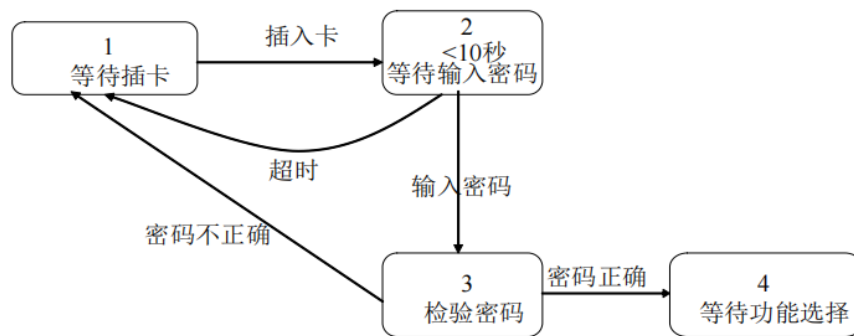
- 在状态3时, 如果发生触发事件T5, 则进入状态1和历史子状态, 历史子状态是上一次离开状态1时的子状态 (1.1或者1.2)。



David Harel 的发展

➤完善内部触发和时序触发

- 虽然FSM在理论上将触发事件分成外部事件、内部事件和时序事件三种类型，但FSM的图示法通常认为事件来自系统外部。
- 为了完善这一点，SC使用条件来表示内部事件，并通过在状态上添加时序限制来表示时序事件。
- 右图为SC的内部事件和时序事件表示的一个示例
- 其中条件“密码不正确 (PIN Invalid)”和“密码正确 (PIN Valid)”表示了两个内部事件，加在状态2上的限制“<10 second”表示了一个时序事件。



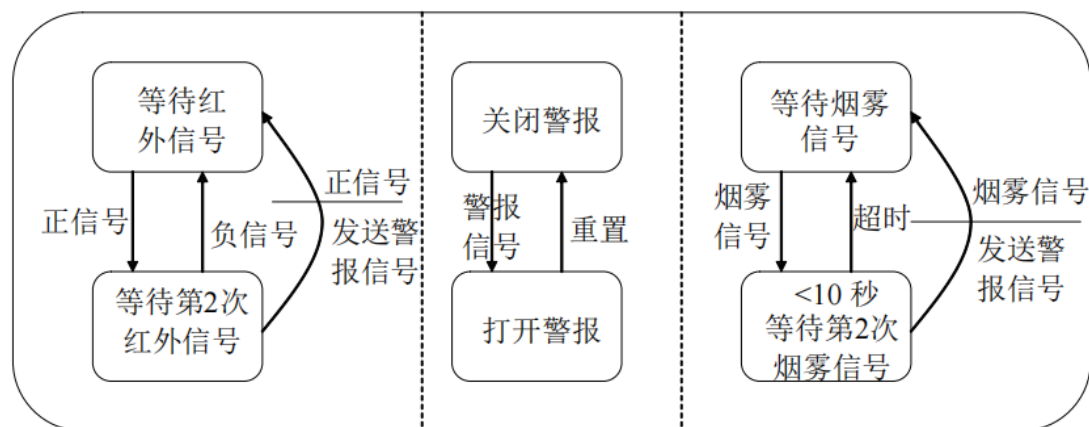
David Harel 的发展

➤ 支持并发

- FSM认为在任意时刻，系统都处于唯一的一种状态。
这样，FSM就拒绝了系统的并发性，因为在并发行为下系统的状态是不确定的。
- 为了支持并发，SC建立了并发的图示机制。**SC把并发表示在一个超状态中，并用虚线隔开**。不同并发之间的交互局限于触发器的传递和相互之间的条件测试。

David Harel 的发展

Example 一个并发的SC示例



- 左图描述了一个建筑内的火警系统。系统有三个并发的部分：红外探测器IR（左边）、烟雾探测器（右边）和警报器（中间）

- 红外探测器信号可分为正负两种情景，两个连续的正信号将触发警报。
- 烟雾探测器信号只取正值，如果在10秒内收到两个信号，那么也会触发警报。
- 警报器收到触发信号后就开始发出警报。复位信号将重置警报器。

UML的状态图

UML的状态图是在SC的基础上进一步发展而成的，它在SC的基础上，增加了新的扩展。

➤ 事件

- 状态图将事件进行了细分

类型	说明	表示法
调用事件	接收一个对象显示的同步的调用请求	op(a:T)
变化事件	对布尔表达式值的修改	when(exp)
信号事件	接收对象间显示的、命名的、异步的通信	sname(a:T)
时间事件	一个绝对时间的到达或者相对时间段的逝去	after(time)

- 事件类型的区分将传统状态机的事件分类进行了明确化：
 - 调用事件和信号事件是外部事件；
 - 变化事件是内部事件；
 - 时间事件是时序事件，把时间条件作为布尔表达式的变化事件也能描述时序事件。



UML的状态图

➤ 转移

- 传统状态机的转移是外部转移，会导致状态机从一种状态转换为另一种状态。**UML状态图增加了内部转移、入口转移、出口转移和do活动**

类型	说明	表示法
入口转移	进入某一状态时执行的入口活动	entry/activity
出口转移	离开某一状态时执行的出口活动	exit/activity
外部转移	对事件做出的响应引起状态变化或自身转换，同时引发一个特定的效果。如果离开或进入状态也可能引起状态的入口转换或出口转换	e(a:T)[guard]/activity
内部转移	对事件做出的响应并引发执行一个特定的效果，但是并不引起状态变化或入口转换、出口转换的执行	e(a:T)[guard]/activity

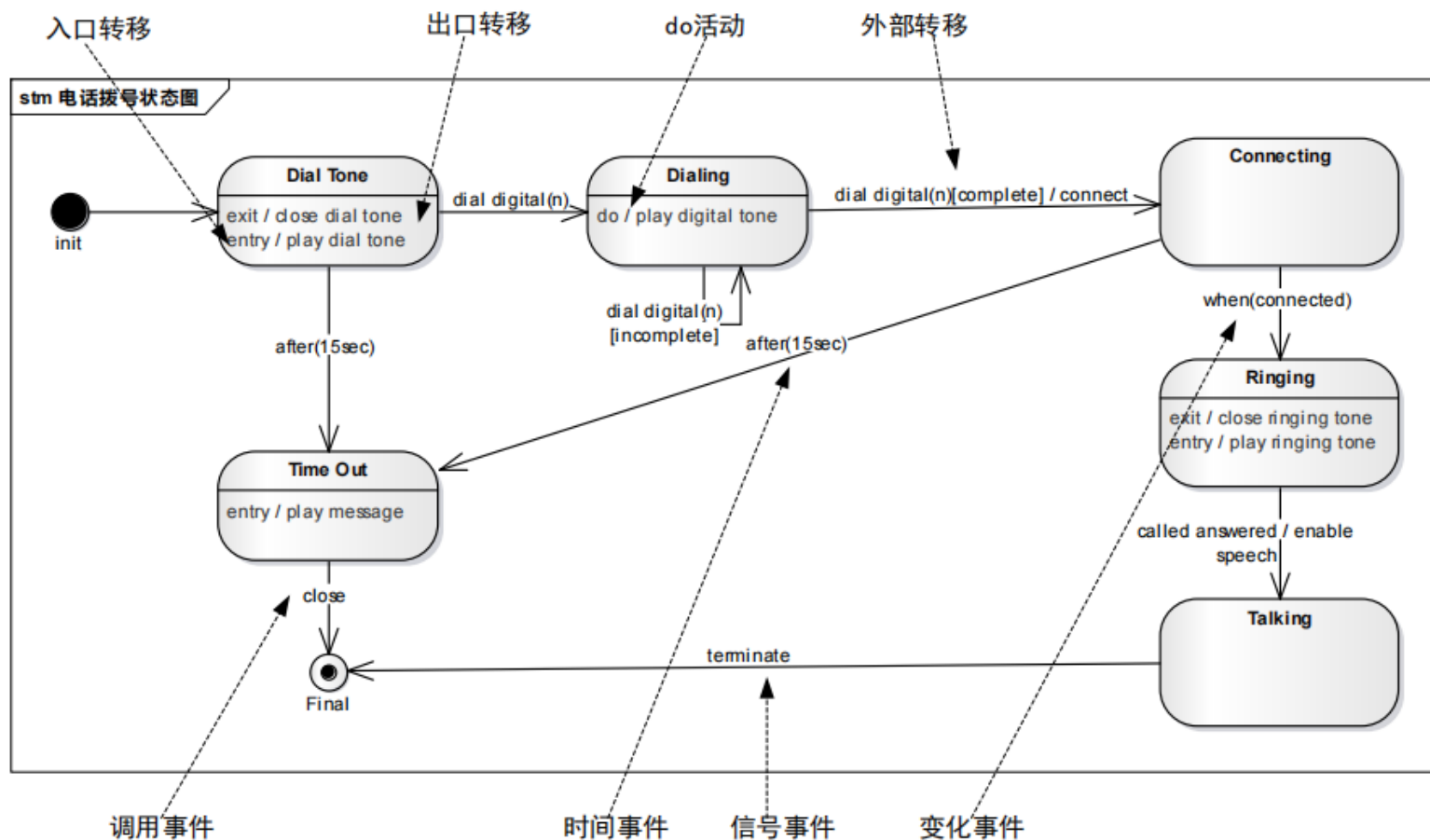
UML的状态图

类型	说明	表示法
do活动	do活动在进入活动之后执行,并且一直运行到它本身完成为止或者状态机退出指定状态	do/activity
组合转移	将转换的片断串联成一个单一的“运行至完成”的转换	●

- 内部转移会导致状态机内部发生变化,但是变化的程度不足以导致旧状态消亡和新状态产生,也就是说**内部转移会在整体状态不变的情况下改变状态内部数据或行为。**
- 当状态转移发生时,从旧状态退出、进入新状态,**退出旧状态时发生的转移称为出口转移,进入新状态时发生的转移称为入口转移。**
- **do活动是一种特殊活动,进入活动之后执行,并且一直运行到它本身完成为止或者状态机退出指定状态。**

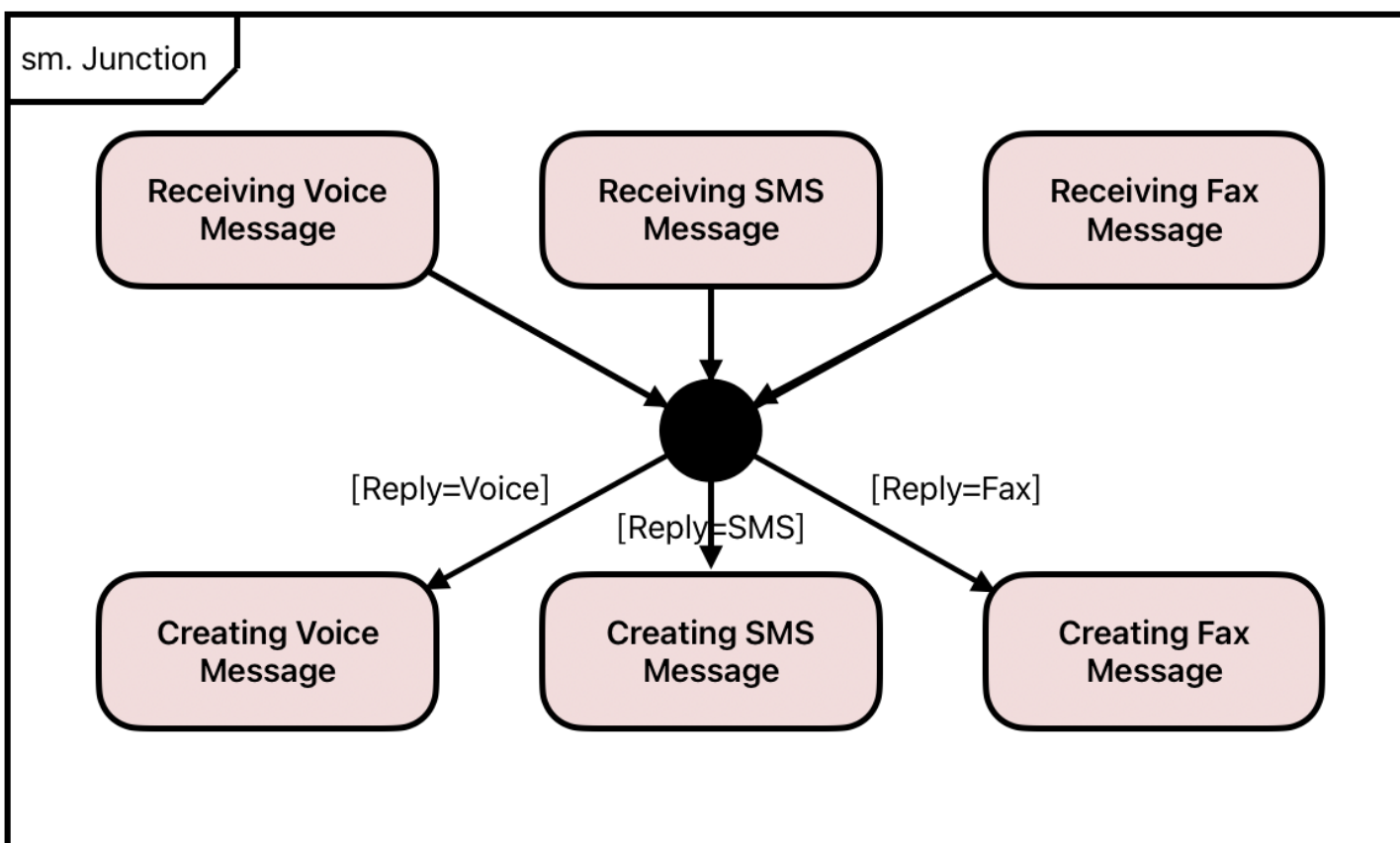
UML的状态图

- 描述电话拨号过程的事件和转移的示例



UML的状态图

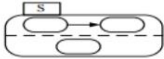
- 组合转移由简单转移组成，这些简单转移通过分支判定、分叉或接合组合在一起



UML的状态图

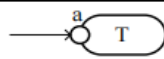
➤ 状态

- UML的状态图吸收了David Harel在SC中发展的所有状态概念

类型	说明	表示法
简单状态	没有子结构的状态	
初始状态	起点状态	
结束状态	表明活动已完成	
类型	说明	表示法
并发状态	被分成两个或多个区域的状态，当复合状态被激活时，每个区域中的一个直接子状态被并发激活	
组合状态	包含一个或多个直接子状态，当复合状态被激活时，只有一个子状态会被激活	
选择	在运行至完成的转换中起动态分支的作用	
历史状态	它被激活时会把复合状态还原成之前被激活时的状态	

UML的状态图

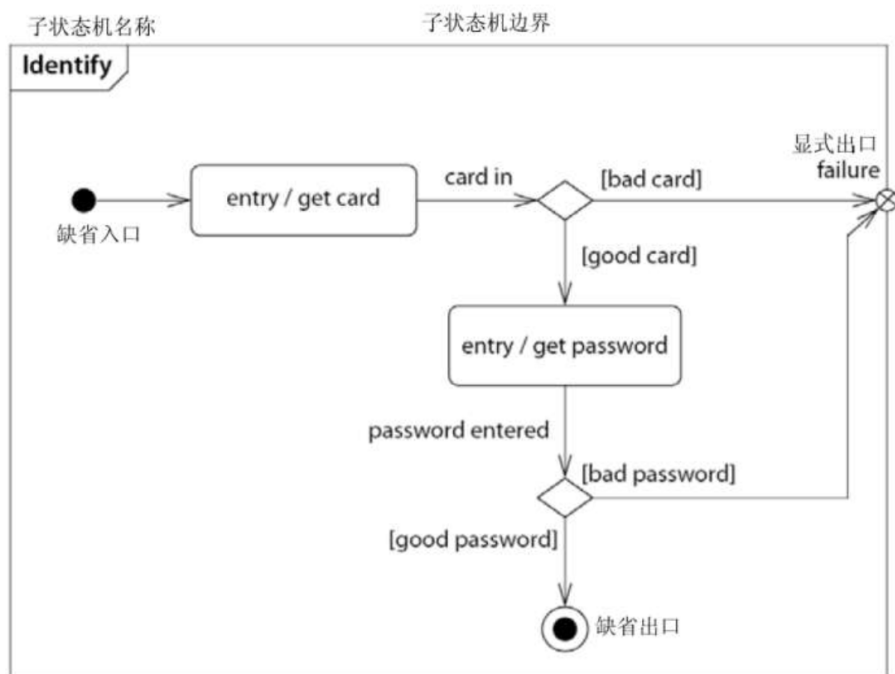
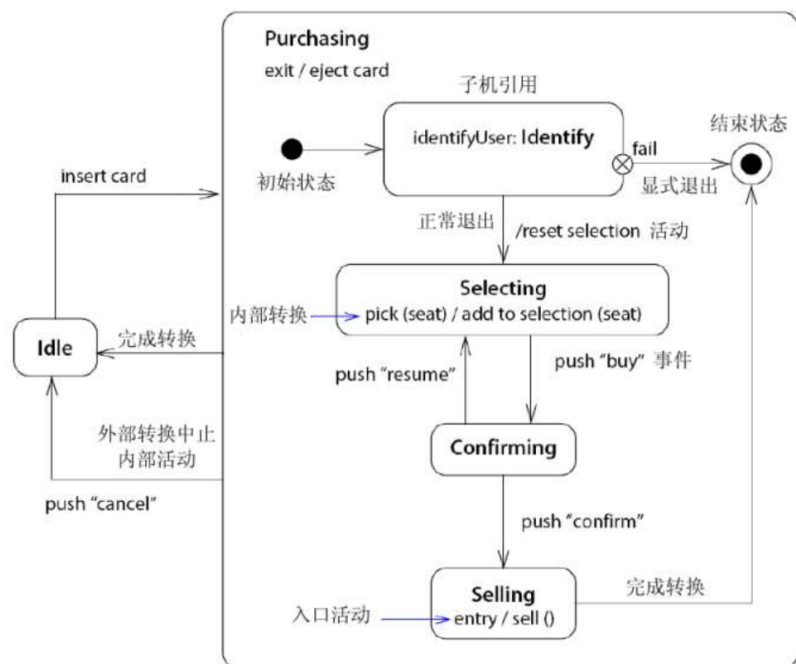
- UML状态图发展了几个新的状态概念：

类型	说明	表示法
终止	终止状态机所描述的对象运行	
子机状态	引用其他状态机的连接，被引用的状态机可以在概念上替换该子机状态	
入口点	标识出一个内部状态作为目标	
出口点	标识出一个内部状态作为源	

- “终止”类似于活动图的“流终结”，表示状态机的异常终止。
- “子机状态”类似于活动图的“活动”元素，可以建立状态图的层次结构。“子机状态”的层次结构与“组合状态”不同，它以引用的方式简化了父状态图，类似于活动图对“活动”元素的使用。

UML的状态图

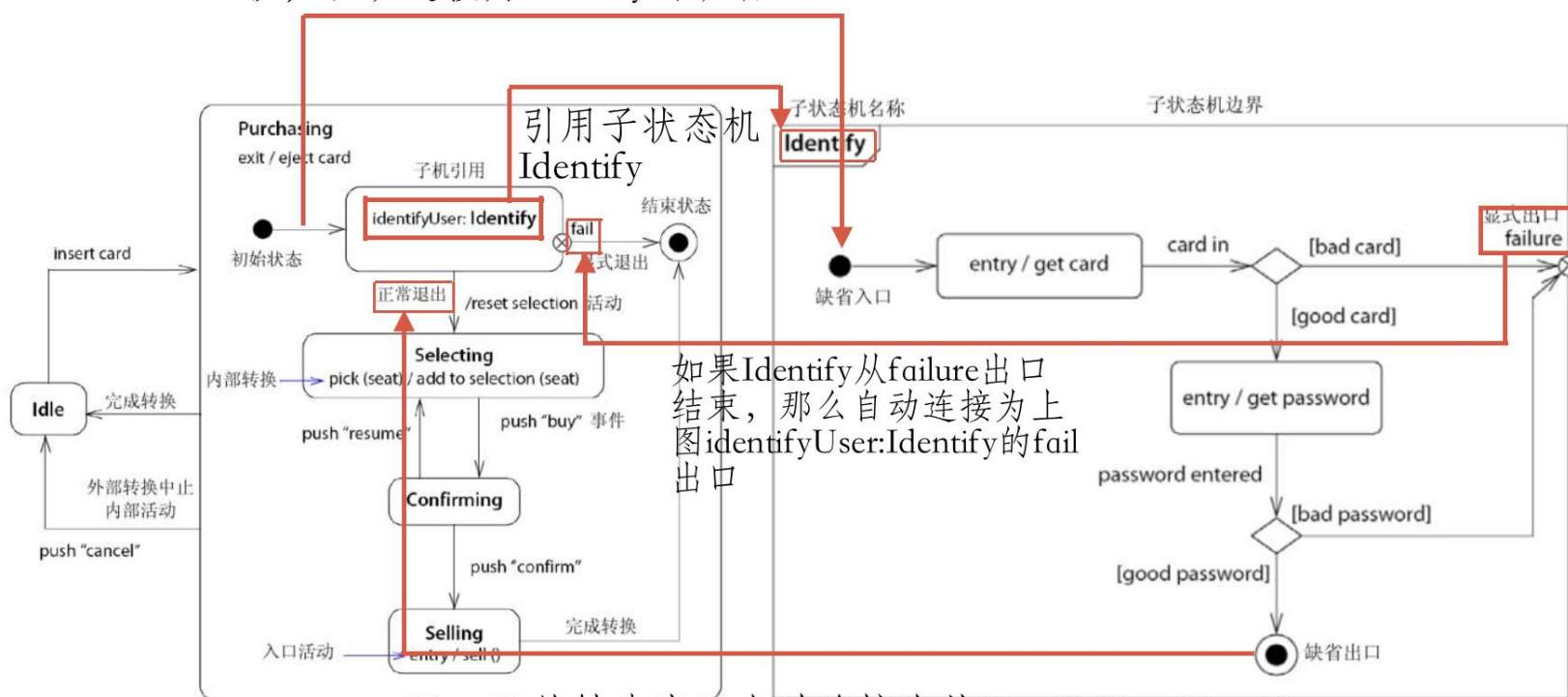
➤一个自动售票系统的行为状态机



UML的状态图

➤一个自动售票系统的行为状态机

从初始状态到identifyUser:Identify的转移，自动连接为Identify的缺省入口



Identify定义了一个出口点 failure，出口点与缺省出口的区别是：缺省出口是子状态机的成功结束出口；出口点是子状态机的异常结束出口。

Identify的缺省出口自动连接为从identifyUser:Identify向Selecting的“正常提出”转移

UML的状态图

➤ 状态图的应用

- UML的状态图是以状态机的方式描述系统的行为，
依据对“系统”范围的不同定义，它可以描述不同的方面：
 - 如果以整个系统为“系统”，那么它描述的就是整个系统的行为
 - 如果以一个或者几个用例为“系统”，那么它描述的就是一个或者几个用例所包含的行为
 - 如果以一个对象为“系统”，那么它描述的就是一个对象的行为

UML的状态图

➤ 状态图的应用

- UML的规范里面，认为状态图的主要用法是描述对象的行为，系统中往往会涉及一些重要而且复杂的对象，它们的行为会出现在很多的用例当中，这时就可以使用状态图，从众多用例当中抽取出这些对象的行为进行集中的描述
- 在面对一些特别复杂的用例（比如拥有很多复杂的场景）时，状态图也是一个比交互图更为好用的描述手段

目录

- 面向对象分析与UML
 - 对象模型
 - 建立领域模型
 - 行为模型--交互图
 - 建立交互图
 - 行为模型--状态图
 - 建立状态图
 - 对象约束语言OCL
 - 基于CRC卡面向对象分析方法
 - 需求细化
- 基于状态转移矩阵建立状态图
- 状态图的分析作用

基于状态转移矩阵建立状态图

① 确定上下文环境

- 搞清楚状态的主体，确定状态的上下文环境。
- 常见的状态主体有：类、用例、多个用例和整个系统。

② 识别状态

- 识别状态主题可能的稳定状态，并且标记出其中的初始状态和结束状态集。
- 在有些情况下，可能会不存在确定的初始状态和结束状态。

③ 建立状态转换

- 根据需求所描述的系统行为，建立各个稳定状态之间可能存在的转换。

④ 补充详细信息，完善状态图

- 添加转换的触发事件、转换行为和监护条件等详细信息。
- 在有些情况下也可能需要建立状态图的层次结构或者进行其他更加复杂的工作。

基于状态转移矩阵建立状态图

➤ 示例：产品销售

① 明确状态图的**主体**：类POS。

② 识别POS可能存在的**稳定状态**：

- 授权状态：POS机已经准备就绪，但还未参加到工作中来的状态；
- 空闲状态：POS可以参与工作的执行，但并没有工作正在进行的状态；
- 销售开始状态：开始一个新销售事务，系统开始执行一个销售任务的状态；
- 商品信息显示状态：刚刚输入了一个物品项，显示该物品描述信息的状态；
- 错误提示状态：输入信息错误的状态；
- 列表显示状态：以列表方式显示所有已输入物品项信息的状态；
- 销售结束状态：收银员指令POS结束正在处理的已有的销售，打印收据。

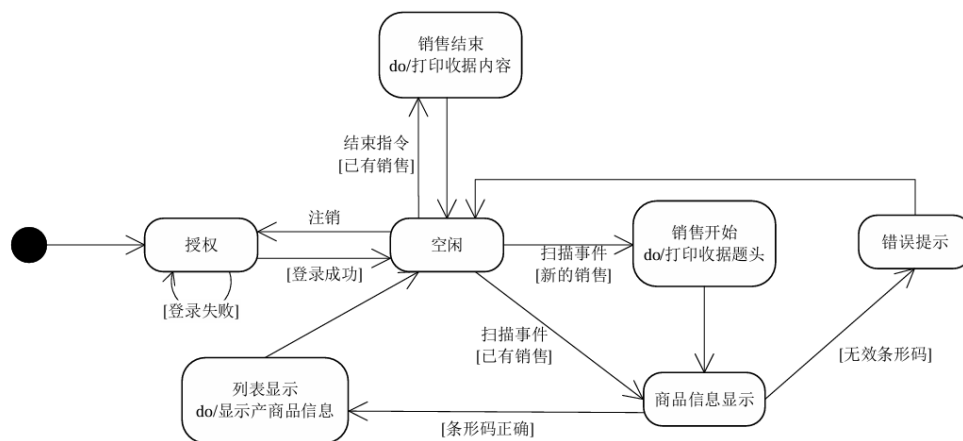
基于状态转移矩阵建立状态图

➤ 示例：产品销售

③ 建立状态转换

	授权	空闲	销售开始	商品信息显示	错误提示	列表显示	销售结束
授权	Y	Y					
空闲	Y		Y	Y			Y
销售开始				Y			
商品信息显示					Y	Y	
错误提示		Y					
列表显示		Y					
销售结束		Y					

④ 在已识别状态和转换的基础上，添加详细的信息说明，建立状态图



状态图的分析作用

- 如果状态图中发现有无法**进入**的状态或者无法**跳出**的状态，就意味着**相应行为的缺失**。
- 如果一个状态在发生触发时**转移路线不确定**，就意味着监护条件**数据缺失**或者**行为需要细化**。
- 如果应该建立的**状态转移**在需求内容中**没有体现**，就需要**修正需求内容**。
- 如果应该**存在的状态**在需求内容中**没有体现**，也需要**修正需求内容**。

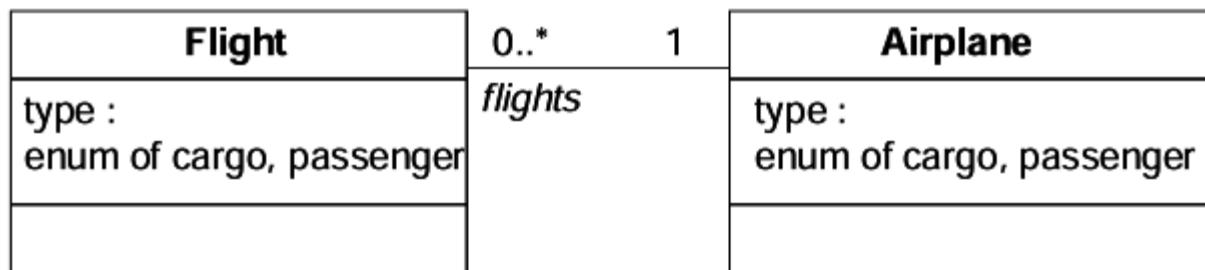
目录

- 面向对象分析与UML
- 对象模型
- 建立领域模型
- 行为模型--交互图
- 建立交互图
- 行为模型--状态图
- 建立状态图
- 对象约束语言OCL 
 - 概述
 - OCL的应用
- 基于CRC卡面向对象分析方法
- 需求细化



概述

- UML使用图形语言来进行系统数据和行为的描述，建立在一定的**模型基础**之上的，可以更加精确进行信息的描述。
- 但是，底层模型的语法和语义**限制**往往又使得它们无法表达足够丰富的内容，描述系统的各个方面。
 - 例子：下面的类图中，客机航班对应客机，货机航班对应货机无法体现。



- 类图中引入**自然语言**可以解决，但是会降低UML的精确性
- OMG在UML的1.1版本中采纳了由IBM工程师建立的对象约束语义**OCL (Object Constraint Language)**

概述

- OCL并不是UML中单独的一个模型，而是被应用在其他模型当中，**丰富**其他模型的**语义**。
- OCL是一种**无副作用的规约语言**
 - 以表达式的方式**定义**对其他模型元素的**约束**
 - **约束和限制**其他模型元素的行为和状态**变化**
 - **不会修改**任何其他模型元素的**表述**
- 在实践中，使用OCL能**表达其意**即可，不展开详细定义



OCL的应用

➤OCL在UML模型中有很多用途

➤最常见的是用来定义UML模型元素的**四类约束**:

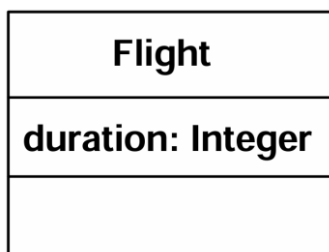
- 不变量 (Invariant)
- 前置条件 (Precondition)
- 后置条件 (Postcondition)
- 监护条件 (Guard) 。

OCL的应用

► 不变量：可以对UML类元施加的约束

- 类元需要保持它的表达式取值在指定的时间范围内或者指定的条件下始终为“真”
- 可以用于UML的多种类元之上，最常见的是用来约束类的属性或者类的方法

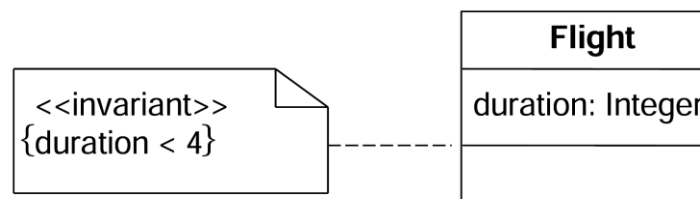
例子：航班Flight类，要限制Flight的时间小于4（小时）。可以使用表达式描述，也可以在UML图中添加特殊注释，两者完全等价



Flight类

context Flight inv:
duration < 4

OCL表达式描述



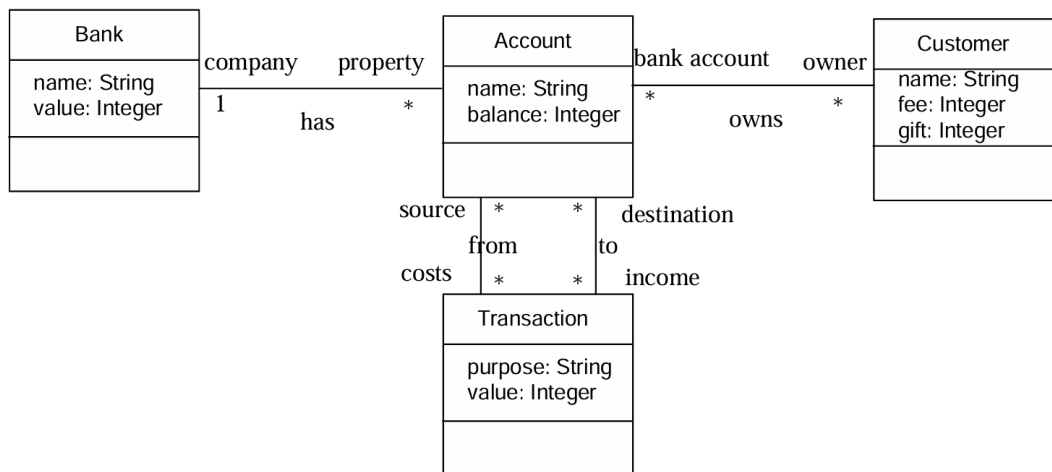
UML图中添加特殊注释

OCL的应用

➤前置条件和后置条件：可以对类元的操作施加的约束

- 类元需要保持它的表达式取值在指定的时间范围内或者指定的条件下始终为“真”
- 可以用于UML的多种类元之上，最常见的是用来约束类的属性或者类的方法

例子：下图所示的类图描述了银行的交易处理工作。现在需要添加一个约束为：如果一个顾客的交易总次数超过了30次，那么就在顾客执行新交易时赠与10点的赠品。



OCL的应用

```
context Customer :: getGift() : Integer
```

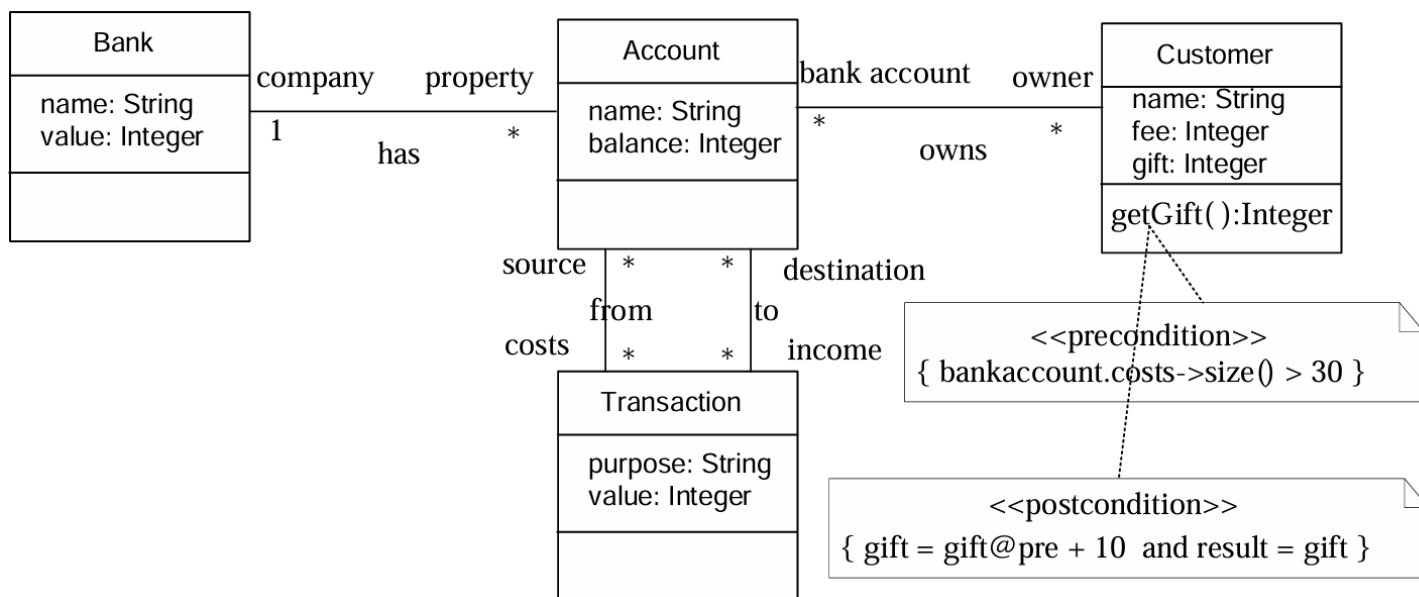
```
preCustStartTransact :
```

```
bankaccount.costs → size() > 30
```

```
postTenDollarsGift :
```

```
gift = gift@pre + 10 and result = gift
```

Customer::getGift() : Integer定义了一个前置条件CustStartTransact和一个后置条件TenDollarsGift。
pre和post是OCL定义前置条件和后置条件的关键字。

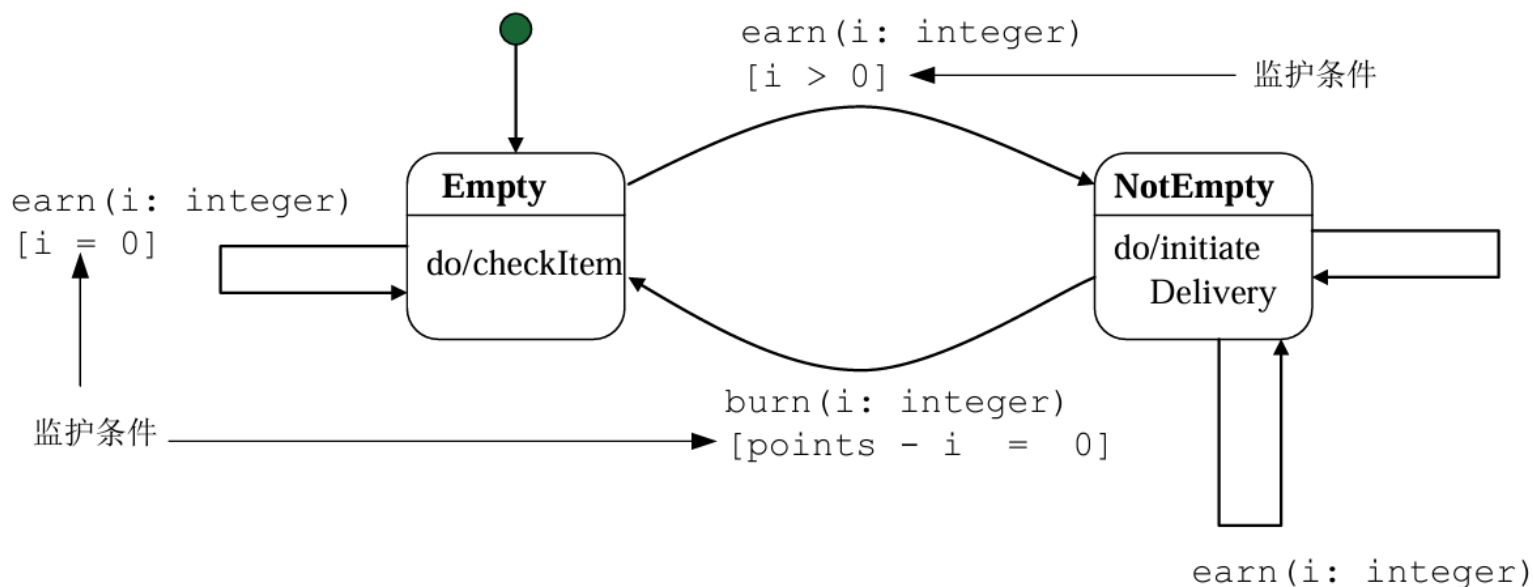


OCL的应用

➤ **监护条件**：是对状态机模型中**状态转移**施加的约束

➤ 在状态机到达转移点时，监护条件的表达式需要根据实际状态进行评估，并只有在表达式实际取值为“真”的情况下才进行转移。

例子：监护条件约束被直接表示在状态图之上，放在转移语句之后，用“[]”进行界定



目录

- 面向对象分析与UML
- 对象模型
- 建立领域模型
- 行为模型--交互图
- 建立交互图
- 行为模型--状态图
- 建立状态图
- 对象约束语言OCL
- 基于CRC卡面向对象分析方法
- 需求细化

CRC卡

基于CRC卡的职责驱动方法



概述

- 前面描述的面向对象分析方法适用于简单情况下的建模方法。需求信息比较明确时，能够发挥很好的作用。
- 在复杂情况下，需求的获取和分析是交织前进的，在进行分析与建模时并没有非常明确和固定的需求信息可资利用。前面描述的建模方法就很难起到好的效果。
- 基于CRC卡的职责驱动方法就是一种用来处理复杂情况的面向对象建模方法，它和需求的获取活动互相交织，齐头并进。
- 基于CRC卡的职责驱动方法在实际开发中的应用要比前面所述的面向对象建模方法复杂得多，需要更多的实践经验和技巧。所以在面对简单的应用时，还是推荐使用前面的简单方法。
- 基于CRC卡的职责驱动方法在应用的细节上包含有很多复杂的经验总结和启发式规则，这里仅介绍它的基本思想和大概框架。



CRC卡

- CRC是**Candidates**、**Responsibilities**和**Collaborators**三者的缩写。基于CRC可以建立一种索引卡片，被称为CRC卡。
- Kent Beck和Ward Cunningham于1988年发明的**设计方法**，用于描述早期的设计思想。后来，因为使用起来方便有效，在面向对象开发当中得到了大量的应用。
- 每个卡片代表了一个**被发现的候选对象**。
- 形式可能是**多种多样的**，卡片、纸张、黑板等等都可以作为CRC卡的介质载体。

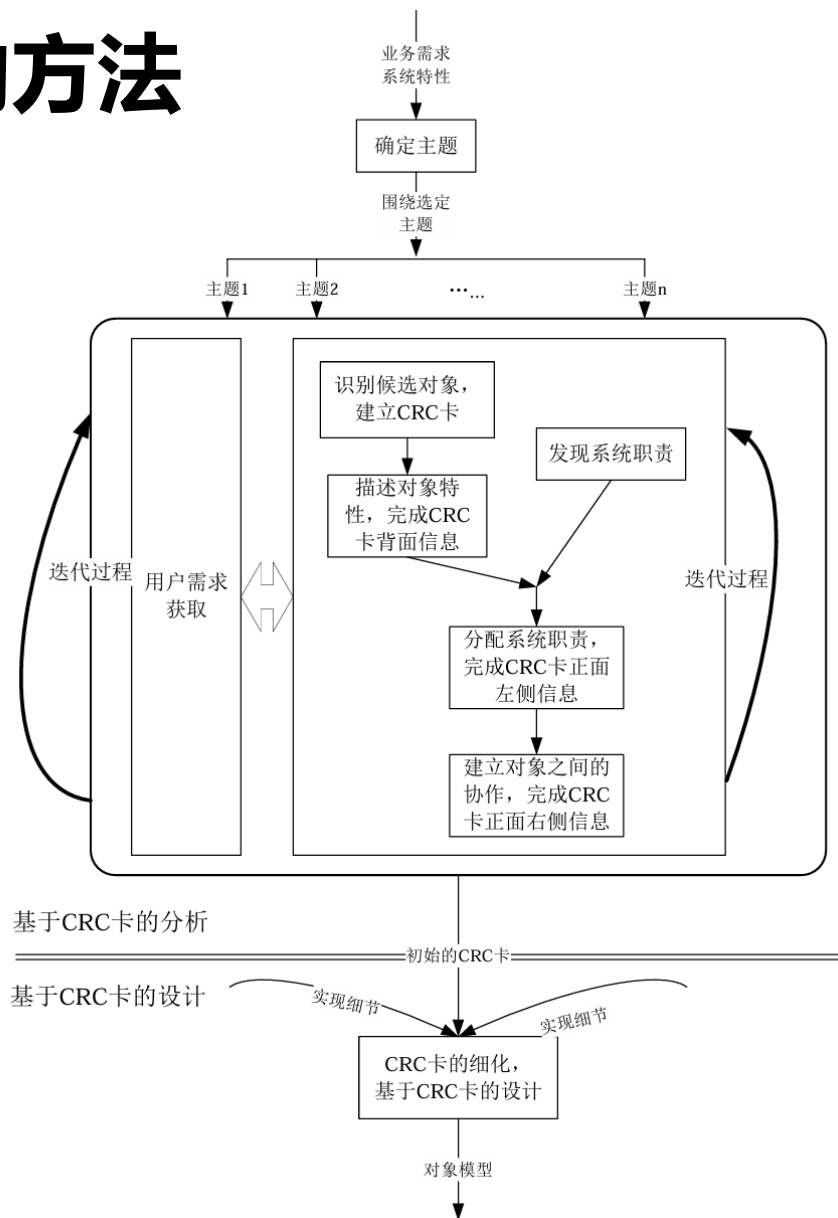
CRC卡



- CRC卡简洁方便，可以随时被移动、修改或者丢弃，所以它特别适合于在复杂的系统当中进行**对象的发现**和**设计思想的挖掘**，即进行**复杂情况**下的面向对象分析与设计。

基于CRC卡的职责驱动方法

- 通过对CRC卡的**建立、描述、修改和完善**等行为，可以为复杂的系统最终建立有效的对象模型。
- 这种面向对象的建模方法被称为基于CRC卡的**职责驱动方法**，它被广泛的用于面向对象的分析与设计。
- 在需求分析阶段，基于CRC卡的职责驱动方法的主要工作框架如右图所示。



基于CRC卡的职责驱动方法

① 确定主题

- 面对复杂的系统应用，基于CRC卡的职责驱动方法首先需要分析系统的**背景信息**，找出系统中重要的**主题（Topic）**。每个主题意味着系统工作的一个**重要部分**，整个系统可以按照主题被划分成有机的不同部分。
- **业务需求**和为满足**业务需求**而规划的**系统特性**是用来确定主题的最好信息。通常可以围绕**业务需求**组织主题，或者将每一个系统特性都限定为一个主题。
- 每个**主题内部**的系统功能是**紧密相关**的，而不同主题之间的功能相关性就会大大降低。因此，主题的确定是利用**模块化的方式**在工作的初始阶段就降低问题处理**复杂度**。

基于CRC卡的职责驱动方法

② 识别候选对象

- 一旦确定了系统的主题，就可以围绕主题进行**候选对象的识别**。
- 识别候选对象时可以从下列几个方面的内容着手：
 - 问题域中重要的对象和结构；
 - 系统的控制和协调行为；
 - 需要被传递的重要信息流；
 - 关联硬件和其他的关联系统。
- 识别出候选对象之后，为它们**分别建立CRC卡**，并给以合适的**命名**。

基于CRC卡的职责驱动方法

③ 描述对象特性

- 对**识别**出的**每个候选对象**，需要在CRC卡的背面进行描述。
- 可以依据下面几个特性来进行对象的描述：
 - 在系统中的地位；
 - 功能行为；
 - 和其他对象的关系；
 - 公共责任；
 - 抽象级别；
 - 复杂度。

④ 发现系统职责

- 职责分为**状态的维护**和**行为的履行**两个部分，所以对职责的发现要注重两个方面的内容：
 - 一、系统所需要和维护的信息；
 - 二、系统的功能和行为。



基于CRC卡的职责驱动方法

⑤ 分配系统职责

- 发现的**系统职责**需要被分配给已发现的**候选对象**，完成CRC卡正面左侧的信息描述。
- 职责分配时要注意下列原则：
 - 集中信息与行为。
 - 维持对象的角色。
 - 保持对象责任的相关性。
 - 保持职责的合适粒度。
 - 保持对象的粒度。
 - 不要重复责任。
 - 必要的时候调整候选对象。



基于CRC卡的职责驱动方法

⑥ 建立对象之间的协作

- 在完成职责的分配之后，就可以着手进行对象之间协作的建立工作，完成CRC卡正面右侧的信息描述。
- 可以从下面几个角度出发建立对象之间的协作：
 - **角色**。角色表明对象在系统和具体工作中的**位置**。在系统和工作中的相邻或者相关的角色之间通常会进行协作，也就是说对象所扮演的角色隐含着特定的协作关系。
 - **任务**。识别任务中的参与对象，分析任务中**责任的分解、衔接和分配**，这可以帮助发现对象之间的协作。
 - **职责**。分析一个对象的职责（尤其是对象**所维护的信息**）也可以帮助发现对象之间的协作，这些协作体现了对象之间的帮助关系。

目录

- 面向对象分析与UML
 - 对象模型
 - 建立领域模型
 - 行为模型--交互图
 - 建立交互图
 - 行为模型--状态图
 - 建立状态图
 - 对象约束语言OCL
 - 基于CRC卡面向对象分析方法
 - 需求细化
- 基于需求分析模型细化需求
细化需求实例
描述细化的需求



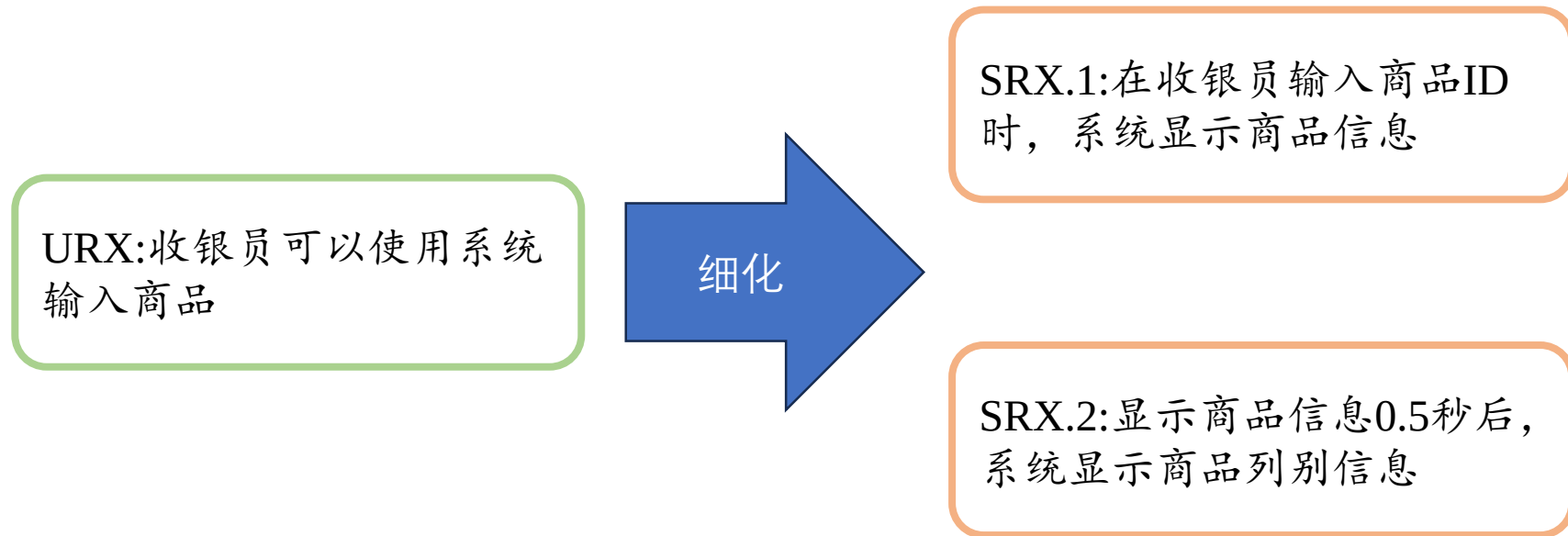
基于需求分析模型细化需求

- **用户需求**需要被细化为**系统级需求**，这个工作以**需求分析**为基础，在需求分析模型中推敲并描述**解决方案细节**，解决方案每一次与外界交互中涉及的细节描述就是系统级需求。
- 需求细化的根本是将从**问题域和业务任务**角度描述的用户需求转换为从**软件和技术**角度描述的系统级需求。在转换过程中：
 - 需要将用户需求**细化**。
 - 需要**补充**隐含因素。
 - **非功能需求**也需要随着功能需求的细化而细化。
 - 会发现新的**细节**。
 - 在足够具体时**停止**。

基于需求分析模型细化需求

➤ 需要将用户需求细化。

➤ 用户需求是以任务为单位的，但一个任务的完成可能需要进行多次的系统交互。



基于需求分析模型细化需求

➤ 需要补充隐含因素。

- 用户需求只是一种**期望**，并不直接包含问题域信息（虽然理解用户需求需要先理解问题域信息）。
- 但系统级需求是解决方案中的一个**系统行为**，需要包含相应的**问题域信息**。
- 所以在细化需求时，需要将问题域信息**补充**到系统级需求中。这些需要补充的问题域信息就是用户表述需求时隐含的因素。

补充到系统级需求的描述中，或单独描述为数据需求：

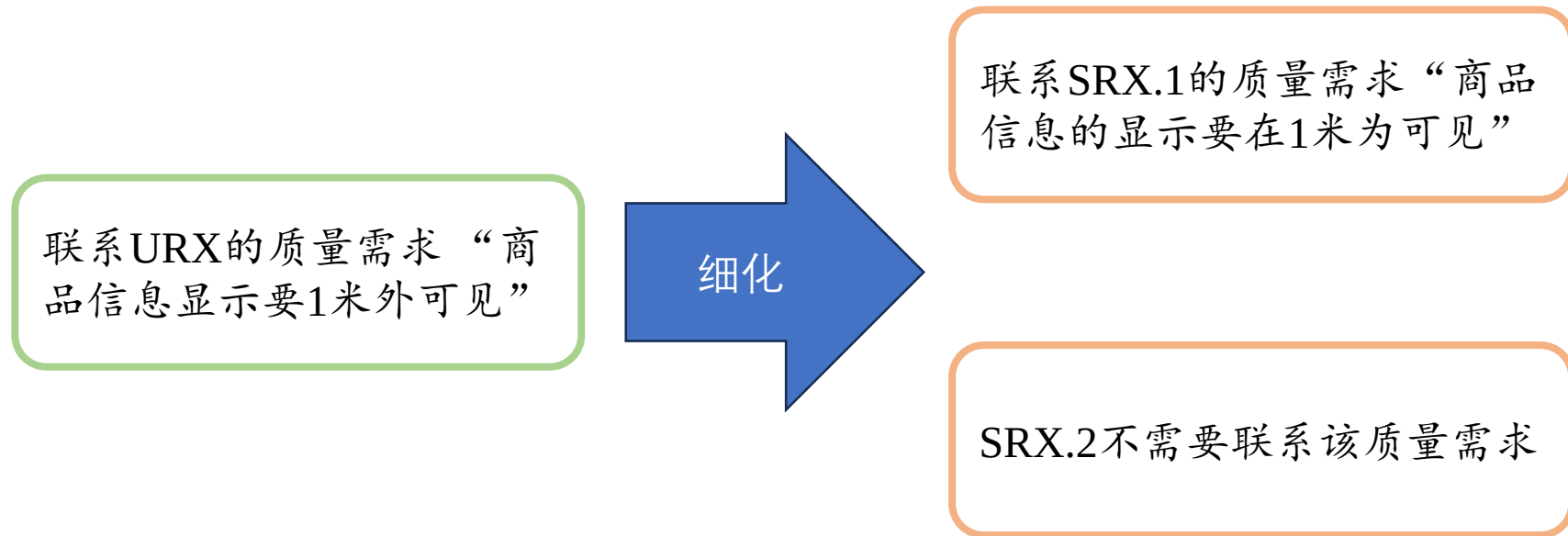
需要显示的商品信息包括价格、名称、描述；

商品ID是符合相关规定的条形码；

...

基于需求分析模型细化需求

➤非功能需求也需要随着功能需求的细化而细化。



基于需求分析模型细化需求

➤ 会发现新的细节

- 细化的过程包含解决方案的创建过程，其中可能会发现一些**新的细节**，这些细节往往是为了**约束和限定**解决方案的实现手段。

例如，在细化URX时，可以发现：

1. 输入商品ID时可以使用扫描仪，也可以使用键盘；
2. 收银员可以逐一输入多个同样的商品，也可以输入商品ID后紧跟着输入数量。

这两个**新的细节需求**。

基于需求分析模型细化需求

➤在足够具体时停止

- 在**需求工程师**相信需求已经**足够具体**的时候，需求细化的工作就可以停止了。
- 这里的“足够具体”意味着需求已经得了**充分的理解**，并且开发者已经可以着手为其进行方案设计了。
- 切记不可将需求进行**过分的细化**，以至于在其中包含了很多设计或者实现的因素，这会**限制**设计人员和实现人员的工作空间。

细化需求实例

➤ 超市销售处理用例的系统顺序图，包含了系统与外界的基本交互序列：

刺激1：收银员输入会员的客户编号

响应：系统标记销售任务的会员

刺激2：收银员输入商品标识

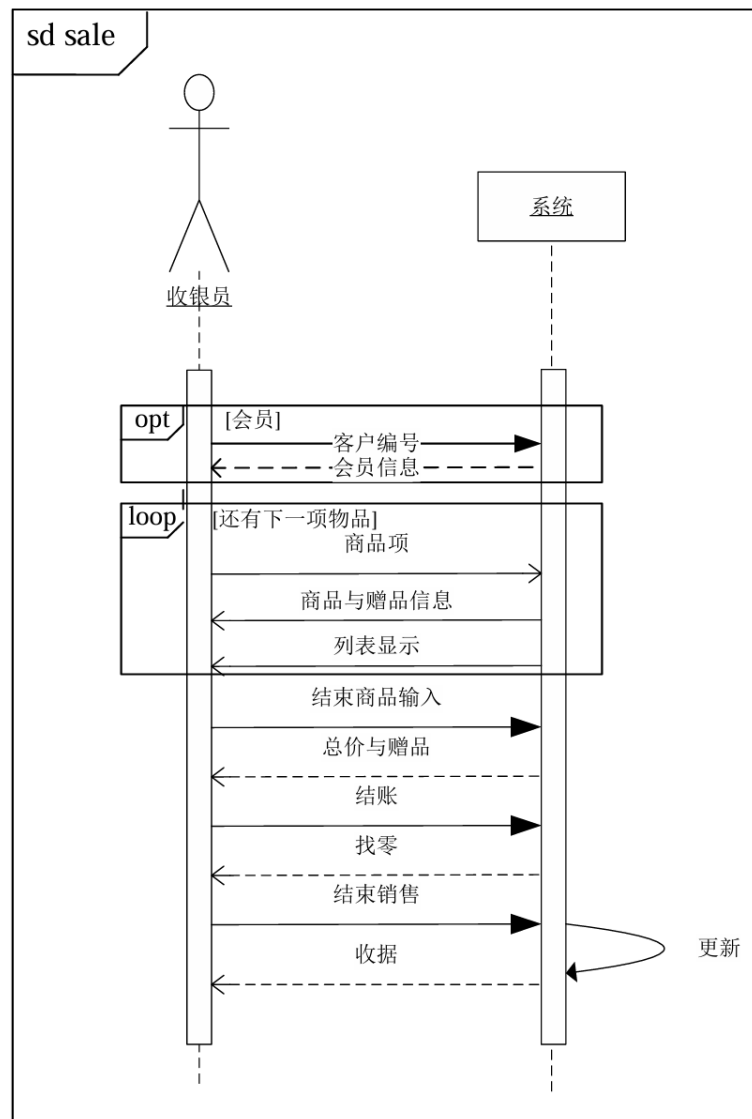
响应：系统显示商品信息，计算价格

刺激3：收银员要求结账，输入付款信息

响应：系统计算账款，显示赠品、找零

刺激4：收银员取消销售任务

响应：系统关闭销售任务

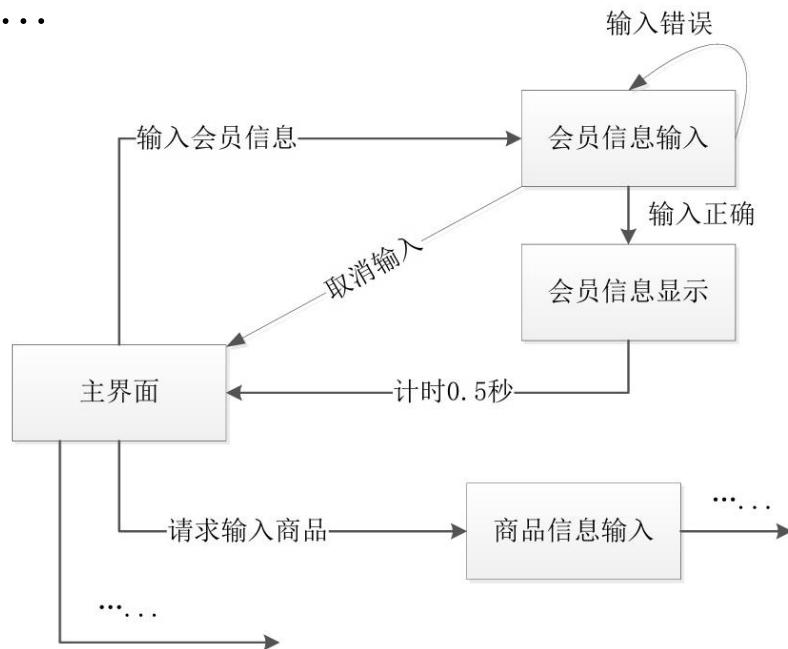


超市销售处理用例的系统顺序图

细化需求实例

➤对于上述交互序列，可以给出一个解决方案如图所示：

- 假设发生了刺激1，解决方案进入了会员信息输入状态；
- 再考虑到人的行为具有**主动性特点**（不规律和无法控制），所以还要设计“输入正确”、“输入错误”和“取消输入”3种后续的刺激……



依据用例明确解决方案细节示例

细化需求实例

➤再结合领域类图、业务规则等相关的知识，就可以将销售用例涉及的部分用户需求转换为下表所示的部分系统级需求。

销售用例的系统级需求（部分）

Sale.Input	系统应该允许收银员在销售任务中进行键盘输入
Sale.Input.Member	在收银员请求输入会员客户编号时，系统要标记会员，参见Sale.Member
Sale.Input.Goods	在收银员输入商品目录中存在的商品标识时，系统执行商品输入任务，参见.....
Sale.Input.Invalid	在收银员输入其他标识时，系统显示输入无效
Sale.Member.Start	在销售任务最开始时请求标记会员，系统要允许收银员进行输入
Sale.Member.Notstart	不是在销售任务最开始时请求标记会员，系统不予处理
Sale.Member.Cancle	在收银员取消会员输入时，系统关闭会员输入任务，返回销售任务，参见Sale.Input
Sale.Member.Valid	在收银员输入已有会员的客户编号时，系统显示该会员的信息
Sale.Member.Valid.List	显示会员信息0.5秒之后，系统返回销售任务，并标记其会员信息
Sale.Member.Invalid	在收银员输入其他输入时，系统提示输入无效
.....	

描述细化的需求

➤ 细化后的需求应该被一一的**标识和记录**下来。这就要求在分析活动当中收集每条需求的**重要属性**，常见的属性有：

- 标识符（ID），每一条需求都应该能够通过ID唯一的标识自己。
- 源头（Source），要能够回溯到需求的源头，例如特定的涉众。
- 理由（Rational），需求被提出的目的。
- 优先级（Priority）。
- 成本（Cost），预估的实现成本。
- 风险（Risk），实现该需求的过程中可能带来的风险。
- 可变性（Volatility），将来发生变化的可能性。

确定需求的属性可以更好地在整个软件开发过程当中进行需求的管理。

本章小结

- 面向对象方法的思路是将分析与设计划分为四个阶段：需求获取、需求分析、软件设计和软件实现。统一建模语言UML是人们综合不同面向对象方法分支建立统一的面向对象方法，在需求分析中涉及的UML技术有：用例图、类图、交互图、状态图、活动图和OCL。
- 对象模型以对象和类的概念为基础，描述了系统中的对象和对象之间的关系。对象是指在一个应用当中具有明确角色的独立可确认的实体；对象之间互相协作的关系称为链接，它描述了对象之间的物理或业务联系。类是将对象进行归类和抽象的结果，类的产生有分类和抽象两种方法，类的封装指封装是指尽可能隐藏构造单位内部的实现细节，只通过有限的对外接口保持对外联系。类之间的关系有关联、聚合与组合、继承和多态。
- 建立领域模型的步骤包括识别候选对象与类、确定概念类、建立类之间的关联、添加类的重要属性，领域模型在需求分析中有着重要作用，尤其是在发现数据方面的需求缺陷与不足。
- UML中的交互图用于描述对象间的协作行为，常见的有顺序图和通信图。顺序图强调消息的时间顺序和对象的交互，通信图则突出对象间的关系与协作。系统顺序图则从系统级别展示外部参与者与系统的交互。

本章小结

- 建立典型场景的系统顺序图的步骤包括确定上下文、找出交互对象、添加消息并描述行为。对于多场景用例，可通过组合片段处理不同的分支和异常情况。交互图有助于发现需求中的缺陷，如行为或数据缺失。
- 状态图起源于有限状态机（FSM），由Harel扩展为SC，解决了FSM的平坦性问题。UML状态图基于SC，增加了事件、转移和层次结构等概念，用于描述复杂系统或对象的行为。
- 状态图是用于描述系统对象在生命周期中的不同状态及其状态之间转移的模型工具。
- OCL用于定义模型中的约束条件，补充图形化模型以提供更加精确的语义。
- CRC方法是面向对象分析在处理复杂问题时的手段，但是它需要了解很多的建模知识才足以进行。
- 需求细化通过层层分解进一步明确用户需求，保证模型的准确性和完整性。